# ARQUITECTURA DE COMPUTADORES

## GRADO DE INGENIERÍA INFORMÁTICA – INGENIERÍA DEL SOFTWARE

## ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

## UNIVERSIDAD DE SEVILLA

**DEPARTAMENTO DE ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES**

http://www.atc.us.es

Autores:
- Raouf Senhadji Navarro
- Ignacio García Vargas

# Material de lectura



- **LECTURAS OBLIGATORIAS:**
  - "*Computer organization and design: the hardware-software interface. RISC-V edition*" David A. Patterson, John L. Hennessy
    - Sección 4.5 (An Overview of Pipelining)
    - Sección 4.6 (Pipelined Datapath and Control)
    - Sección 4.7 (Data Hazards: Forwarding)
    - Sección 4.10 (Parallelism via Instructions)
    - Sección 6.4 (Hardware Multithreading)
  - Como alternativa puede consultarse el siguiente libro: "Estructura y Diseño de Computadores" David A. Patterson, John L. Hennessy
    - Secciones 4.5, 4.6, 4.7 y 4.10
    - Sección 7.5

- **BIBLIOGRAFÍA DE CONSULTA:**


  - John L. Hennessy and David A. Patterson, "Computer Architecture: A Quantitative Approach", 6th edition, Elsevier - Morgan Kaufmann, 2017.
  - "*Estructura y diseño de computadores*", 2ª ed. David A. Patterson, John L. Henness
  - "*Arquitectura de Computadores. Un Enfoque Cuantitativo*" J. L. Hennessy, D. A. Patterson

# Índice

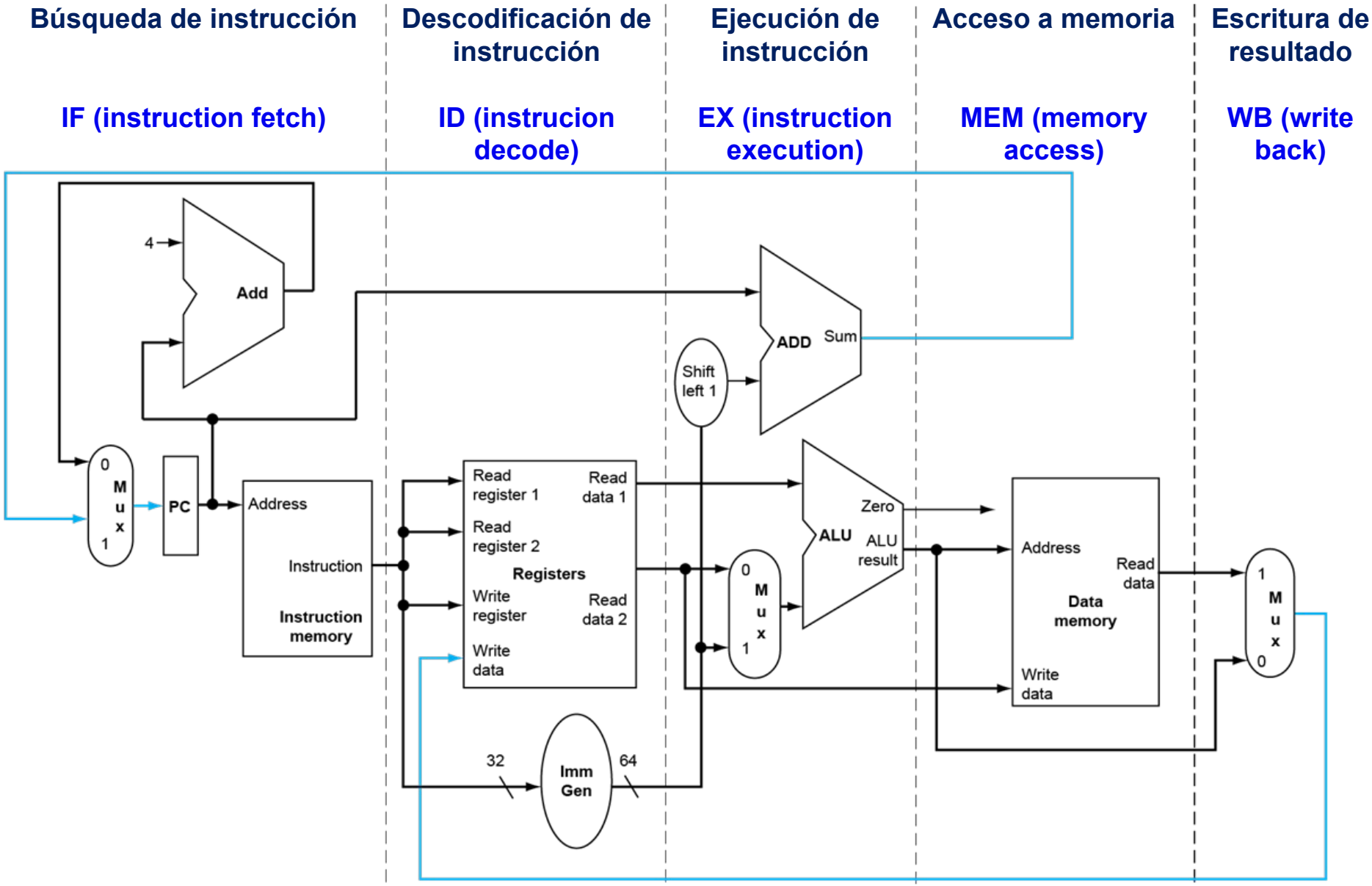


- Introducción a la segmentación de cauce
  - Camino de datos básico del RISC-V
  - Segmentación del RISC-V
- Riesgos y bloqueos
  - Estructurales
  - De datos
  - De control
- Paralelismo a nivel de instrucción avanzado
  - Planificación de instrucciones estática vs. dinámica
  - Estrategias para aumentar las prestaciones
    - Aumentar la profundidad del pipeline
    - Ejecución múltiple de instrucciones
      - Procesadores VLIW
      - Procesadores superescalares
- Ejecución multihilo en hardware

La implementación de la arquitectura RISC-V y las imágenes utilizadas en las diapositivas pertenecen a "*Computer Organization and Design: The Hardware/Software Interface – RISC-V edition*" de David A. Patterson and John L. Hennessy.

# Camino de datos básico del RISC-V





# Camino de datos del RISC-V
# Instrucciones de carga: ejemplo (I)



**lw t1, 100(t5)**

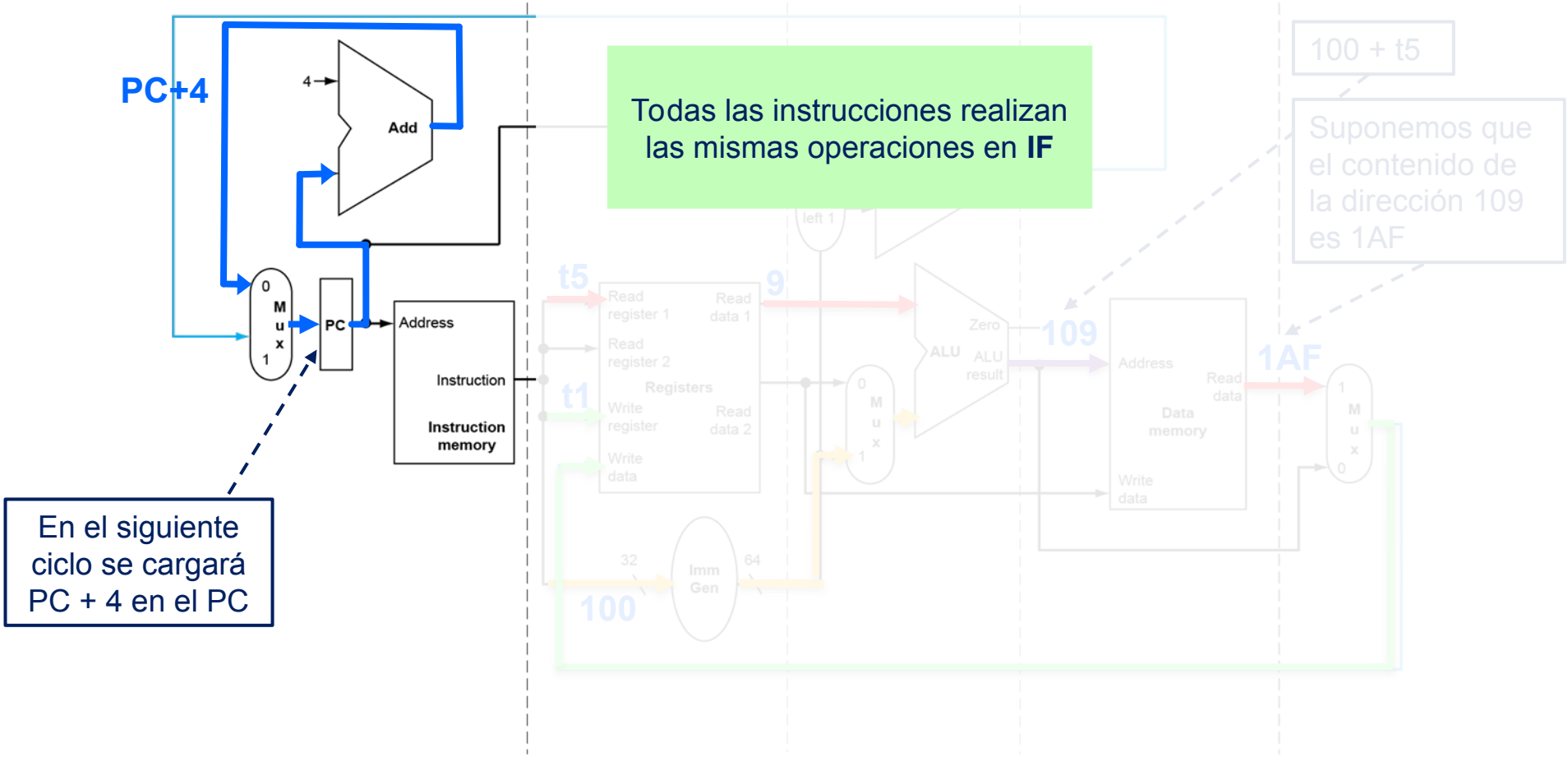
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 100 y el valor de t5

**EX**: Se calcula la dirección efectiva: 100 + t5

**MEM**: Se lee el dato de la dirección efectiva 109

**WB**: Se escribe el valor leído de memoria en t1




En el siguiente ciclo se cargará PC + 4 en el PC

# Camino de datos del RISC-V
# Instrucciones de carga: ejemplo (II)



## lw t1, 100(t5)

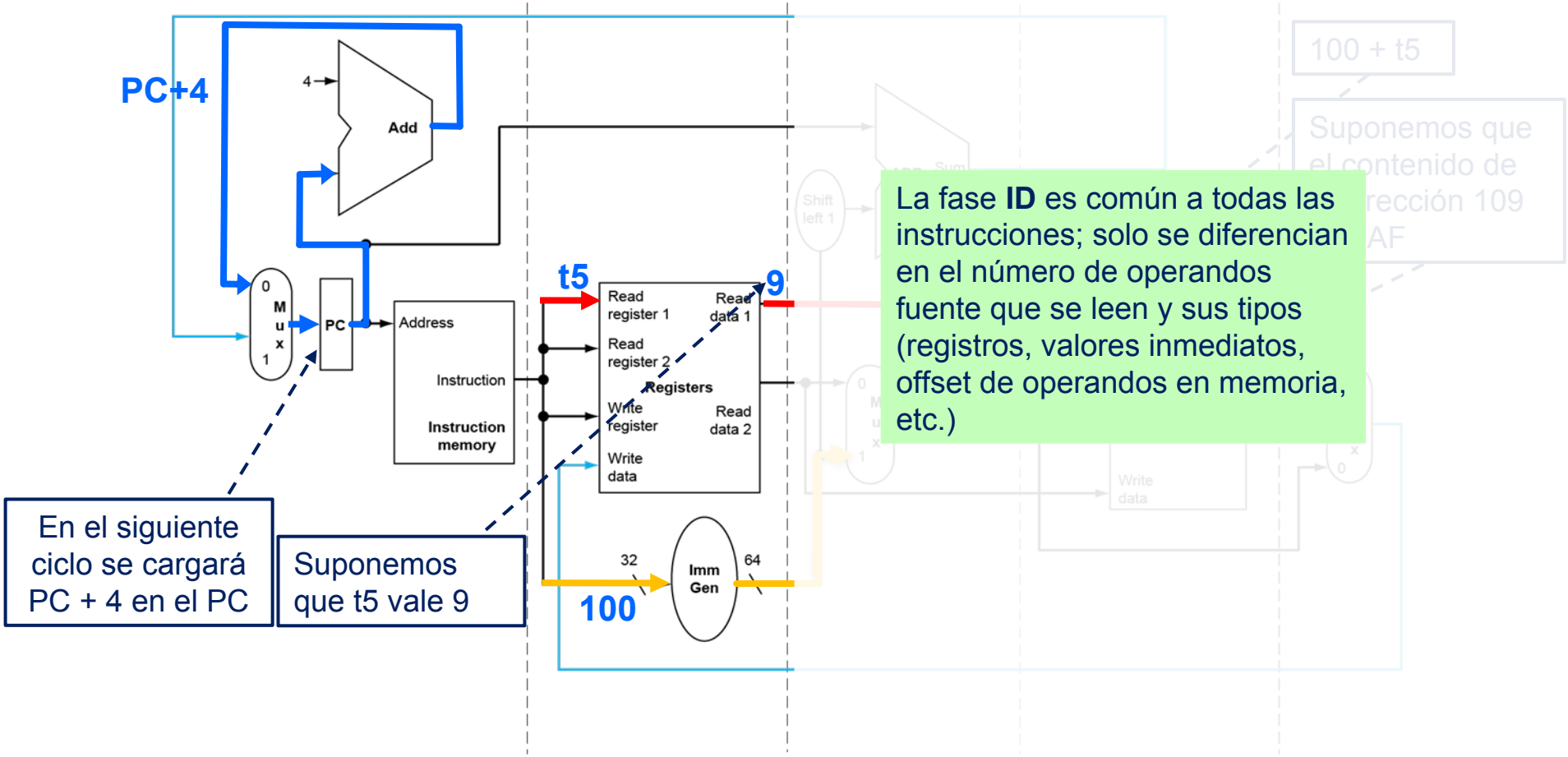
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 100 y el valor de t5

**EX**: Se calcula la dirección efectiva: 100 + t5

**MEM**: Se lee el dato de la dirección efectiva 109

**WB**: Se escribe el valor leído de memoria en t1




La fase **ID** es común a todas las instrucciones; solo se diferencian en el número de operandos fuente que se leen y sus tipos (registros, valores inmediatos, offset de operandos en memoria, etc.)

En el siguiente ciclo se cargará PC + 4 en el PC

Suponemos que t5 vale 9

# Camino de datos del RISC-V
# Instrucciones de carga: ejemplo (III)



## lw t1, 100(t5)

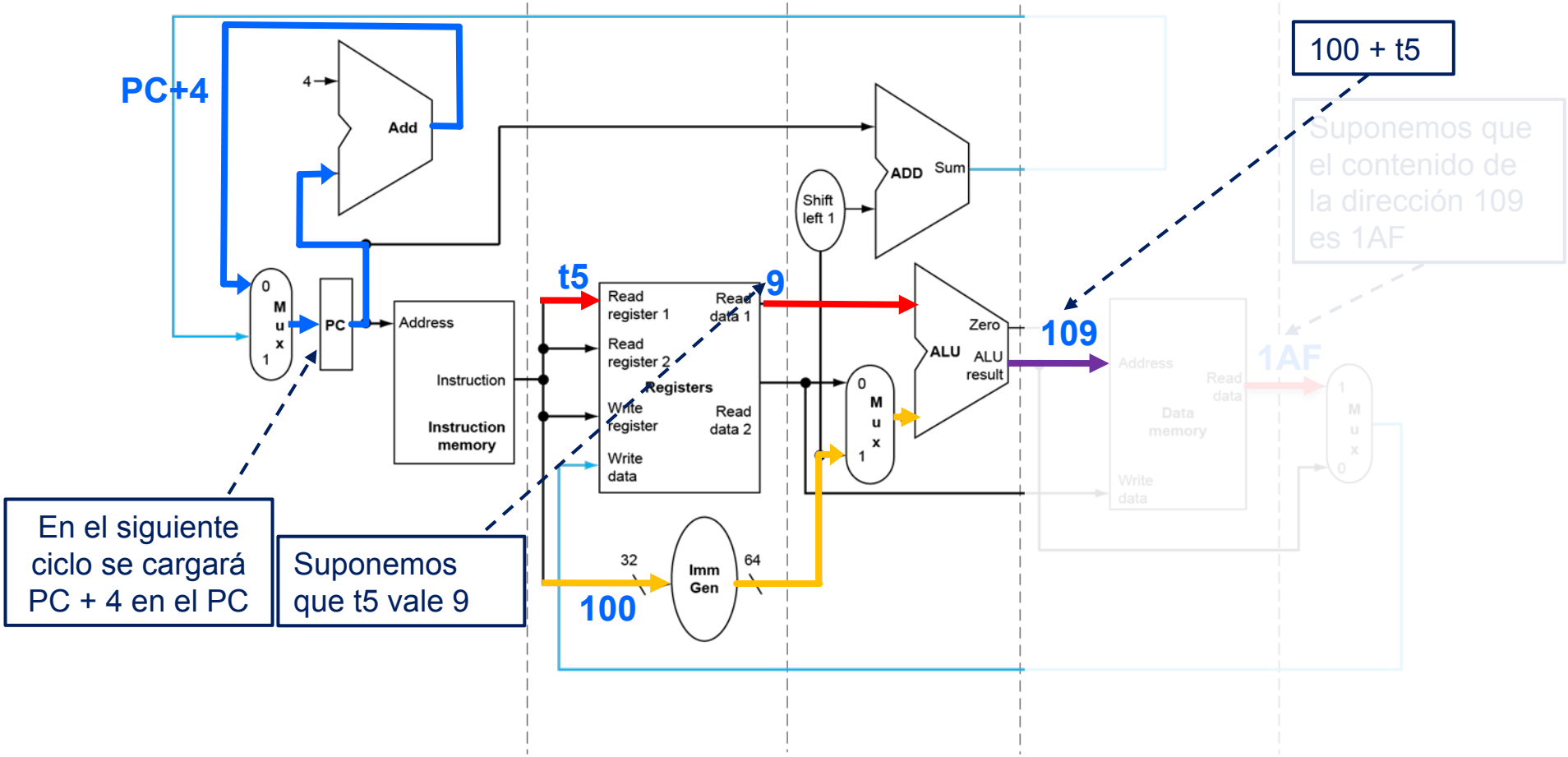
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 100 y el valor de t5

**EX**: Se calcula la dirección efectiva: 100 + t5

**MEM**: Se lee el dato de la dirección efectiva 109

**WB**: Se escribe el valor leído de memoria en t1



# Camino de datos del RISC-V
# Instrucciones de carga: ejemplo (IV)



## lw t1, 100(t5)

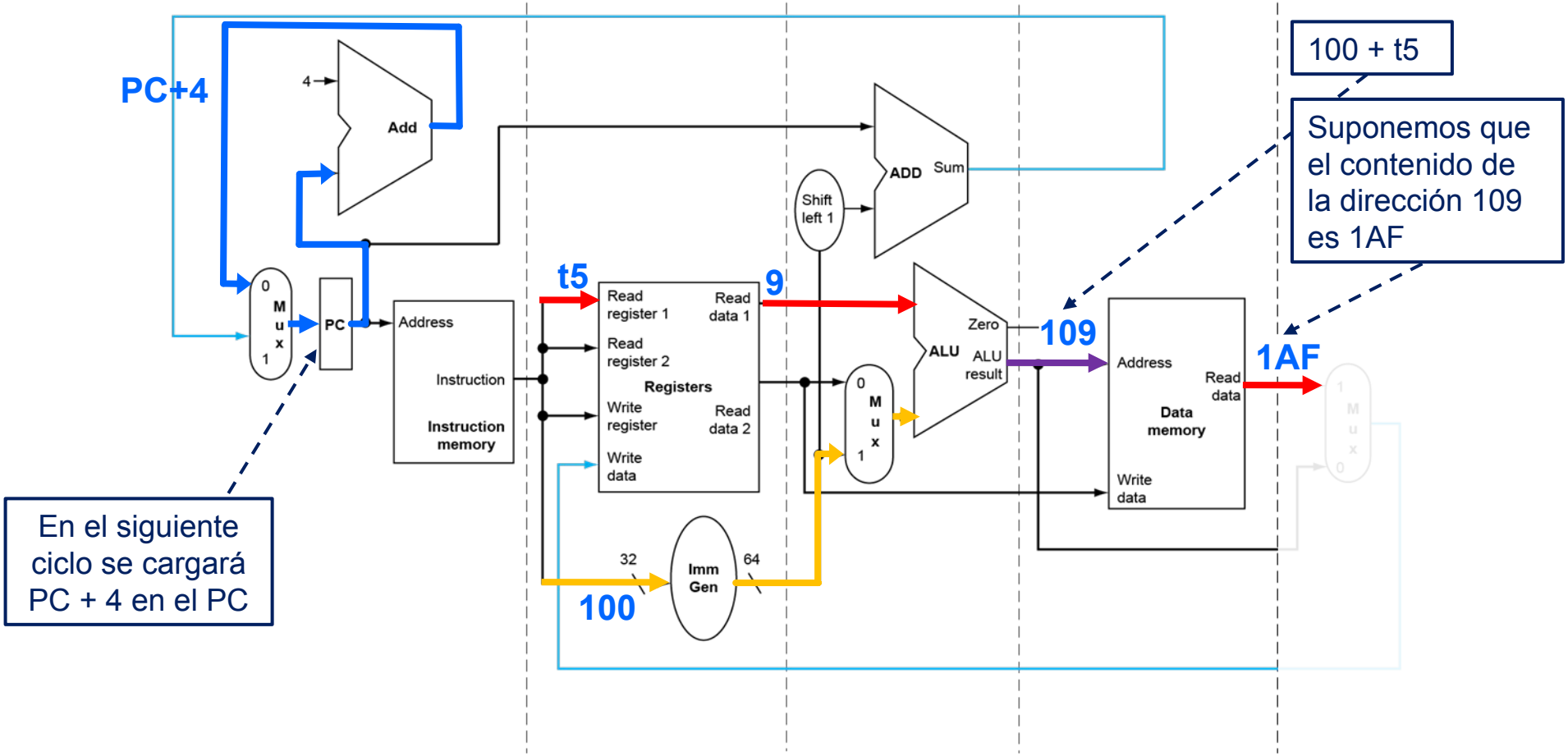
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 100 y el valor de t5

**EX**: Se calcula la dirección efectiva: 100 + t5

**MEM**: Se lee el dato de la dirección efectiva 109

**WB**: Se escribe el valor leído de memoria en t1



# Camino de datos del RISC-V
# Instrucciones de carga: ejemplo (V)



## lw t1, 100(t5)

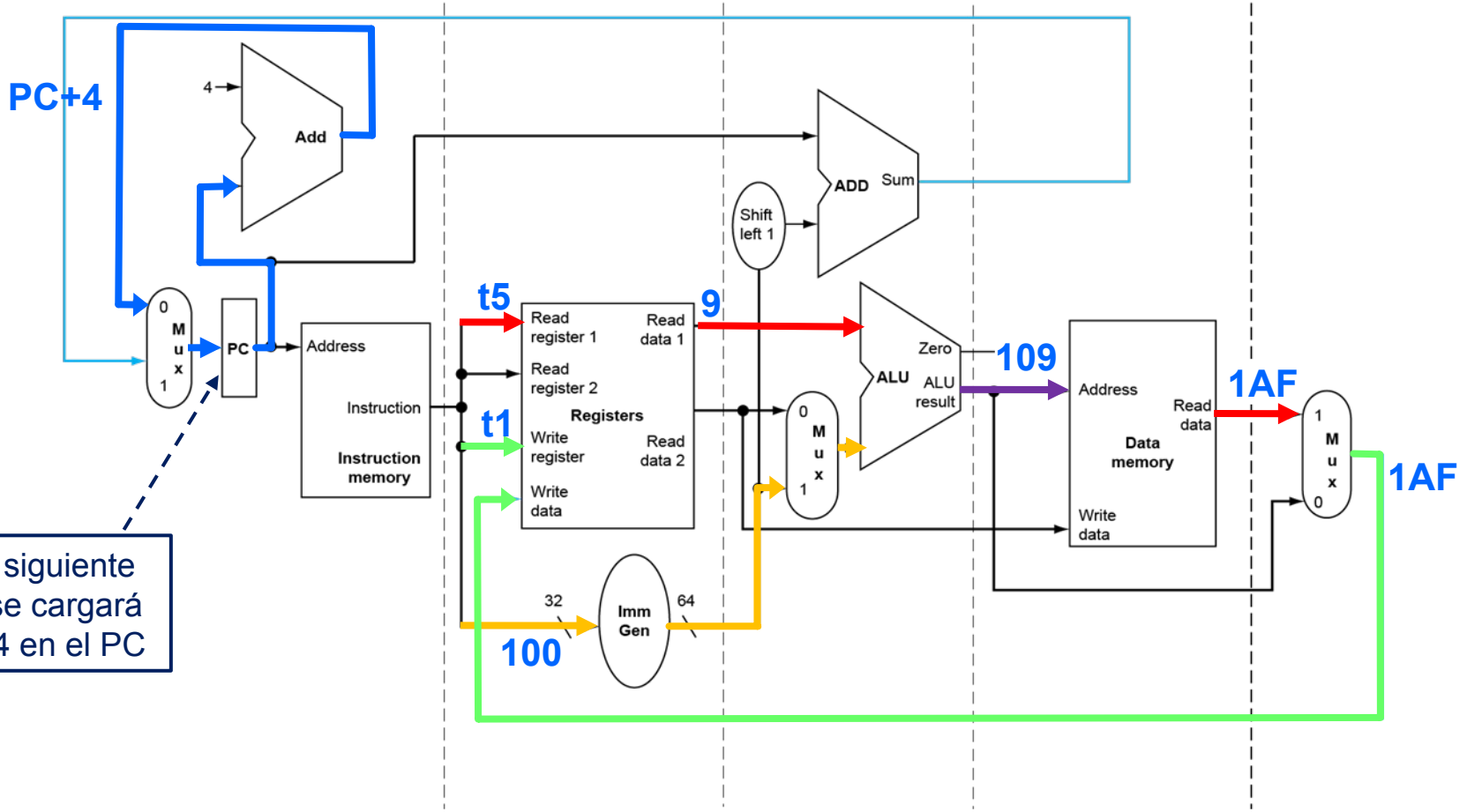
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 100 y el valor de t5

**EX**: Se calcula la dirección efectiva: 100 + t5

**MEM**: Se lee el dato de la dirección efectiva 109

**WB**: Se escribe el valor leído de memoria en t1




En el siguiente ciclo se cargará PC + 4 en el PC

# Camino de datos del RISC-V
# Instrucciones de almacenamiento: ejemplo



**sw t1, 100(t5)**

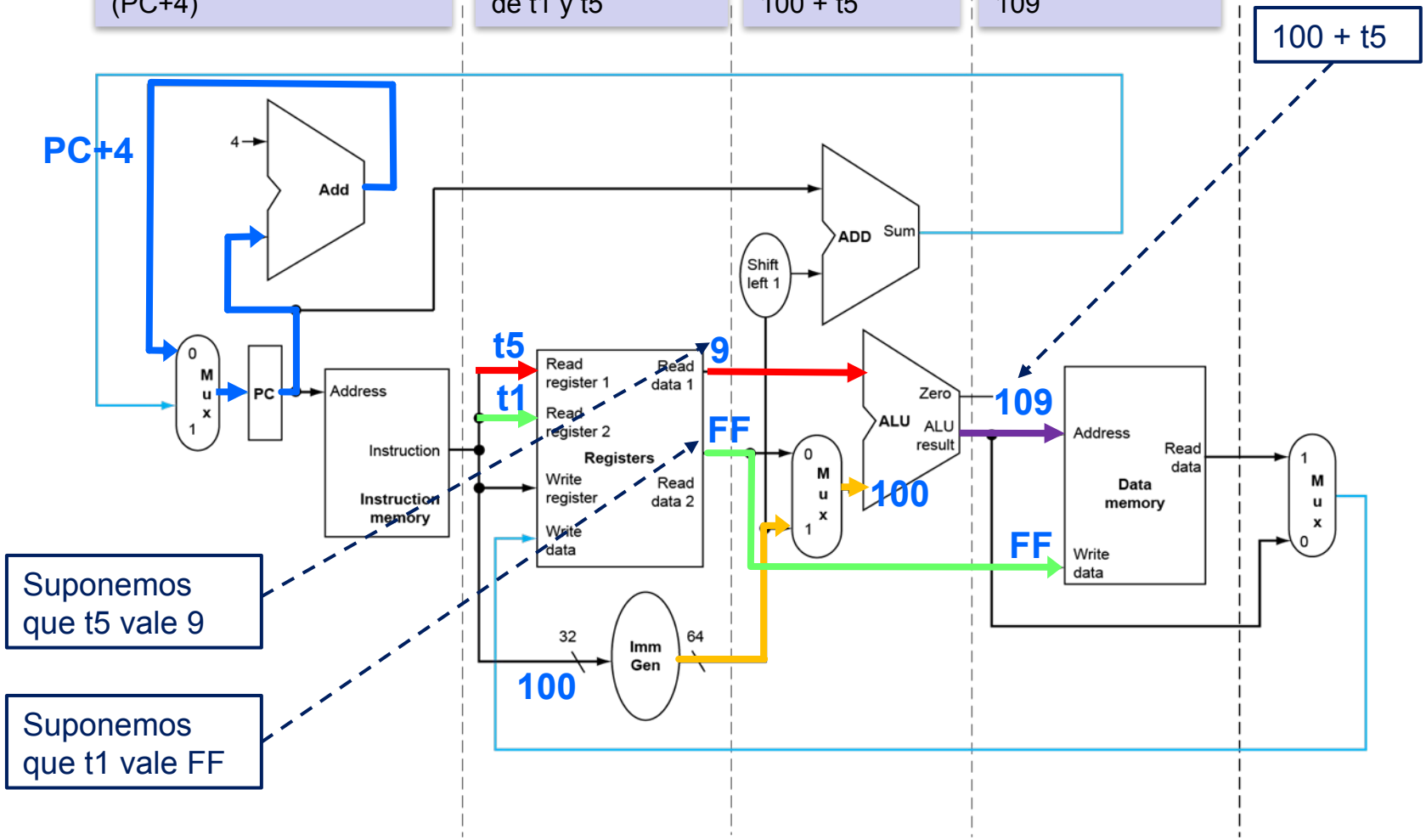
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 100 y los valores de t1 y t5

**EX**: Se calcula la dirección efectiva: 100 + t5

**MEM**: Se escribe el dato en la dirección efectiva 109

No tiene fase WB



# Camino de datos del RISC-V
# Instrucciones ALU: ejemplo



**add t1, t2, t3**

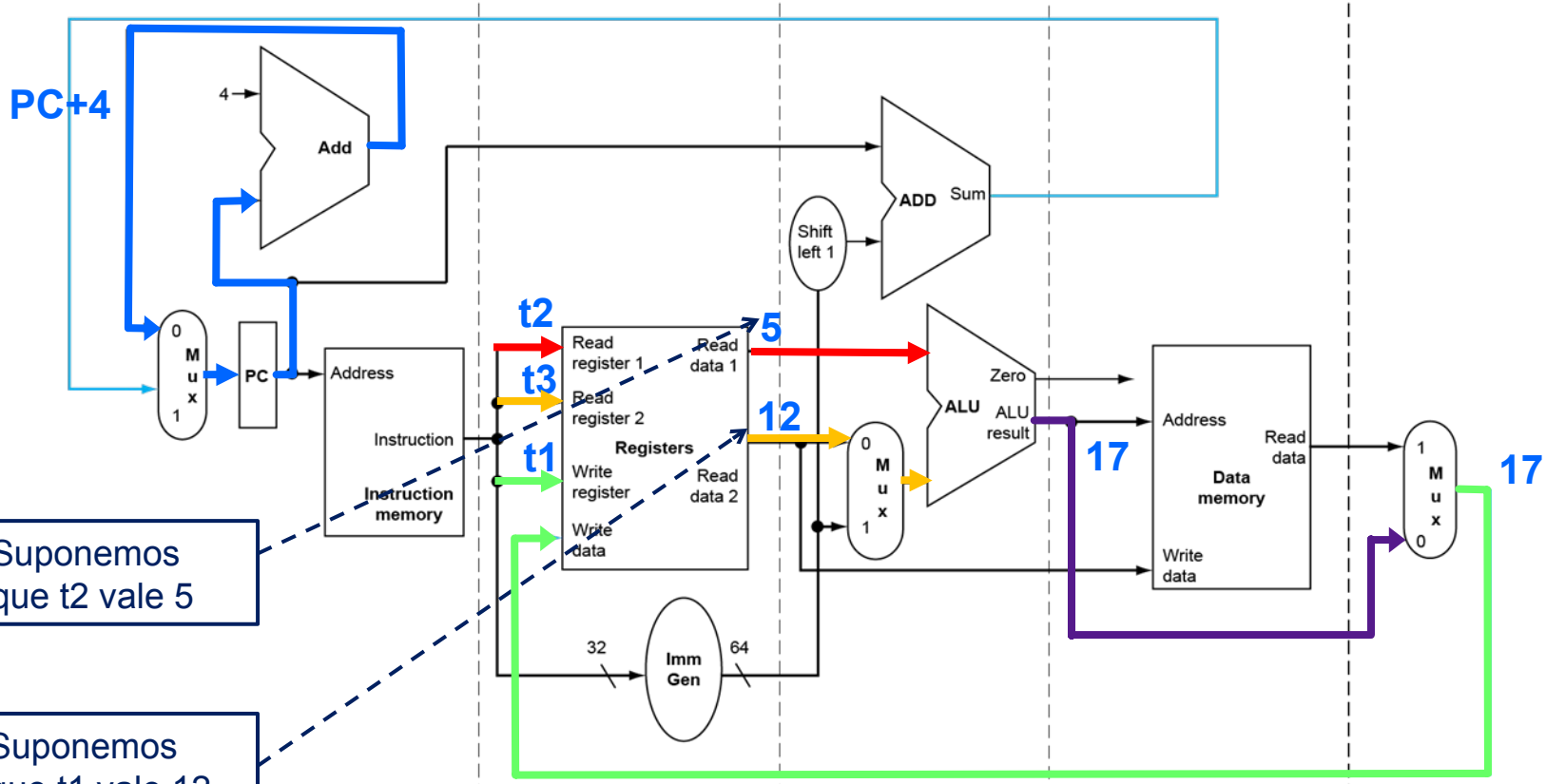
**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: los valores de t2 y t3

**EX**: Se realiza la operación aritmética: t2 + t3

No tiene fase MEM

**WB**: Se escribe el resultado en t1




Suponemos que t2 vale 5

Suponemos que t1 vale 12

# Camino de datos del RISC-V
# Instrucciones de salto: ejemplo



**beq t1, t2, 40**

**IF**: Se lee la instrucción y se calcula la dirección de la instrucción siguiente (PC+4)

**ID**: Se decodifica la instrucción y se obtienen los operandos fuente: 40 y los valor de t1 y t2

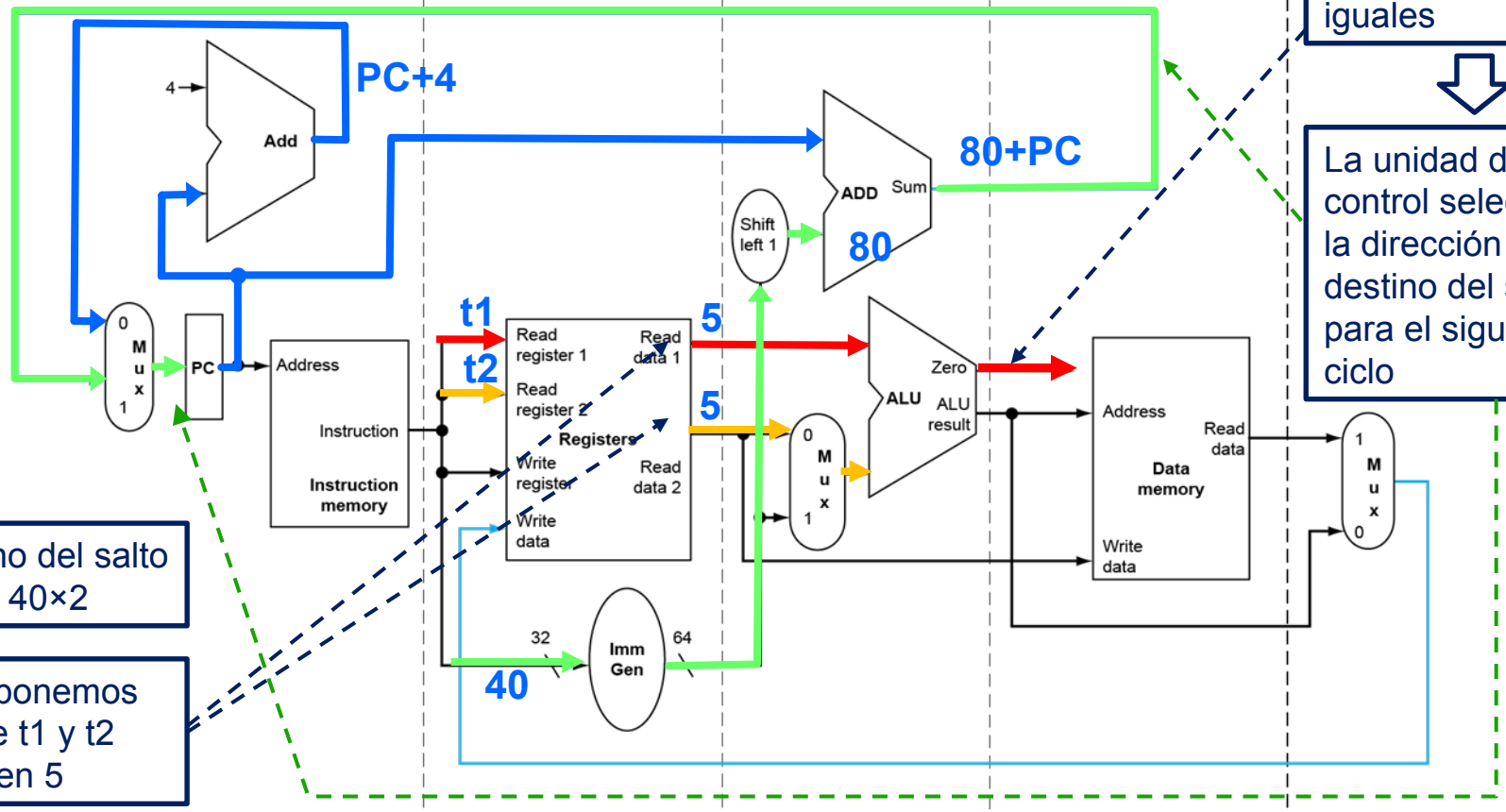
**EX**: Se comparan los valores de t1 y t2. Se calcula el destino del salto

**MEM**: Se actualiza el PC con el destino del salto

No tiene fase WB

Se activa la línea que indica que los registros son iguales

La unidad de control selecciona la dirección destino del salto para el siguiente ciclo



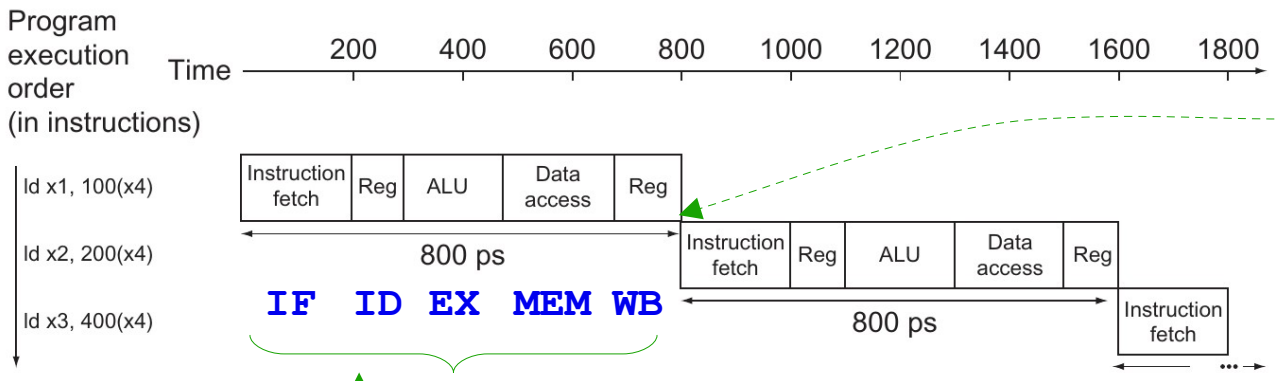

El destino del salto es PC + 40×2

Suponemos que t1 y t2 valen 5

# Introducción a la segmentación de cauce
# Procesadores no segmentados

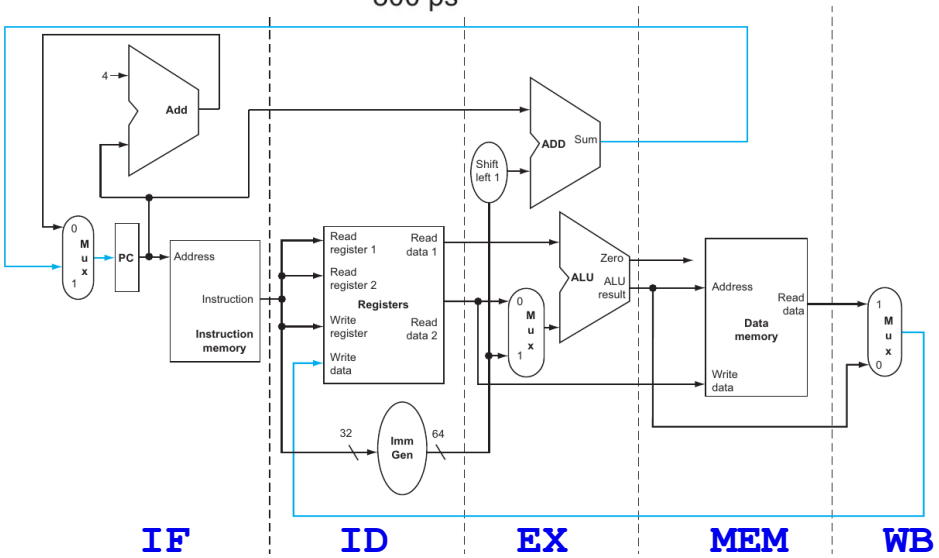


- Un procesador **no** segmentado ejecuta cada instrucción completamente antes de comenzar a ejecutar la siguiente.
  - Por ejemplo, el siguiente diagrama muestra la ejecución de instrucciones en un RISC-V monociclo no segmentado:




Hasta que no se completa una instrucción no comienza la siguiente

La ejecución de una instrucción en el RISC-V requiere 5 fases como máximo



# Introducción a la segmentación de cauce
# Procesadores segmentados

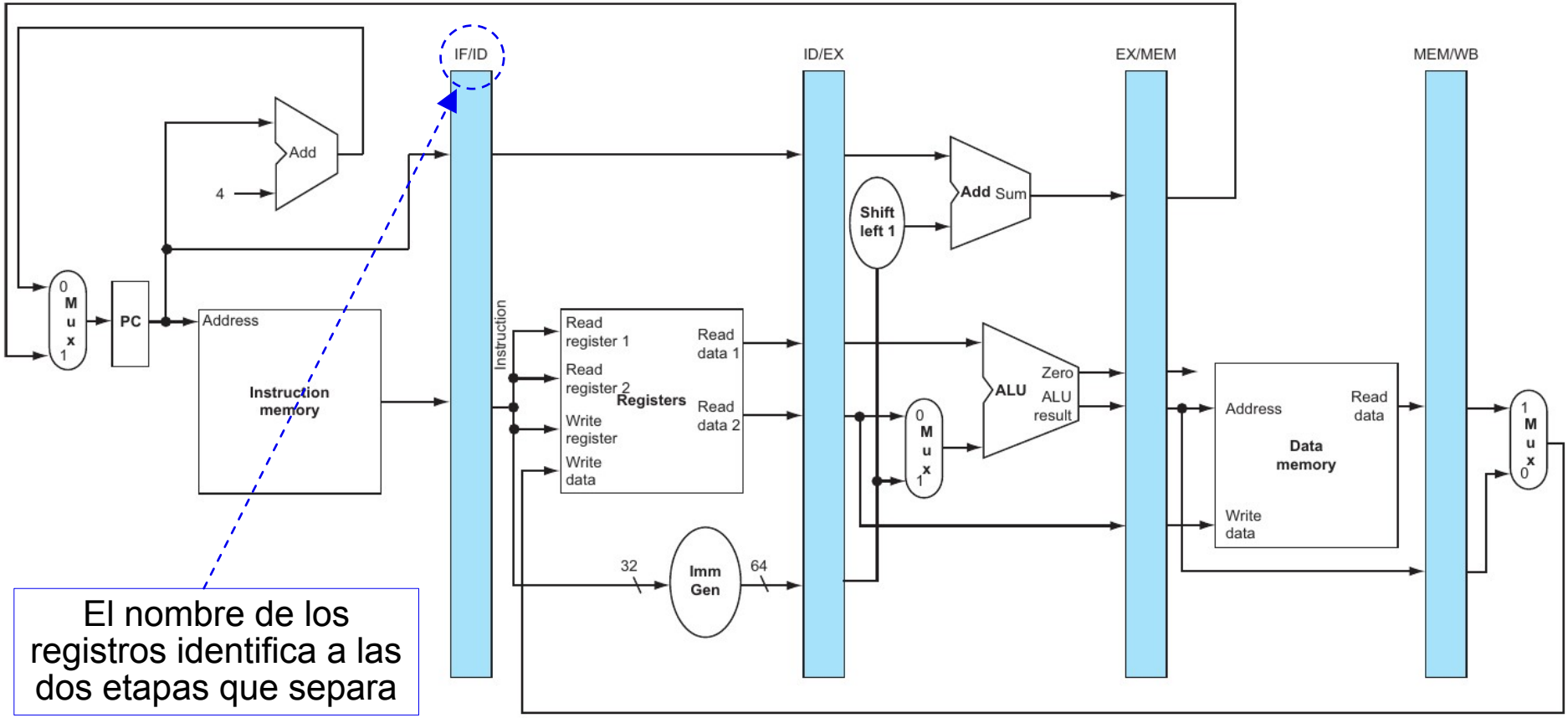


- La segmentación de cauce (*pipelining*) es una técnica de diseño que permite solapar la ejecución de las instrucciones en el camino de datos del procesador.
  - El solapamiento potencial de instrucciones se denomina paralelismo a nivel de instrucción o ILP (*Instruction-Level Parallelism*).
  - La ejecución de una instrucción se divide en etapas (*pipe stages* o *pipe segments*) que realizan parte de las operaciones de la instrucción.
  - Todas las etapas se ejecutan en paralelo, pero cada etapa ejecuta una instrucción diferente.
    - La ejecución de las distintas etapas está sincronizada por un reloj cuyo periodo se denomina ciclo de máquina.
    - Todas las etapas duran un ciclo de máquina.
  - Llamaremos pipeline al camino de datos segmentado.
  - Al número de etapas se le denomina profundidad del pipeline.

# Introducción a la segmentación de cauce
# Segmentación del RISC-V



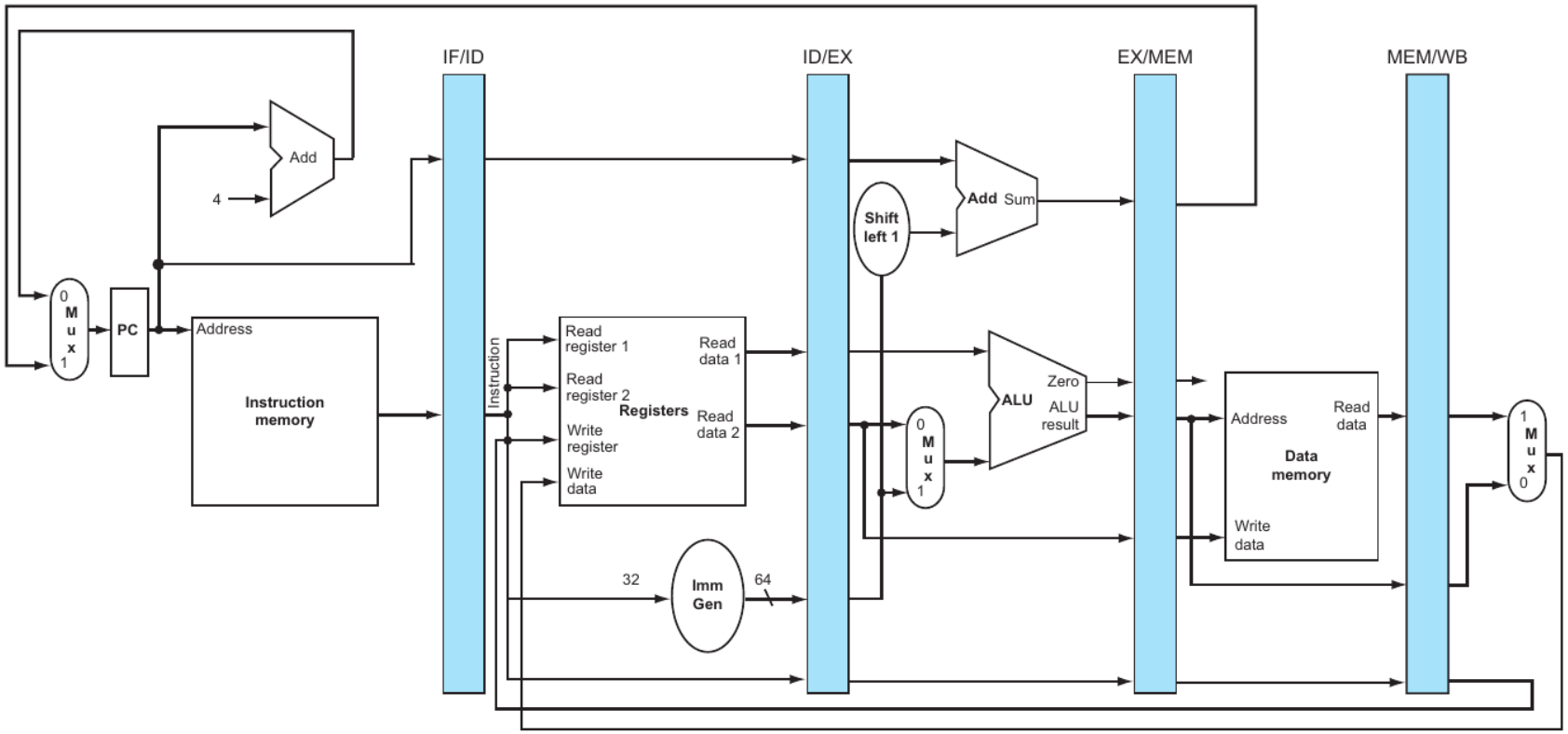
- La segmentación del RISC-V consiste básicamente en añadir registros entre las etapas de segmentación (registros de segmentación).
  - Estos registros permiten que en cada etapa se ejecute una instrucción diferente sin afectar a las etapas adyacentes.




El nombre de los registros identifica a las dos etapas que separa

# Introducción a la segmentación de cauce
# Ejemplo de ejecución en un RISC-V segmentado





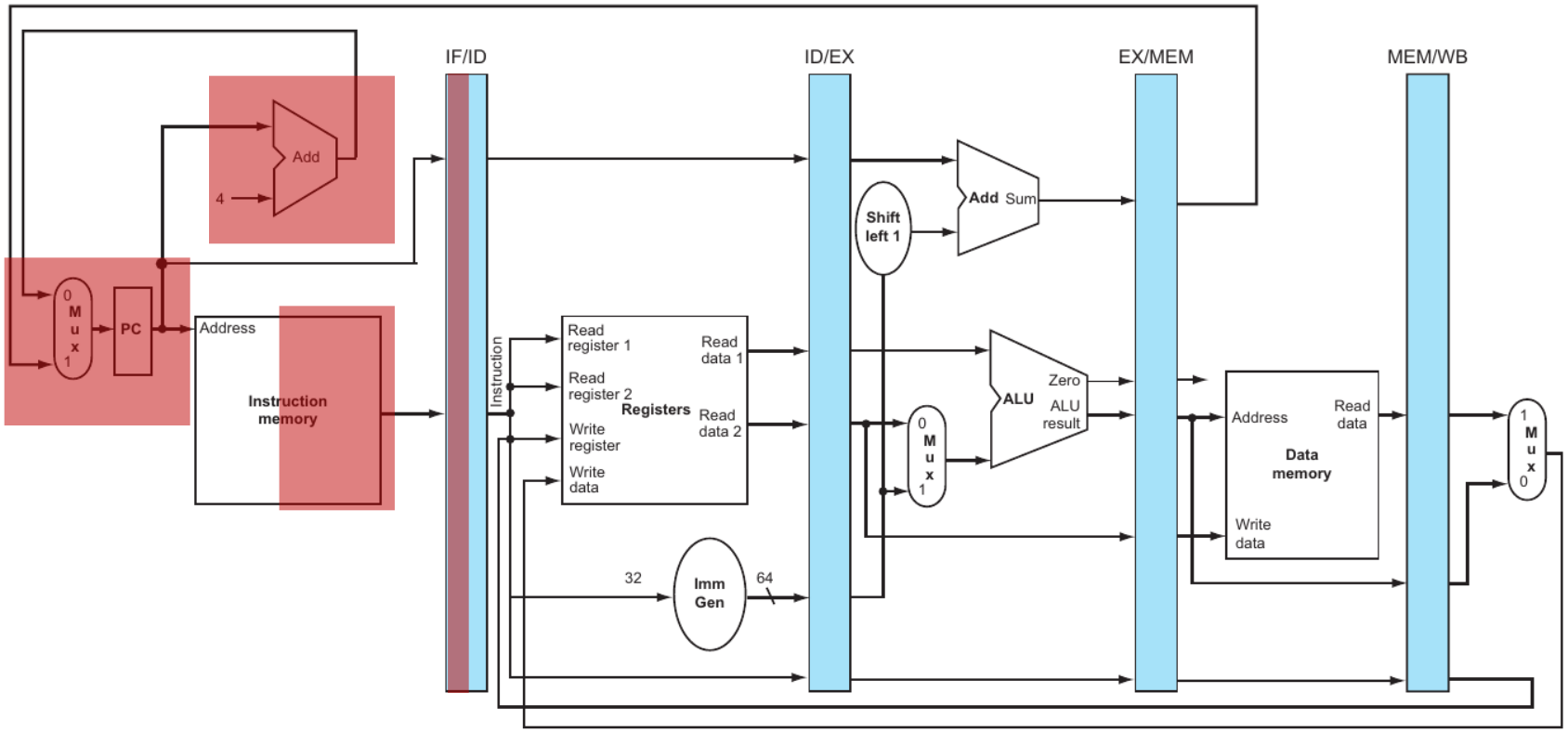

| Instrucciones ejecutadas | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 |
| lw t1, 100(t5) | | | | | | |
| or t7, t7, t5 | | | | | | |
| slt t2, t2, t5 | | | | | | |
| sub t3, t1, t5 | | | | | | |
| sw t1, 4(t6) | | | | | | |

**Diagrama del pipeline**:
muestra la evolución de las instrucciones en el pipeline

# Introducción a la segmentación de cauce
# Ejemplo de ejecución en un RISC-V segmentado



**lw t1, 100(t5)**

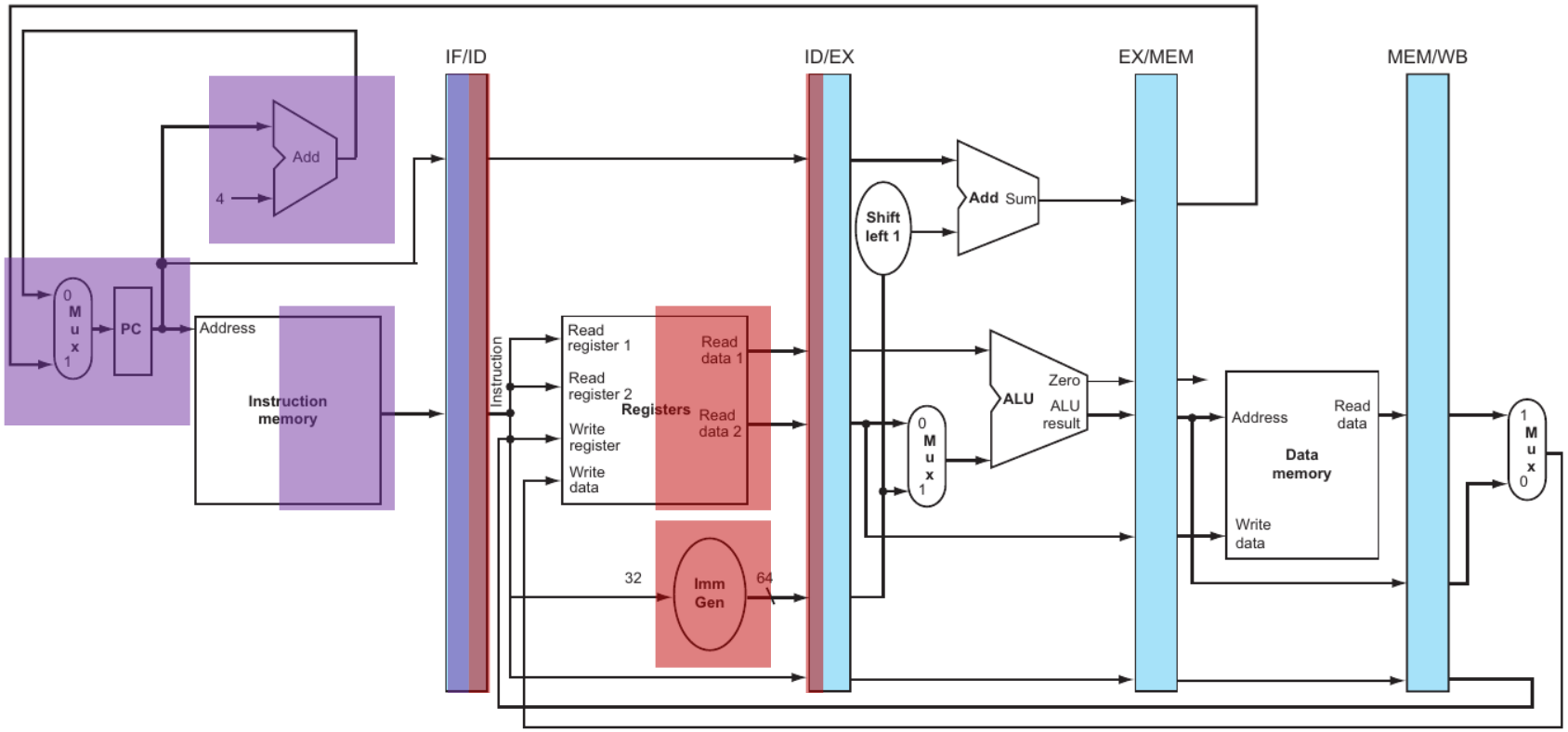



| Instrucciones ejecutadas | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 |
| lw t1, 100(t5) | IF | | | | | |
| or t7, t7,t5 | | | | | | |
| slt t2,t2,t5 | | | | | | |
| sub t3,t1,t5 | | | | | | |
| sw t1, 4(t6) | | | | | | |

# Introducción a la segmentación de cauce
# Ejemplo de ejecución en un RISC-V segmentado



**or t7, t7,t5** **lw t1, 100(t5)**

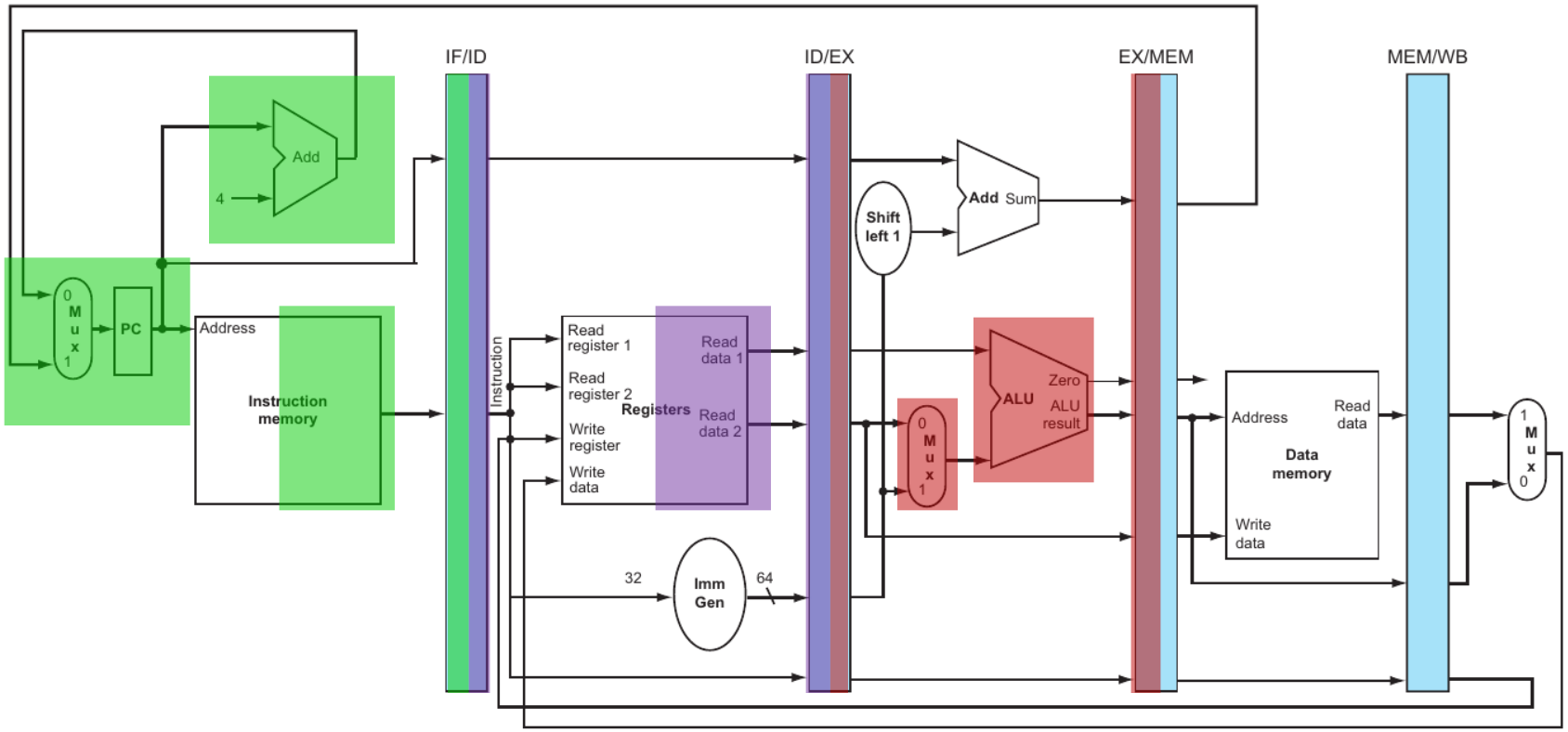



| Instrucciones ejecutadas | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | **2** | 3 | 4 | 5 | 6 |
| lw t1, 100(t5) | IF | **ID** | | | | |
| ➡ **or t7, t7,t5** | | **IF** | | | | |
| slt t2,t2,t5 | | | | | | |
| sub t3,t1,t5 | | | | | | |
| sw t1, 4(t6) | | | | | | |

# Introducción a la segmentación de cauce
# Ejemplo de ejecución en un RISC-V segmentado



**slt t2,t2,t5** **or t7, t7,t5** **lw t1, 100(t5)**

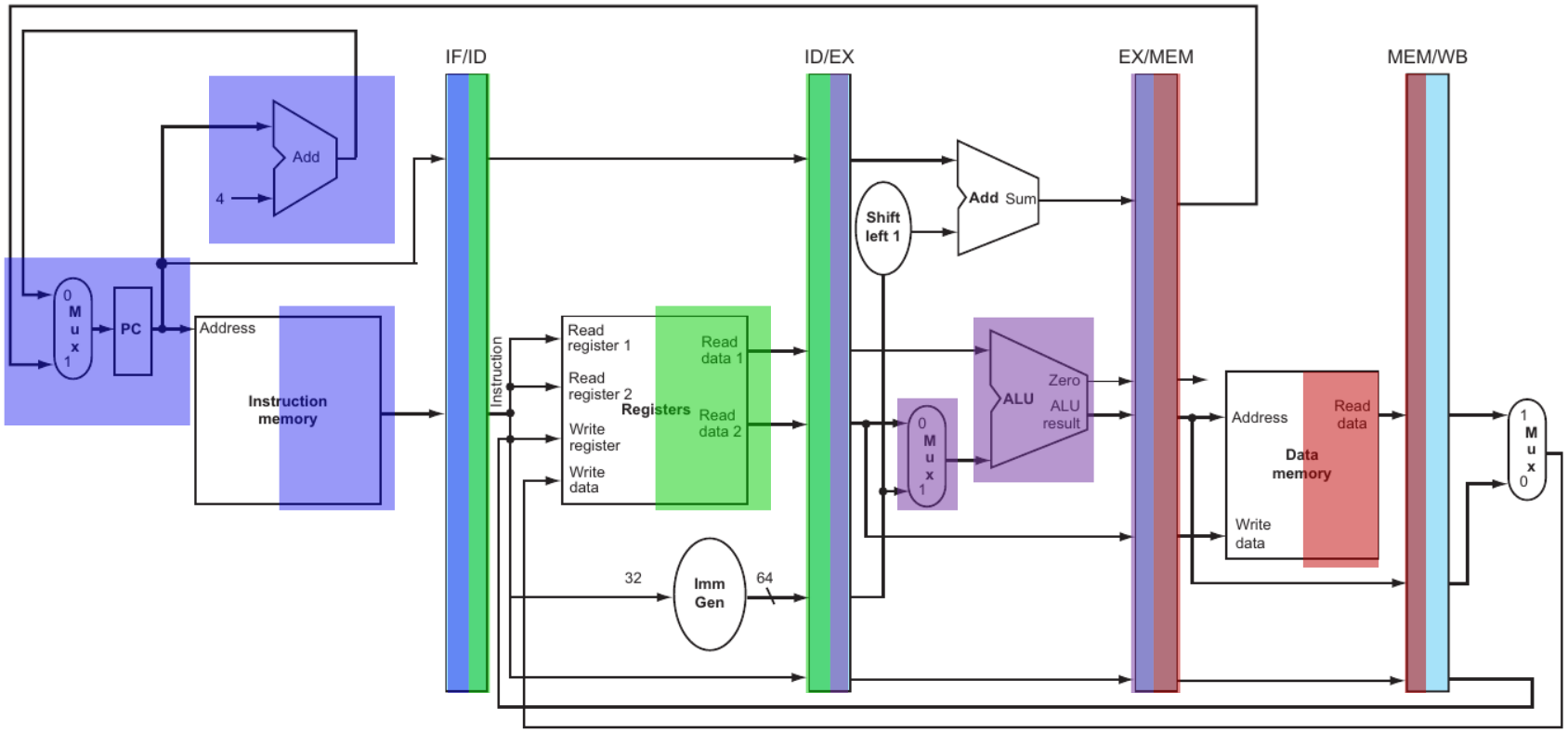



| Instrucciones ejecutadas | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | **3** | 4 | 5 | 6 |
| lw t1, 100(t5) | IF | ID | **EX** | | | |
| or t7, t7,t5 | | IF | **ID** | | | |
| **slt t2,t2,t5** | | | **IF** | | | |
| sub t3,t1,t5 | | | | | | |
| sw t1, 4(t6) | | | | | | |

# Introducción a la segmentación de cauce
# Ejemplo de ejecución en un RISC-V segmentado



**sub t3,t1,t5** **slt t2,t2,t5** **or t7, t7,t5** **lw t1, 100(t5)**

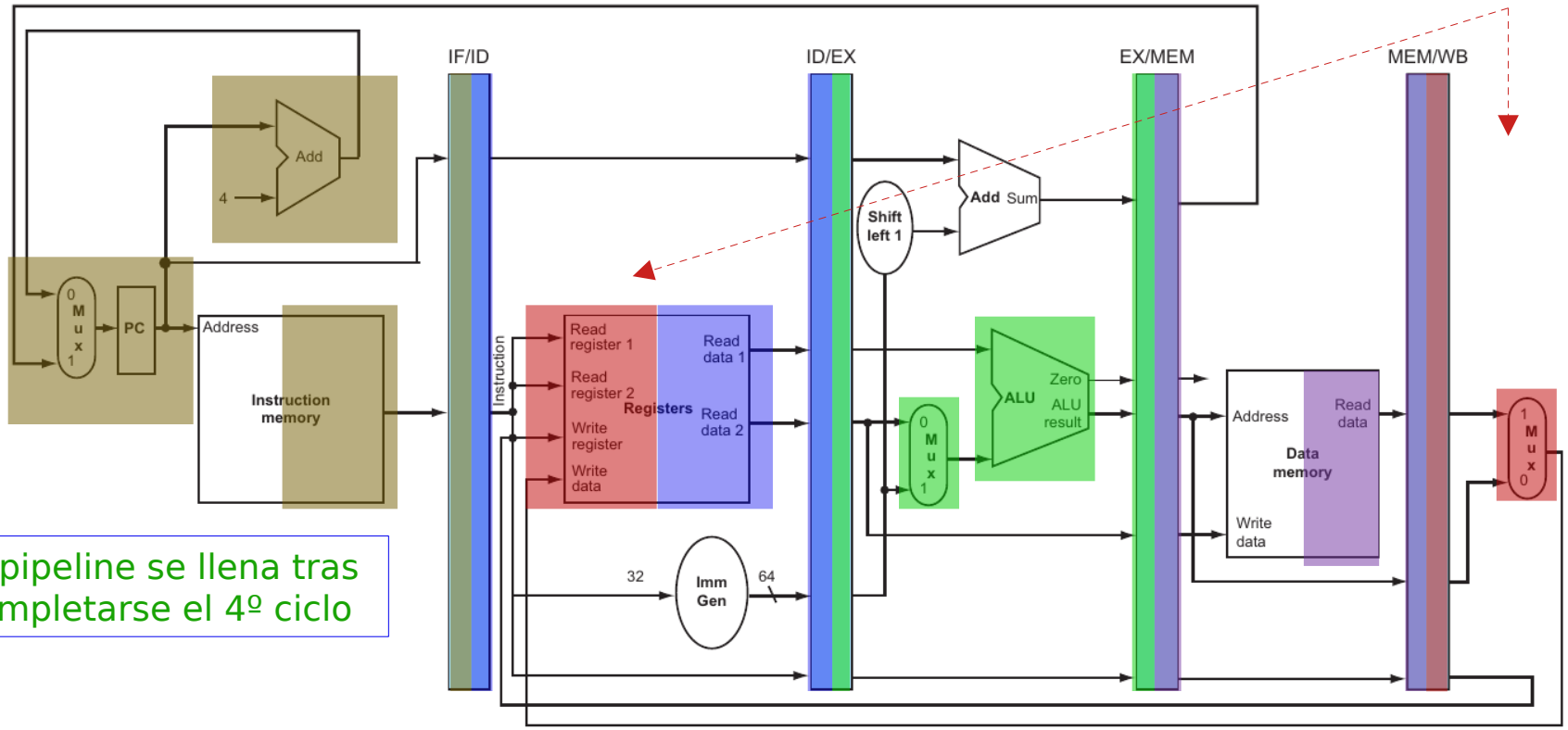



| Instrucciones ejecutadas | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | **4** | 5 | 6 |
| lw t1, 100(t5) | IF | ID | EX | **MEM** | | |
| or t7, t7,t5 | | IF | ID | **EX** | | |
| slt t2,t2,t5 | | | IF | **ID** | | |
| **sub t3,t1,t5** | | | | **IF** | | |
| sw t8, 4(t6) | | | | | | |

# Introducción a la segmentación de cauce
# Ejemplo de ejecución en un RISC-V segmentado



**sw t8, 4(t6)** **sub t3,t1,t5** **slt t2,t2,t5** **or t7, t7,t5** **lw t1, 100(t5)**




El pipeline se llena tras completarse el 4º ciclo

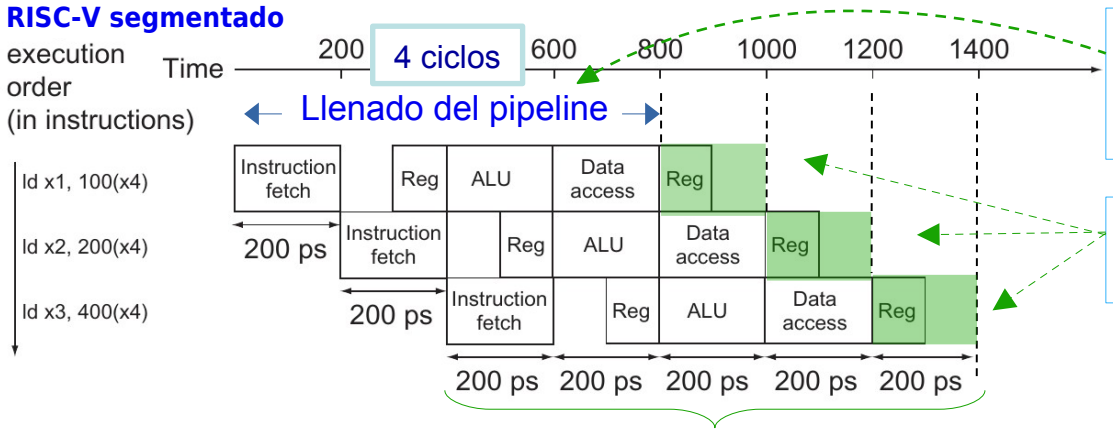
| Instrucciones ejecutadas | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | **5** | 6 |
| lw t1, 100(t5) | IF | ID | EX | MEM | **WB** | |
| or t7, t7,t5 | | IF | ID | EX | **MEM** | |
| slt t2,t2,t5 | | | IF | ID | **EX** | |
| sub t3,t1,t5 | | | | IF | **ID** | |
| **sw t8, 4(t6)** | | | | | **IF** | |

5 instrucciones ejecutándose en paralelo

# Introducción a la segmentación de cauce
# Propiedades: rendimiento



- La segmentación de cauce aumenta el rendimiento (medido como *instrucciones / segundo*).
  - Ejemplo: Cálculo del rendimiento en el RISC-V segmentado

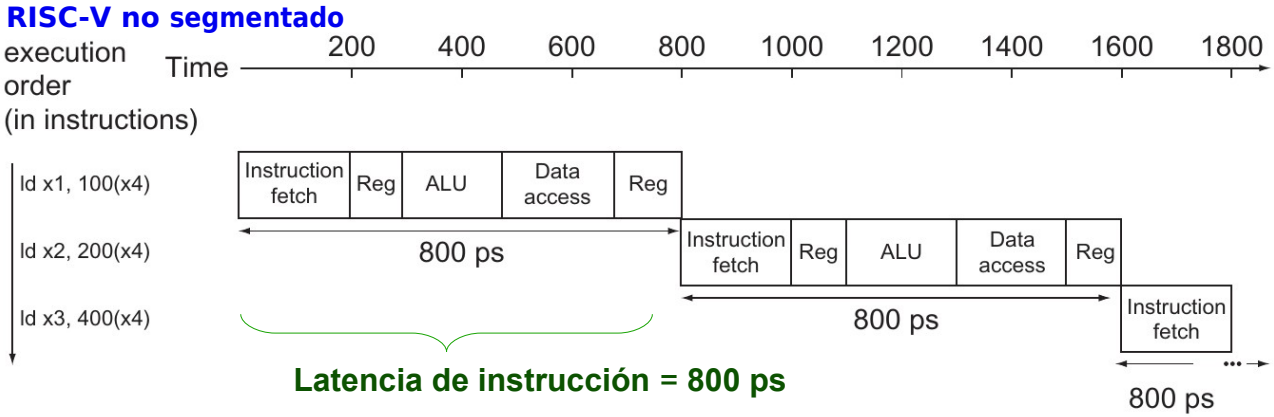



N.º de ciclos necesario para que las primeras instrucciones llenen el pipeline (es la profundidad – 1). En RISC-V son 4 ciclos.

A partir del llenado, se completa una instrucción cada ciclo de procesador.

Rendimiento ≈ 1 instr. / 200 ps = $5 \times 10^9$ instr/seg.

Los 4 ciclos del llenado se pueden despreciar frente a los millones de ciclos de ejecución de un programa.




Rendimiento = 1 instr. / 800 ps = $1.25 \times 10^9$ instr/seg.

- Sin embargo, también aumenta la latencia de instrucción (es decir, el tiempo de ejecución de cada instrucción individual).

# Introducción a la segmentación de cauce
# Propiedades: aceleración ideal



- En condiciones ideales la aceleración conseguida con la segmentación es igual al número de etapas (esta es la máxima aceleración que se podría alcanzar).
  - Suponiendo que todas las etapas están equilibradas, la duración del ciclo del procesador segmentado ($T_s$) sería:

$$T_s = \frac{T_{ns}}{n}$$

Periodo de reloj del procesador monociclo no segmentado.

Profundidad del pipeline.

    - Por lo tanto, la aceleración sería:

Núm. instr. ejecutadas

$$a = \frac{t_{ns}}{t_s} = \frac{N_{inst} \cdot T_{ns}}{N_{ciclos} \cdot T_s} \approx \frac{N_{inst} \cdot T_{ns}}{N_{inst} \cdot T_s} = \frac{T_{ns}}{T_s} = \frac{T_s \cdot n}{T_s} = n$$

Núm. de ciclos

Se desprecian los ciclos de llenado del pipeline

- Ejemplo: RISC-V segmentado (diapositiva anterior)
  - La aceleración ideal es 5.
  - Sin embargo, las etapas no están equilibradas; por lo tanto, la aceleración es

$$\frac{T_{ns}}{T_s} = \frac{800\ ps}{200\ ps} = 4$$

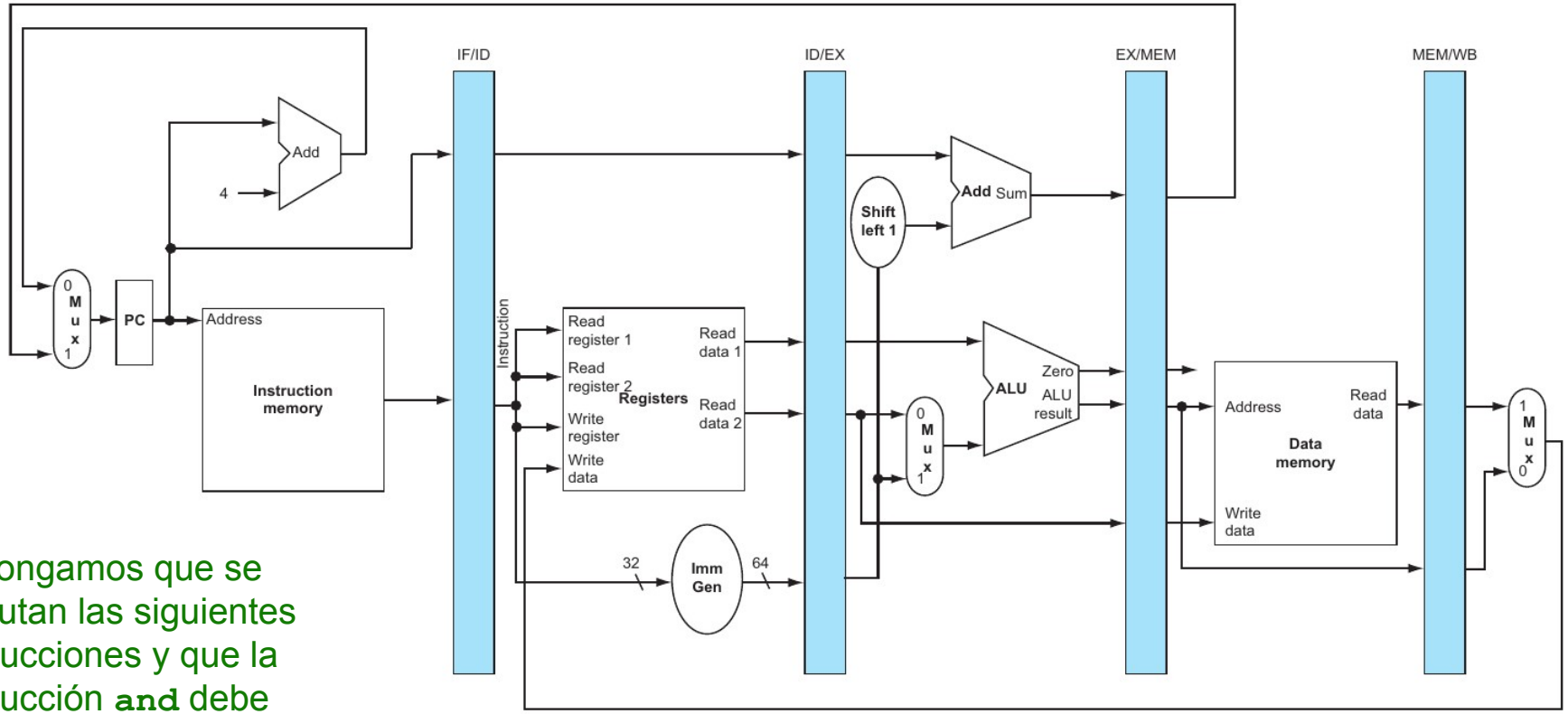
# Riesgos y bloqueos



- Un riesgo (*hazard*) es una situación que impide ejecutar con normalidad alguna etapa de una instrucción.
- Según el motivo que lo cause, se clasifican como:
  - Riesgos estructurales
  - Riesgos de datos
  - Riesgos de control
- Una posibilidad para evitar un riesgo es provocar un bloqueo (*stall*) que retrase la ejecución de la etapa conflictiva.
  - Este retraso afecta a la instrucción que produce el riesgo y a todas las que se encuentran en etapas anteriores (que comenzaron su ejecución posteriormente).
  - Los bloqueos producen un impacto significativo en el rendimiento, por lo que existen numerosas técnicas software y hardware para reducir su número.
  - En el RISC-V, todos los riesgos pueden ser detectados en la etapa ID.

# Ejemplo de bloqueo en el RISC-V





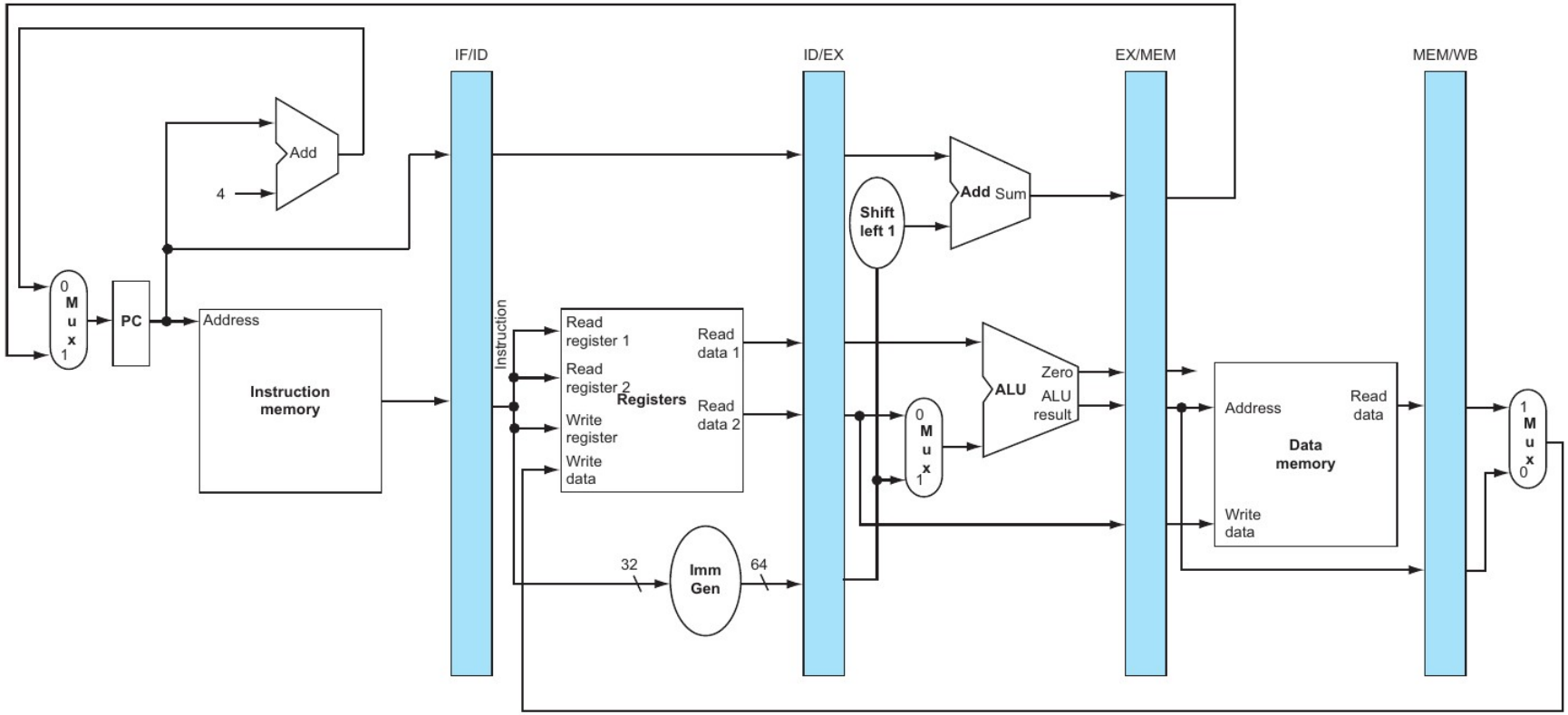

Supongamos que se ejecutan las siguientes instrucciones y que la instrucción **`and`** debe bloquearse

| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 |
| lw t2, 20(t1) | | | | | | |
| and t4, t2, t5 | | | | | | |
| or t8, t2, t6 | | | | | | |
| and t9,t4,t2 | | | | | | |

# Ejemplo de bloqueo en el RISC-V



lw t2, 20(t1)

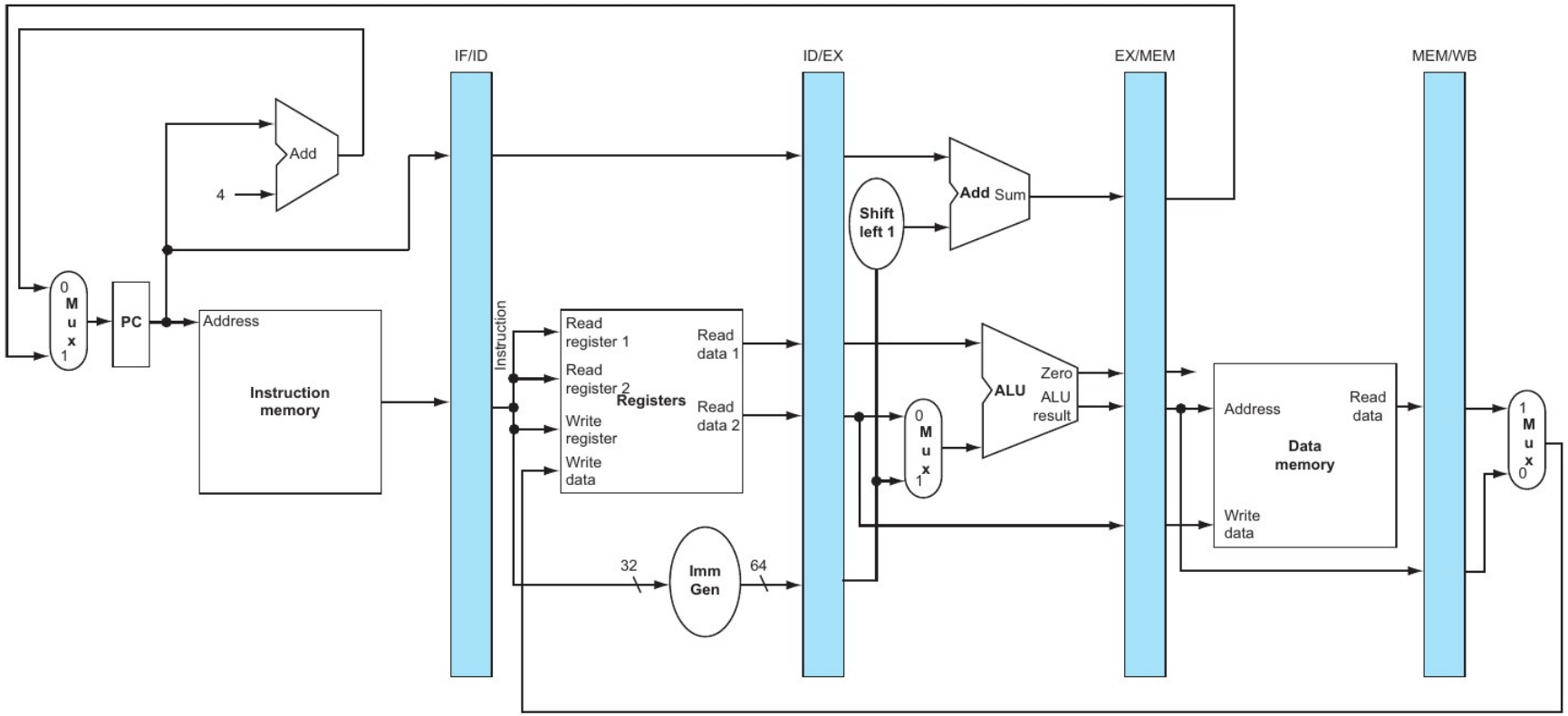



| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | **1** | 2 | 3 | 4 | 5 | 6 |
| lw t2, 20(t1) | **IF** | | | | | |
| and t4, t2, t5 | | | | | | |
| or t8, t2, t6 | | | | | | |
| and t9,t4,t2 | | | | | | |

# Ejemplo de bloqueo en el RISC-V



and t4, t2, t5 lw t2, 20(t1)

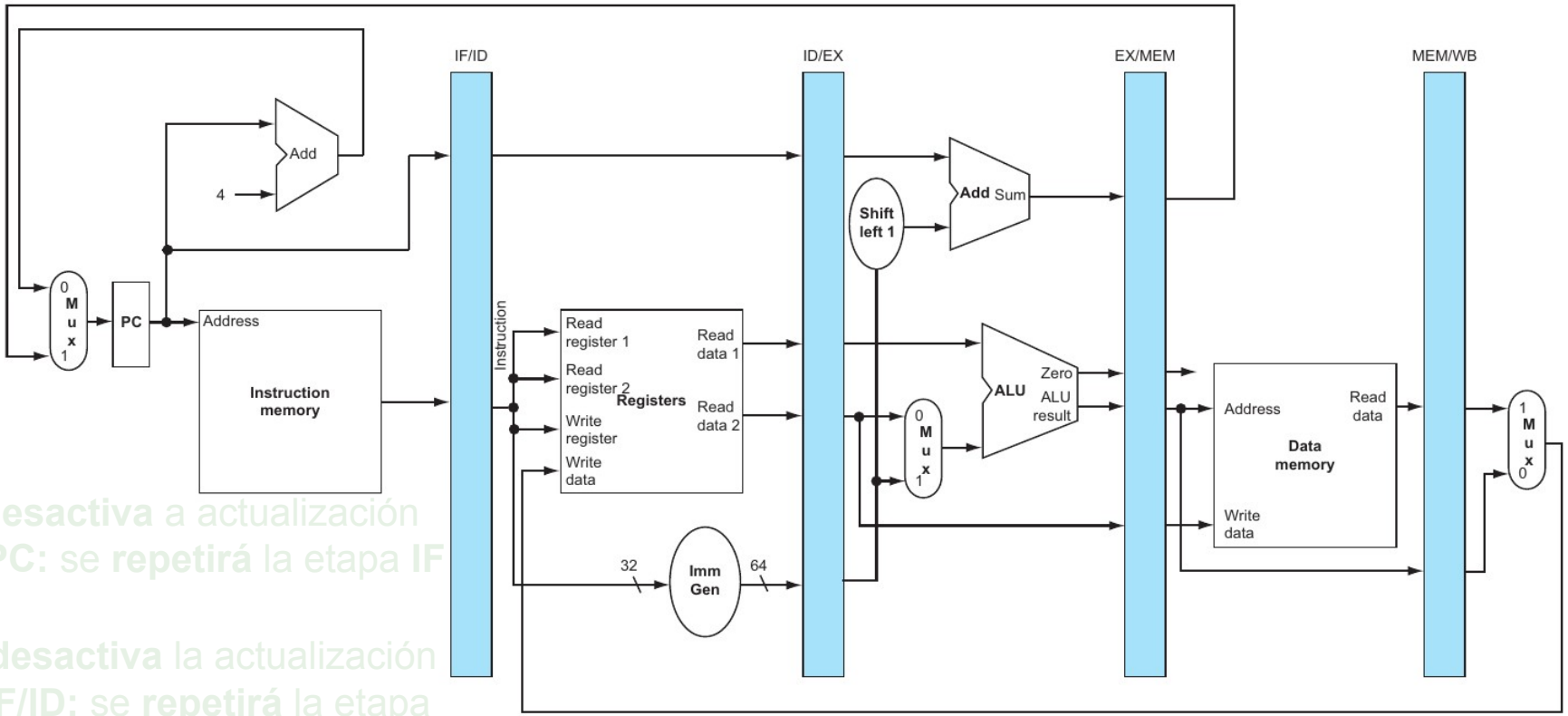



| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | **2** | 3 | 4 | 5 | 6 |
| lw t2, 20(t1) | IF | **ID** | | | | |
| and t4, t2, t5 | | **IF** | | | | |
| or t8, t2, t6 | | | | | | |
| and t9,t4,t2 | | | | | | |

# Ejemplo de bloqueo en el RISC-V



or t8, t2, t6 and t4, t2, t5 lw t2, 20(t1)




Se **desactiva** a actualización del **PC:** se **repetirá** la etapa **IF**

Se **desactiva** la actualización de **IF/ID:** se **repetirá** la etapa **ID)**

La instrucción **and** debe bloquearse

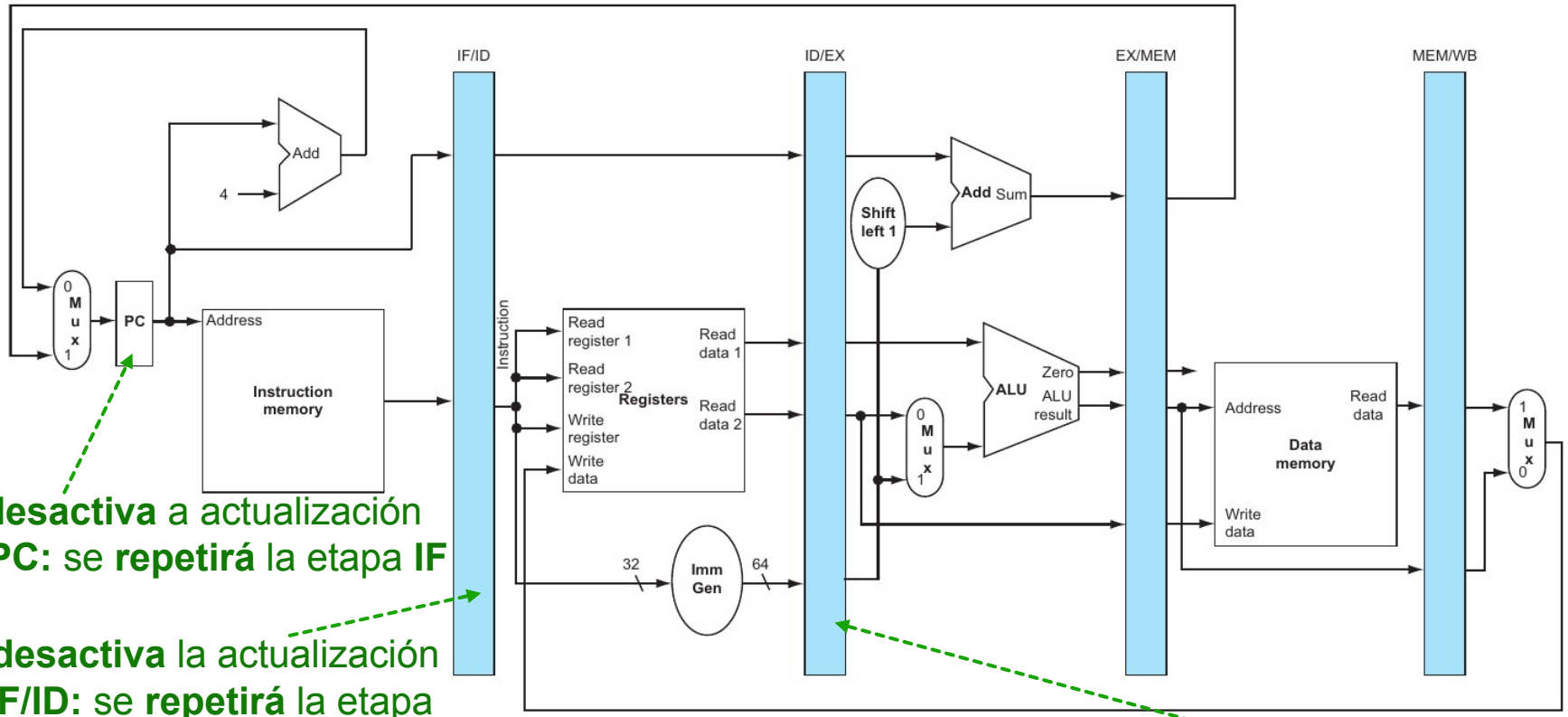
| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | **3** | 4 | 5 | 6 |
| lw t2, 20(t1) | IF | ID | **EX** | | | |
| **and t4, t2, t5** | | IF | **ID** | | | |
| or t8, t2, t6 | | | **IF** | | | |
| and t9,t4,t2 | | | | | | |

En el siguiente ciclo se **inserta** una **nop** en ID/EX

# Ejemplo de bloqueo en el RISC-V



or t8, t2, t6 and t4, t2, t5 lw t2, 20(t1)




Se **desactiva** a actualización del **PC:** se **repetirá** la etapa **IF**

Se **desactiva** la actualización de **IF/ID:** se **repetirá** la etapa **ID)**

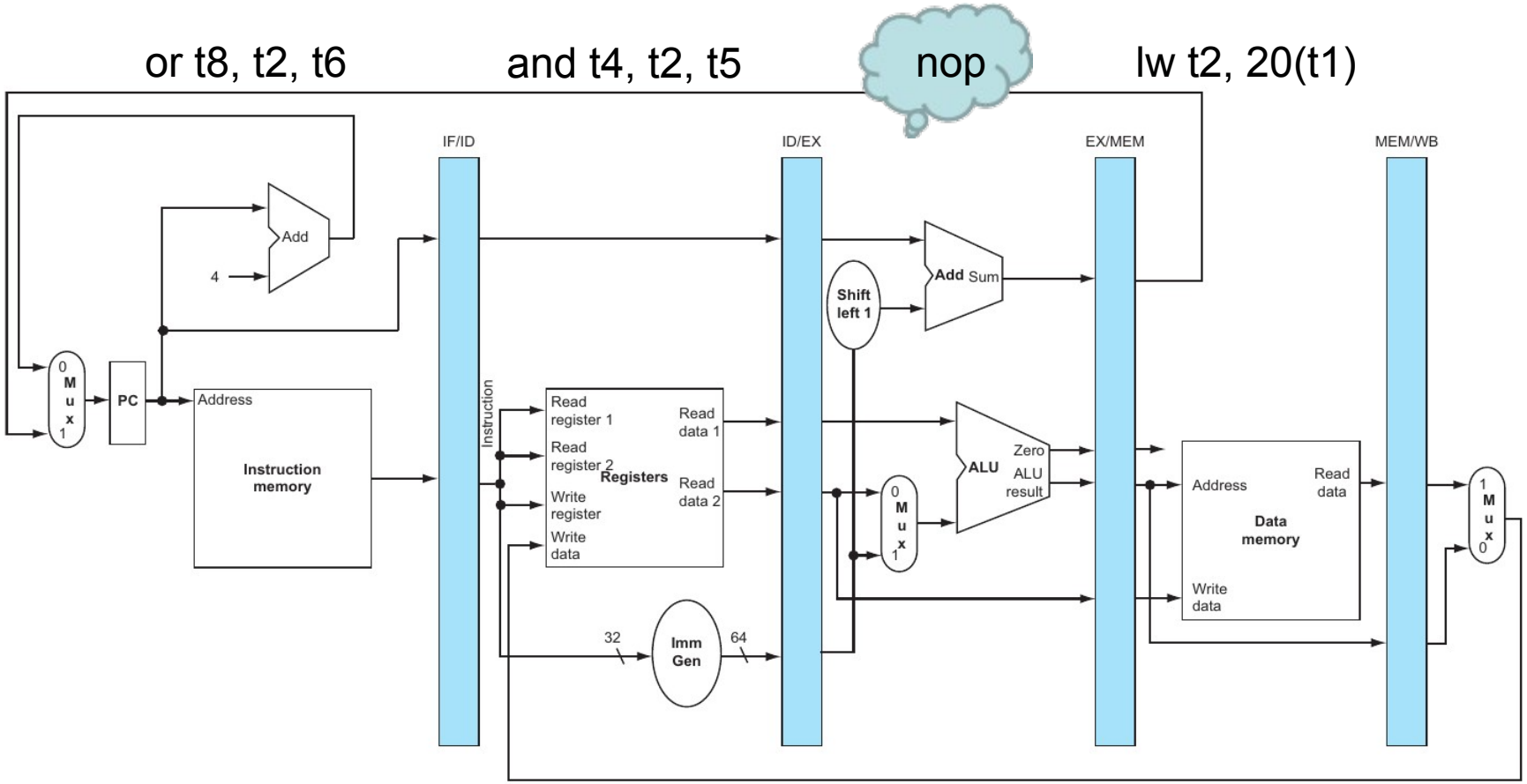
| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | **3** | 4 | 5 | 6 |
| lw t2, 20(t1) | IF | ID | **EX** | | | |
| **and t4, t2, t5** | | IF | **ID** | | | |
| or t8, t2, t6 | | | **IF** | | | |
| and t9,t4,t2 | | | | | | |

La instrucción **and** debe bloquearse

En el siguiente ciclo se **inserta** una **nop** en ID/EX

# Ejemplo de bloqueo en el RISC-V



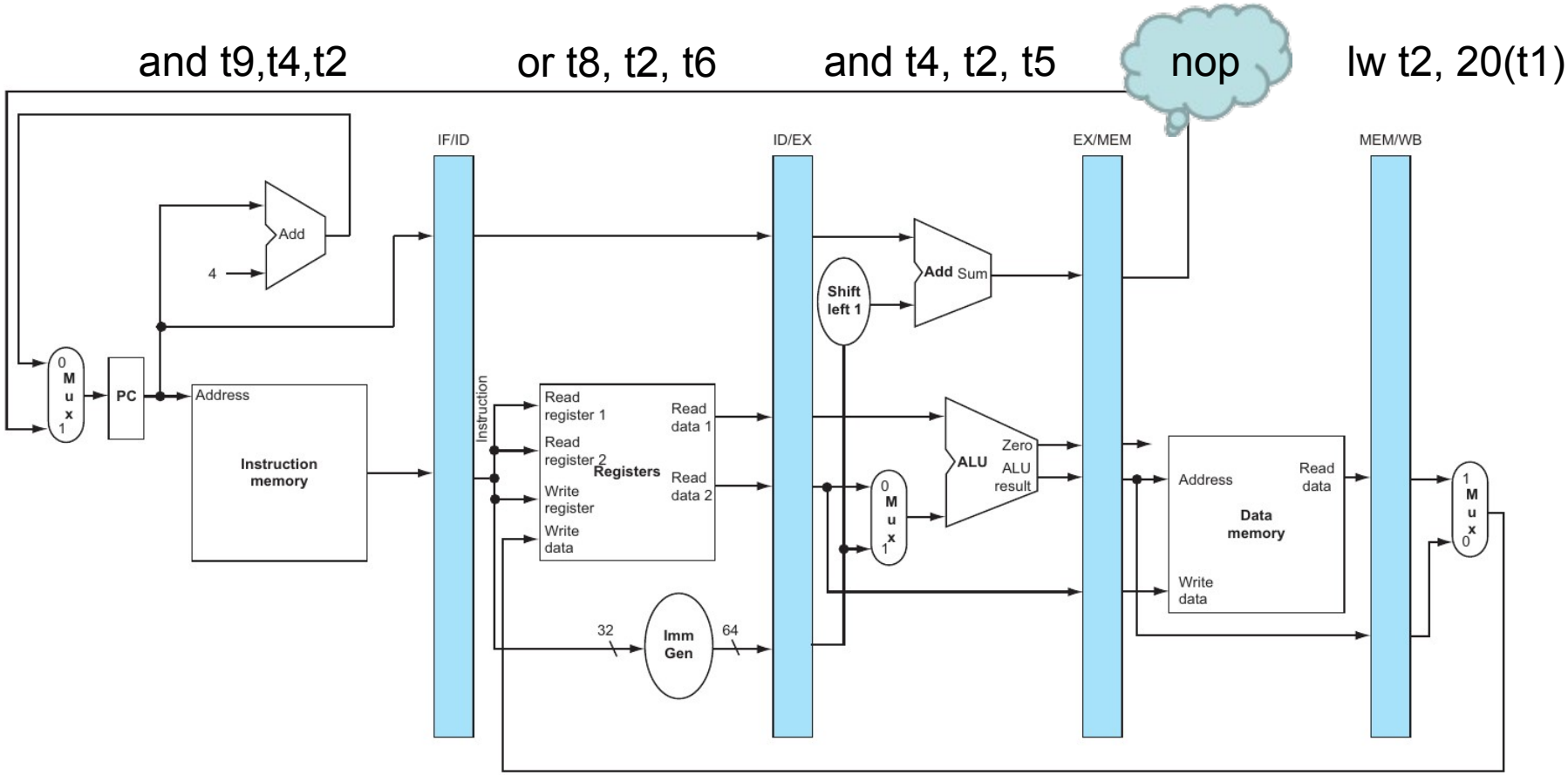



| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | **4** | 5 | 6 |
| lw t2, 20(t1) | IF | ID | EX | **MEM** | | |
| and t4, t2, t5 | | IF | ~~ID~~ | **ID** | | |
| or t8, t2, t6 | | | ~~IF~~ | **IF** | | |
| and t9,t4,t2 | | | | | | |

**Se repiten las etapas ID e IF. Se marcan con guiones para indicar un bloqueo**

# Ejemplo de bloqueo en el RISC-V






| Instrucciones | Ciclos de procesador | | | | | |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | **5** | 6 |
| lw t2, 20(t1) | IF | ID | EX | MEM | **WB** | |
| and t4, t2, t5 | | IF | – | ID | **EX** | |
| or t8, t2, t6 | | | – | IF | **ID** | |
| and t9,t4,t2 | | | | | **IF** | |

# Riesgos estructurales



- Los riesgos estructurales (o conflicto de recursos) se producen cuando no existen suficientes unidades funcionales en el pipeline para permitir que se solapen en el tiempo instrucciones de cualquier tipo.
- Un bloqueo estructural es un bloqueo consecuencia de un riesgo estructural.
- Los bloqueos estructurales se pueden evitar replicando unidades funcionales.
  - En algunos diseños, se permiten bloqueos estructurales porque el coste de replicar una unidad funcional no compensa el beneficio en el rendimiento.
- Ejemplo: En un RISC-V con **caché unificada** se producen bloqueos estructurales debido a que las etapas **IF** y **MEM** necesitan acceder a la misma caché.

| **Instrucciones ejecutadas** | **Ciclos de procesador** | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | **1** | **2** | **3** | **4** | **5** | **6** | | | | |
| lw t1, 4(t2) | **IF** | ID | EX | **MEM** | WB | | | | | |
| add t3, t4,t6 | | **IF** | ID | EX | MEM | WB | | | | |
| sub t5,t5,t6 | | | **IF** | ID | EX | MEM | WB | | | |
| or t7,t9,t6 | | | | – | **IF** | ID | EX | MEM | WB | |
| sub t8,t8,t6 | | | | | | **IF** | ID | EX | MEM | WB |

Las dos etapas no pueden acceder a la caché en el mismo ciclo: la etapa IF se bloquea.

# Riesgos de datos
# Dependencias y riesgos de datos



- Existe dependencias de datos entre dos instrucciones cuando una de ellas modifica un operando que es leído o escrito por la otra.
- Según el orden en el se realicen las lecturas y escrituras de los operandos, la dependencias se pueden clasificar en los siguientes tipos:
  - **RAW** (lectura después de escritura) o **dependencia real**.
  - **WAR** (escritura después de lectura) o **antidependencia**.
  - **WAW** (escritura después de escritura) o **dependencia de salida**.
- Los riesgos de datos se presentan en instrucciones con dependencias de datos cuando el orden de acceso a los operandos es modificado por la segmentación.
  - Se pueden dar entre operandos almacenados en registros o en memoria.

| Dependencia | Instrucciones | Operación |
|---|---|---|
| **RAW** | Instrucción *n*<br>…<br>Instrucción *m* | Escritura del operando A<br>…<br>Lectura del operando A |
| **WAR** | Instrucción *n*<br>…<br>Instrucción *m* | Lectura del operando A<br>…<br>Escritura del operando A |
| **WAW** | Instrucción *n*<br>…<br>Intrucción *m* | Escritura del operando A<br>…<br>Escritura del operando A |

# Riesgos de datos
# Dependencias y riesgos de datos



- Existe dependencias de datos entre dos instrucciones cuando una de ellas modifica un operando que es leído o escrito por la otra.
- Según el orden en el se realicen las lecturas y escrituras de los operandos, la dependencias se pueden clasificar en los siguientes tipos:
  - **RAW** (lectura después de escritura) o **dependencia real**.
  - **WAR** (escritura después de lectura) o **antidependencia**.
  - **WAW** (escritura después de escritura) o **dependencia de salida**.
- Los riesgos de datos se presentan en instrucciones con dependencias de datos cuando el orden de acceso a los operandos es modificado por la segmentación.
  - Se pueden dar entre operandos almacenados en registros o en memoria.

| Dependencia | Instrucciones | Operación |
|---|---|---|
| **RAW** | Instrucción *n*<br>…<br>Instrucción *m* | Escritura del operando A<br>…<br>Lectura del operando A |
| **WAR** | Instrucción *n*<br>…<br>Instrucción *m* | Lectura del operando A<br>…<br>Escritura del operando A |
| **WAW** | Instrucción *n*<br>…<br>Intrucción *m* | Escritura del operando A<br>…<br>Escritura del operando A |

# Riesgos de datos
# Dependencias y riesgos de datos



- Existe dependencias de datos entre dos instrucciones cuando una de ellas modifica un operando que es leído o escrito por la otra.
- Según el orden en el se realicen las lecturas y escrituras de los operandos, la dependencias se pueden clasificar en los siguientes tipos:
  - **RAW** (lectura después de escritura) o **dependencia real**.
  - **WAR** (escritura después de lectura) o **antidependencia**.
  - **WAW** (escritura después de escritura) o **dependencia de salida**.
- Los riesgos de datos se presentan en instrucciones con dependencias de datos cuando el orden de acceso a los operandos es modificado por la segmentación.
  - Se pueden dar entre operandos almacenados en registros o en memoria.

| Dependencia | Instrucciones | Operación |
|---|---|---|
| **RAW** | Instrucción *n*<br>…<br>Instrucción *m* | Escritura del operando A<br>…<br>Lectura del operando A |
| **WAR** | Instrucción *n*<br>…<br>Instrucción *m* | Lectura del operando A<br>…<br>Escritura del operando A |
| **WAW** | Instrucción *n*<br>…<br>Intrucción *m* | Escritura del operando A<br>…<br>Escritura del operando A |

# Riesgos de datos
# Dependencias y riesgos de datos



- Existe dependencias de datos entre dos instrucciones cuando una de ellas modifica un operando que es leído o escrito por la otra.
- Según el orden en el se realicen las lecturas y escrituras de los operandos, la dependencias se pueden clasificar en los siguientes tipos:
  - **RAW** (lectura después de escritura) o **dependencia real**.
  - **WAR** (escritura después de lectura) o **antidependencia**.
  - **WAW** (escritura después de escritura) o **dependencia de salida**.
- Los riesgos de datos se presentan en instrucciones con dependencias de datos cuando el orden de acceso a los operandos es modificado por la segmentación.
  - Se pueden dar entre operandos almacenados en registros o en memoria.

| Dependencia | Instrucciones | Operación | Se produce riesgo: |
|---|---|---|---|
| **RAW** | Instrucción *n*<br>…<br>Instrucción *m* | Escritura del operando A<br>…<br>Lectura del operando A | Si la lectura se realiza primero.<br>Lee el valor de A sin actualizar |
| **WAR** | Instrucción *n*<br>…<br>Instrucción *m* | Lectura del operando A<br>…<br>Escritura del operando A | Si la escritura se realiza primero.<br>Lee el valor modificado de A |
| **WAW** | Instrucción *n*<br>…<br>Intrucción *m* | Escritura del operando A<br>…<br>Escritura del operando A | Si la escritura de la instr. *m* se realiza primero.<br>Valor final de A |

# Riesgos de datos en el RISC-V



- En el RISC-V, solo producen riesgos las dependencias RAW entre operandos en registros.

| RAW | WAR | WAW |
|---|---|---|
| add **t1**, t2, t3<br>…<br>lw t4, 100(**t1**) | add t4, **t2**, t3<br>…<br>subi **t2**, t1, 50 | or **t2**, t4, t3<br>…<br>subi **t2**, t1, 50 |

| | | | | | |
|---|---|---|---|---|---|
| IF | ID | EX | MEM | **WB** | |
| | IF | **ID** | EX | MEM | WB |

| | | | | | |
|---|---|---|---|---|---|
| IF | **ID** | EX | MEM | WB | |
| | IF | ID | EX | MEM | **WB** |

| | | | | | |
|---|---|---|---|---|---|
| IF | ID | EX | MEM | **WB** | |
| | IF | ID | EX | MEM | **WB** |

**Riesgo**: Se **lee** el operando en la etapa **ID** **antes** de que haya sido **escrito** en la etapa **WB**

**No hay riesgo** porque las escrituras en registro se hacen en la última etapa (WB) → no puede adelantarse a las lecturas

**No hay riesgo** porque solo hay una etapa de escritura en registro (WB) → las escrituras se hacen en orden.

- Las dependencias entre **operandos en memoria** No producen riesgos, pues solo existe una etapa en la que se accede a memoria → Los accesos a memoria se realizan en orden.

sw t4, **100(t1)**

lw t2, **100(t1)**

| | | | | | |
|---|---|---|---|---|---|
| IF | ID | EX | **MEM** | WB | |
| | IF | ID | EX | **MEM** | WB |

# Riesgos de datos en el RISC-V
# Ejemplo 1



Supongamos que se ejecuta el siguiente código

Program execution order (in instructions)

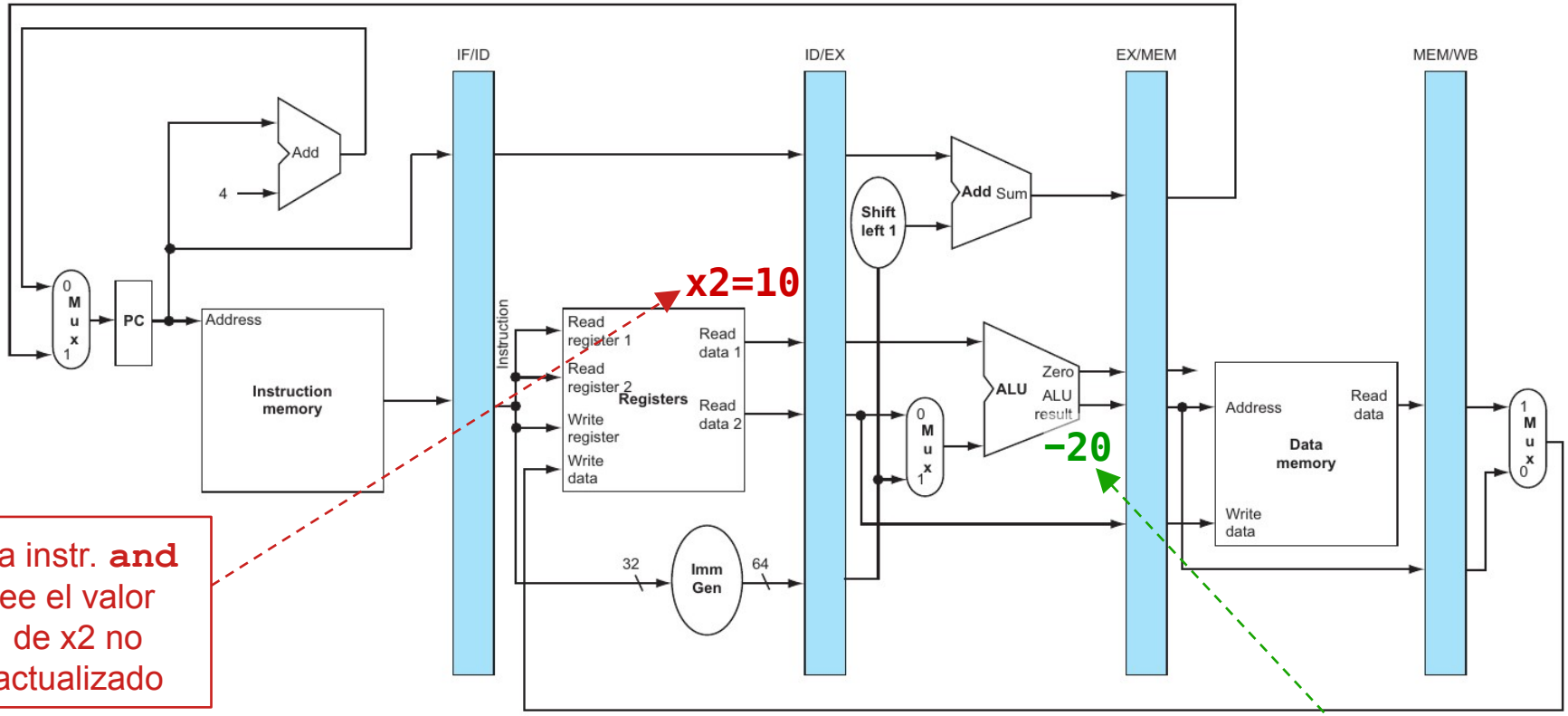
```
sub x2, x1, x3
and x12, x2, x5
or x13, x6, x2
add x14, x2, x2
sd x15, 100(x2)
```

Estas instrucciones presentan una dependencia RAW con la instrucción **sub**

# Riesgos de datos en el RISC-V
# Ejemplo 1 (II)



```
or x13,x6,x2      and x12,x2,x5    sub x2,x1,x3
```




La instr. **and** lee el valor de x2 no actualizado

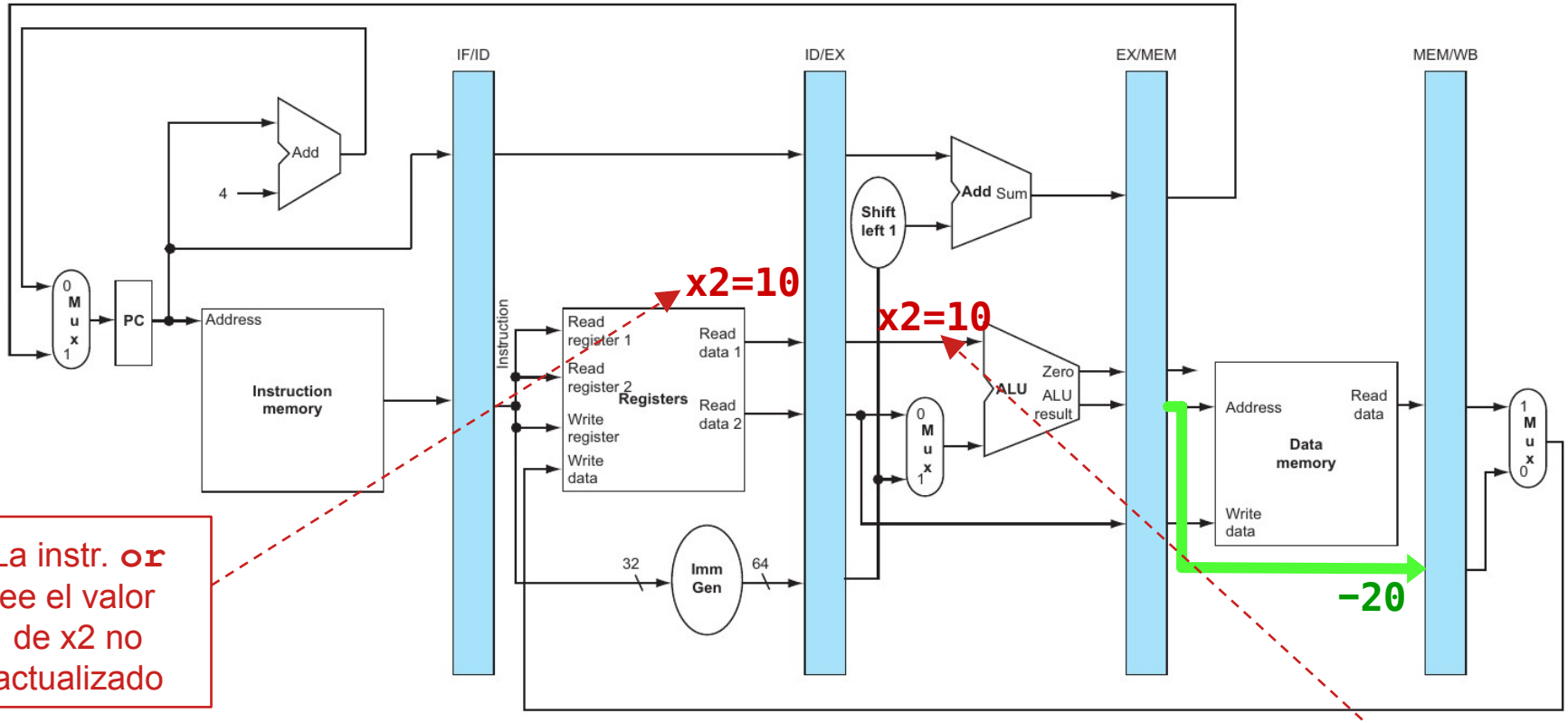
| | | | | | |
|---|---|---|---|---|---|
| sub t2,t1,t3 | IF | ID | EX | MEM | WB |
| and t12,t2,t5 | | IF | ID | EX | MEM |
| or t13,t6,t2 | | | IF | ID | EX |
| add x14,x2,x2 | | | | IF | |

Nuevo valor de x2 generado por **sub**

# Riesgos de datos en el RISC-V
# Ejemplo 1 (III)



```
add x14,x2,x2     or x13,x6,x2    and x12,x2,x5 sub x2,x1,x3
```




La instr. **or** lee el valor de x2 no actualizado

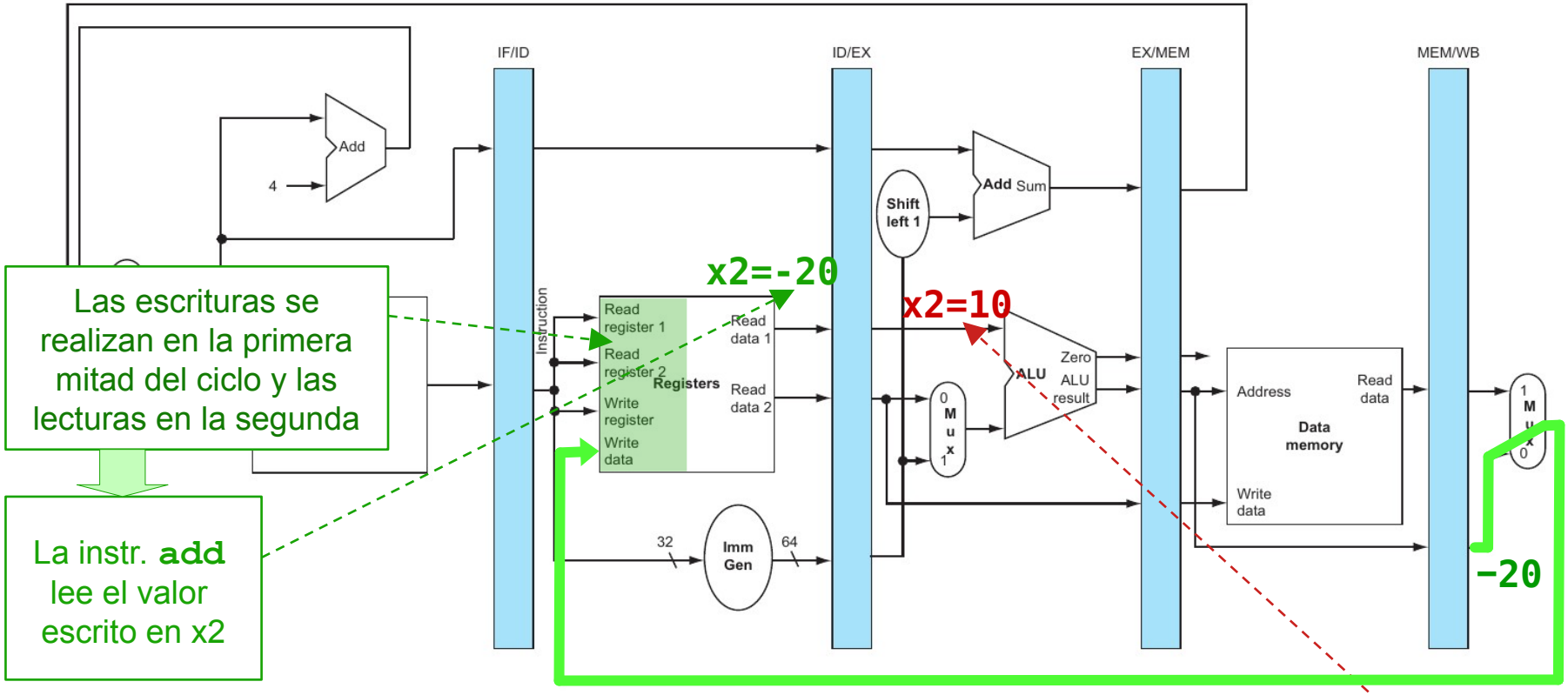
| | | | | | |
|---|---|---|---|---|---|
| sub t2,t1,t3 | IF | ID | EX | MEM | WB |
| and t12,t2,t5 | | IF | ID | EX | MEM |
| or t13,t6,t2 | | | IF | ID | EX |
| add x14,x2,x2 | | | | IF | |

Si no se soluciona el riesgo de datos, la instrucción **and** usará el valor incorrecto.

# Riesgos de datos en el RISC-V
# Ejemplo 1 (IV)



```
sd x15,100(x2)    add x14,x2,x2    or x13,x6,x2 and x12,x2,x5  sub x2,x
```




Las escrituras se realizan en la primera mitad del ciclo y las lecturas en la segunda

La instr. **add** lee el valor escrito en x2

| | | | | | |
|---|---|---|---|---|---|
| sub t2,t1,t3 | IF | ID | EX | MEM | WB |
| and t12,t2,t5 | | IF | ID | EX | MEM |
| or t13,t6,t2 | | | IF | ID | EX |
| add x14,x2,x2 | | | | IF | ID |

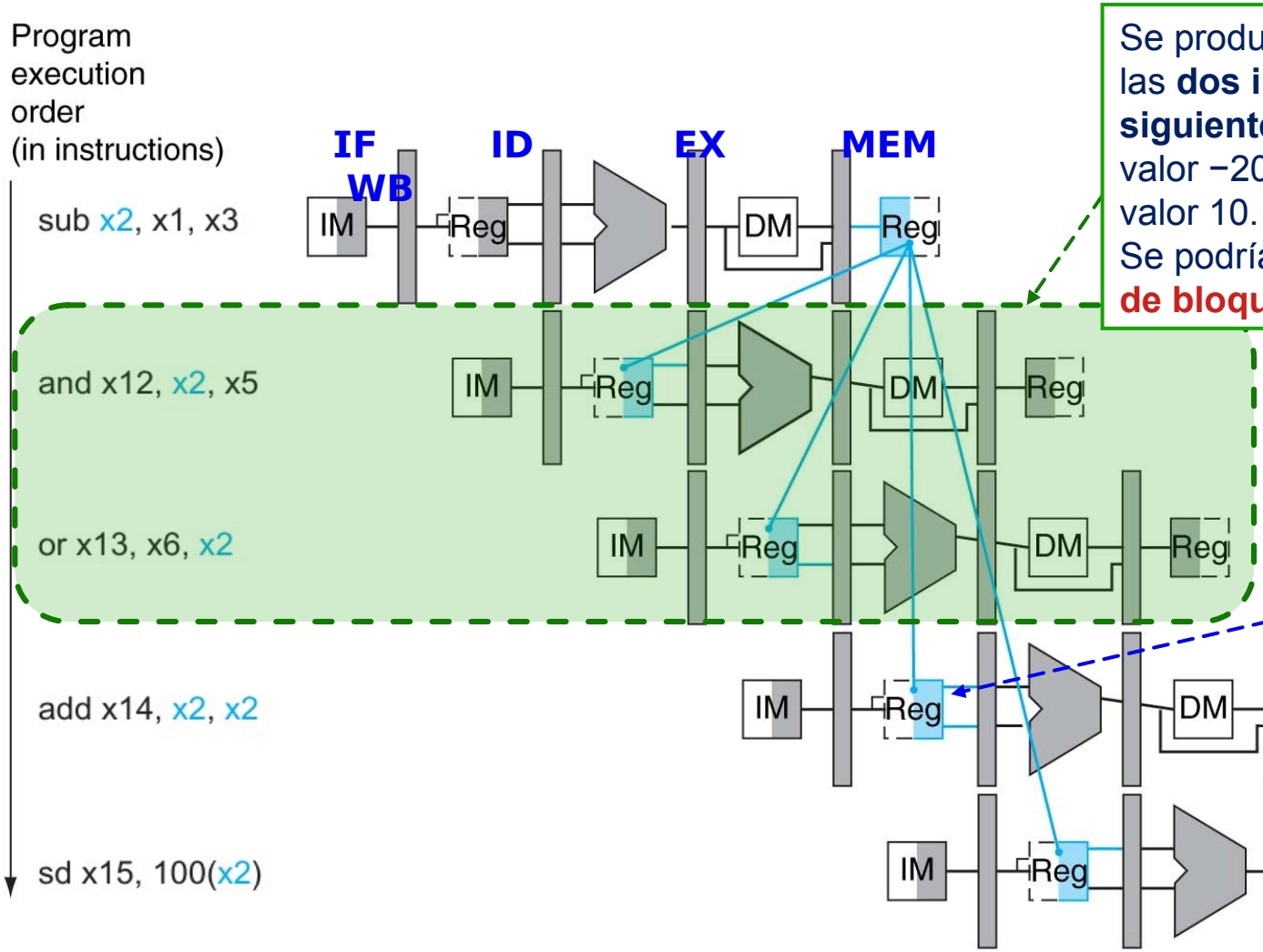
Si no se soluciona el riesgo de datos, la instrucción **or** usará el valor incorrecto.

# Riesgos de datos en el RISC-V
## Bloqueos de datos: Ejemplo 1 (V) continuación



Valor del registro x2 en cada ciclo

| Time (in clock cycles) | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 7 | CC 8 | CC 9 |
|---|---|---|---|---|---|---|---|---|---|
| Value of register $2: | 10 | 10 | 10 | 10 | 10/–20 | –20 | –20 | –20 | –20 |




Se producen riesgos **solo** con las **dos instrucciones siguientes.** Deberían leer el valor −20 en x2, pero leen el valor 10.
Se podría resolver con **2 ciclos de bloqueo** (sig. diapositiva).

Esta instrucción lee el valor correcto gracias a que el operando se escribe en **WB** durante la **primera mitad** del ciclo y se lee en **ID** durante la **segunda mitad**.

# Riesgos de datos en el RISC-V
# Bloqueos de datos: Ejemplo 1 (VI)



Valor del registro x2 en cada ciclo

| Time (in clock cycles) | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 7 | CC 8 | CC 9 |
|---|---|---|---|---|---|---|---|---|---|
| Value of register $2: | 10 | 10 | 10 | 10 | 10/–20 | –20 | –20 | –20 | –20 |

Program execution order (in instructions)

sub $2, $1, $3

add $14, $2,$2

sw $15, 100($2)

Se producen riesgos **solo** con las **dos instrucciones siguientes.** Deberían leer el valor −20 en x2, pero leen el valor 10.
Se podría resolver con **2 ciclos de bloqueo** (sig. diapositiva).

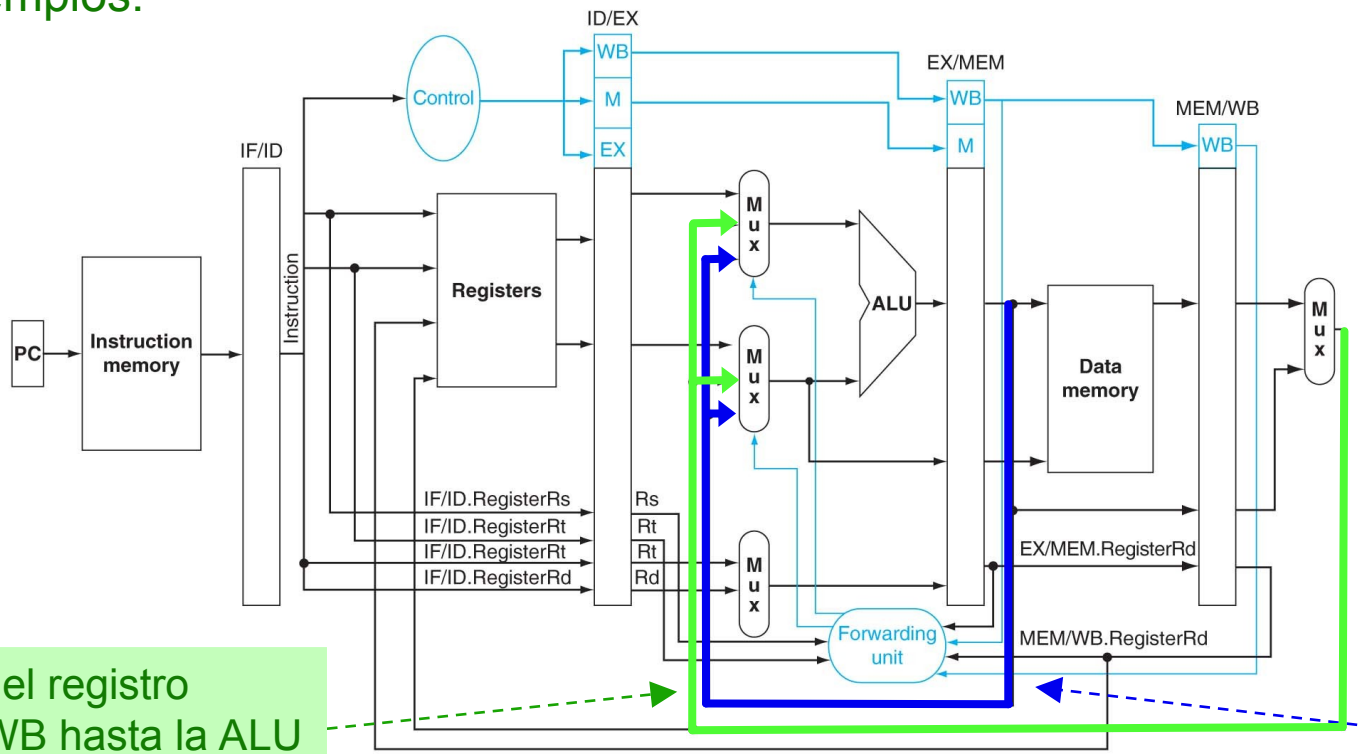
```
sub x2, x1, x3        IF  ID  EX  MEM WB
and x12, x2, x5           IF  -   -   ID  EX  MEM WB
or x13, x6, x2                        IF  ID  EX  MEM WB
```

Las dependencias de datos son muy frecuentes en los programas → insertar 2 ciclos de bloqueo supone una **reducción significativa** del **rendimiento**.

# Anticipación de resultados



- La anticipación de resultados (*forwarding*) o desviación (*bypassing*) permite reducir el número de bloqueos de datos.
  - Consiste en proporcionar el operando a la unidad funcional que lo necesita antes de que esté disponible en el banco de registros.
  - Se utilizan desvíos (*bypasses*), que se implementan mediante multiplexores y buses que llevan el operando desde los registros de segmentación hasta las unidades funcionales (o hasta otro registro de segmentación)
- Ejemplos:




Desde el registro MEM/WB hasta la ALU

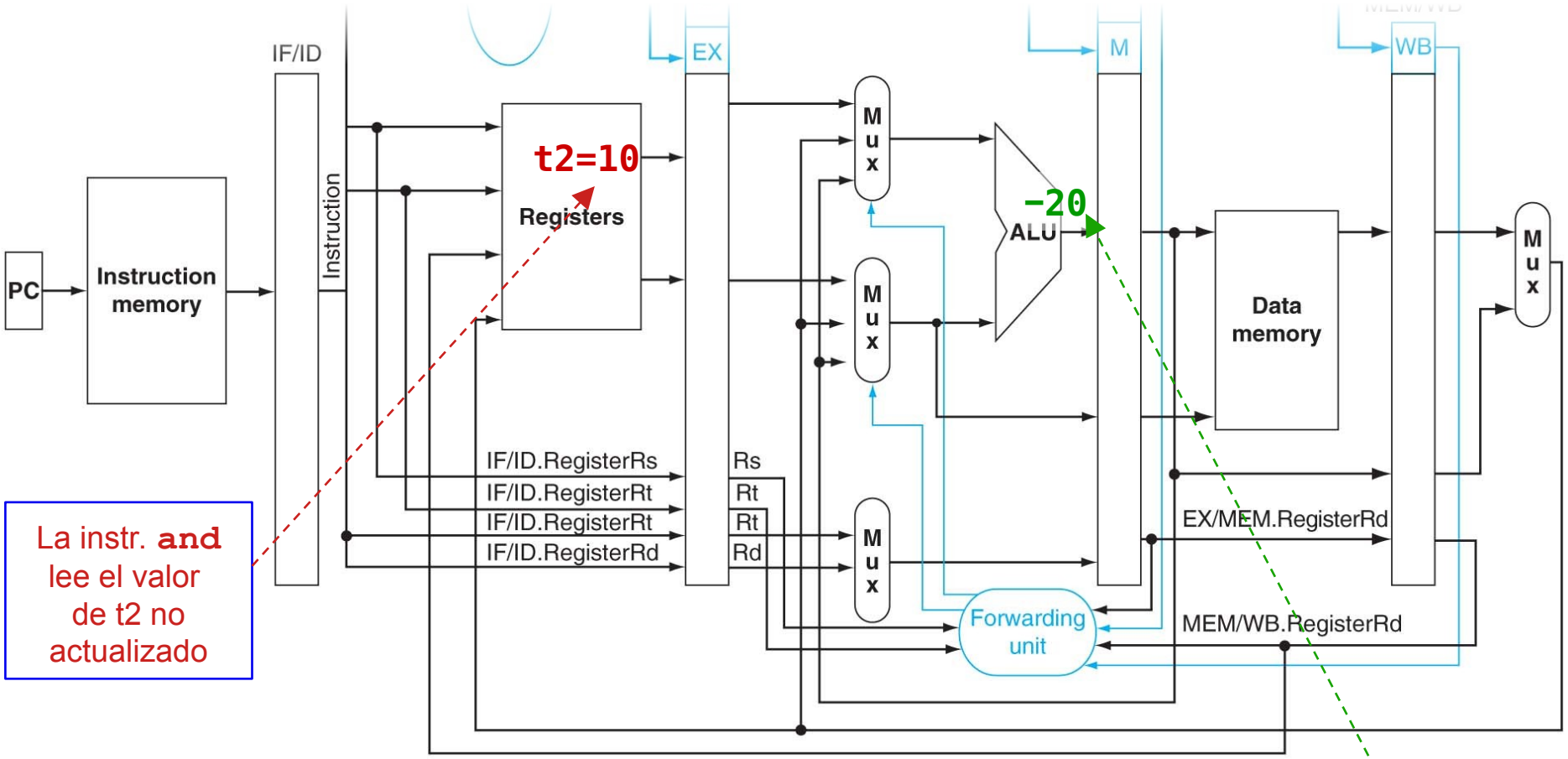
Desde el registro EX/MEM hasta la ALU

# Anticipación de resultados
# Continuación del Ejemplo 1 con desvíos (I)



Anticipación de resultados para evitar riesgos en el código del Ejemplo 1

```
or t13,t6,t2      and t12,t2,t5      sub t2,t1,t3
```




La instr. **and** lee el valor de t2 no actualizado

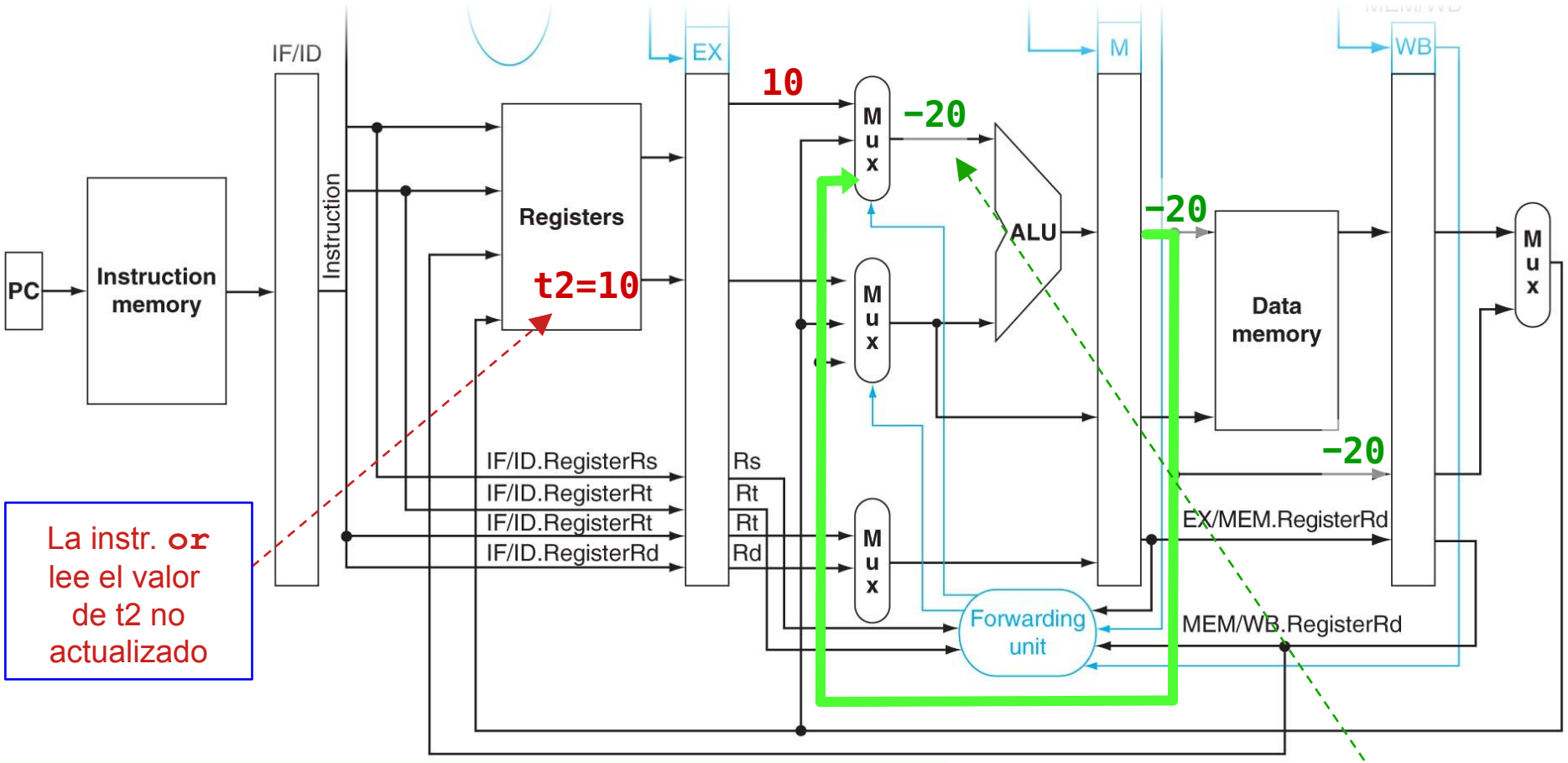
| | | | | | | |
|---|---|---|---|---|---|---|
| sub t2,t1,t3 | IF | ID | EX | MEM | WB | |
| and t12,t2,t5 | | IF | ID | EX | MEM | WB |
| or t13,t6,t2 | | | IF | ID | EX | MEM | WB |

Nuevo valor de t2 generado por sub

# Anticipación de resultados
# Continuación del Ejemplo 1 con desvíos (II)



```
add t14,t2,t2      or t13,t6,t2      and t12,t2,t5      sub t2,t1,t3
```



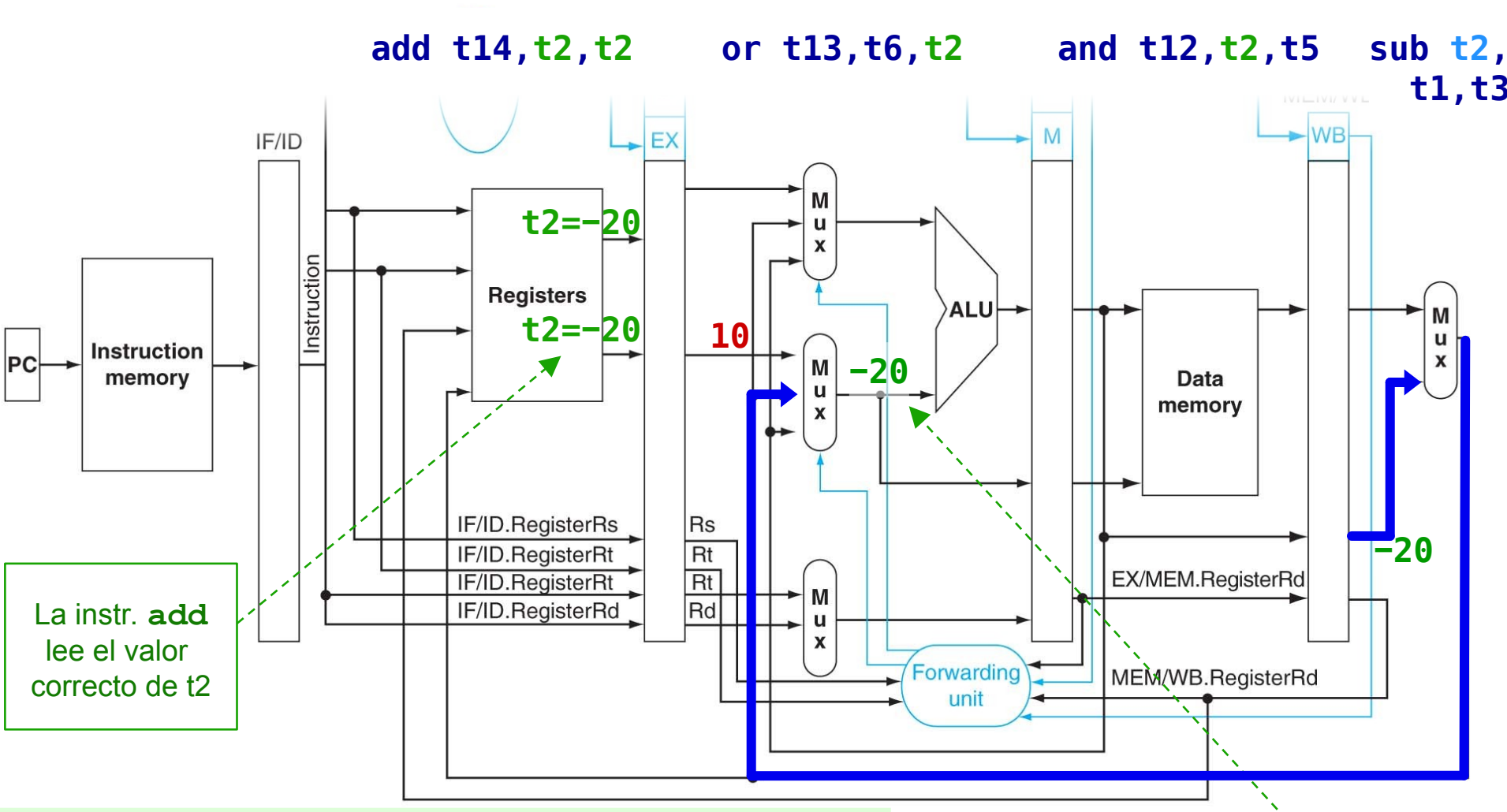

La instr. **or** lee el valor de t2 no actualizado

| | | | | | | |
|---|---|---|---|---|---|---|
| sub t2,t1,t3 | IF | ID | EX | MEM | WB | |
| and t12,t2,t5 | | IF | ID | EX | MEM | WB |
| or t13,t6,t2 | | | IF | ID | EX | MEM | WB |

Se adelanta el nuevo valor de t2 hasta la ALU desde EX/MEM

# Anticipación de resultados
# Continuación del Ejemplo 1 con desvíos (III)






La instr. **add** lee el valor correcto de t2

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| sub t2,t1,t3 | IF | ID | EX | MEM | WB | | |
| and t12,t2,t5 | | IF | ID | EX | MEM | WB | |
| or t13,t6,t2 | | | IF | ID | EX | MEM | WB |

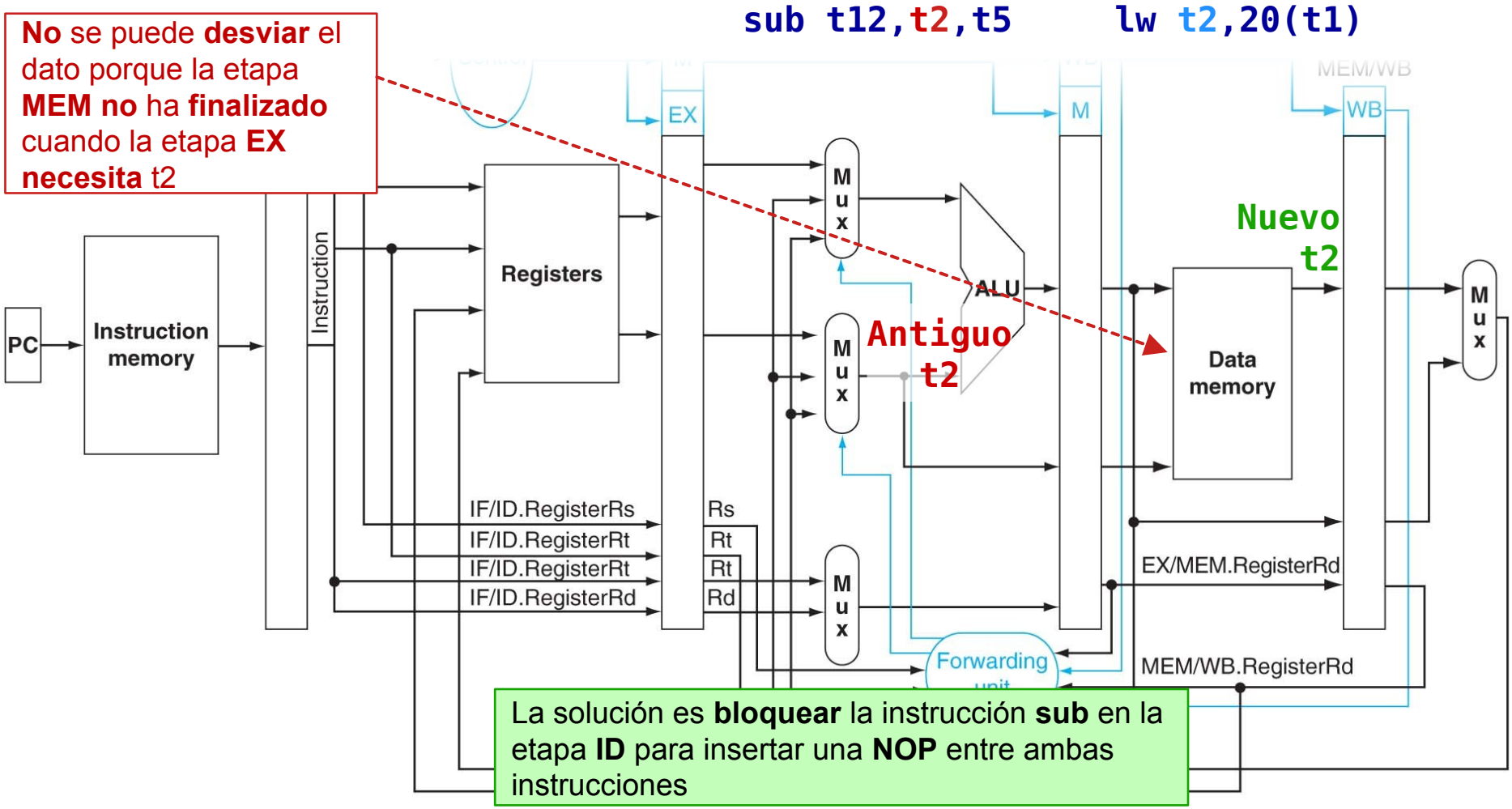
Se adelanta el nuevo valor de t2 hasta la ALU desde MEM/WB

# Anticipación de resultados
# Bloqueos inevitables (I)



- A pesar de los desvíos, existen casos en los que es no se pueden evitar los bloqueos
  - Sin embargo, el número de bloqueos necesarios suele reducirse
  - En el siguiente ejemplo se reduce de 2 a 1.

**No** se puede **desviar** el dato porque la etapa **MEM no** ha **finalizado** cuando la etapa **EX necesita** t2

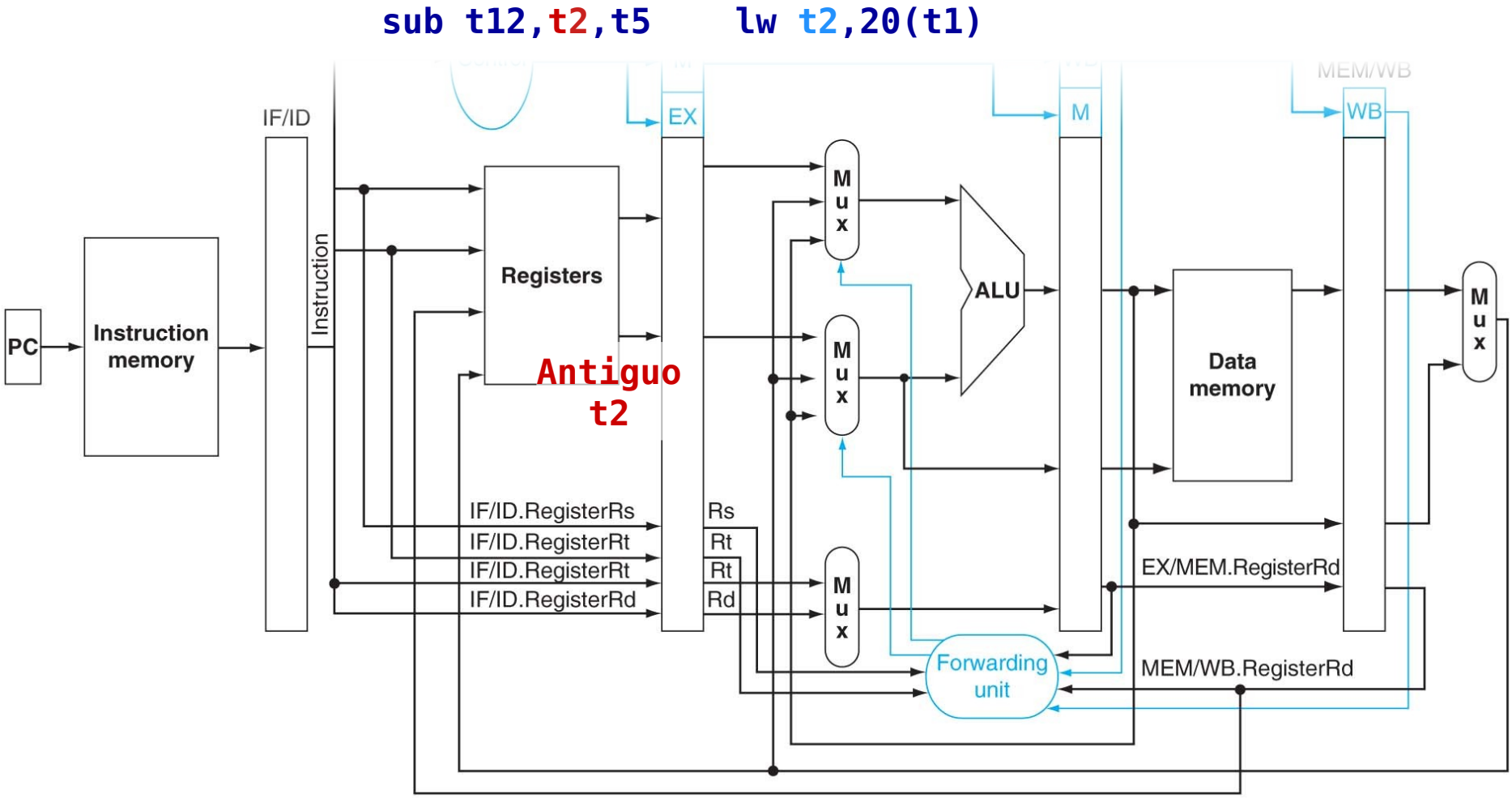



La solución es **bloquear** la instrucción **sub** en la etapa **ID** para insertar una **NOP** entre ambas instrucciones

# Anticipación de resultados
# Bloqueos inevitables (II)



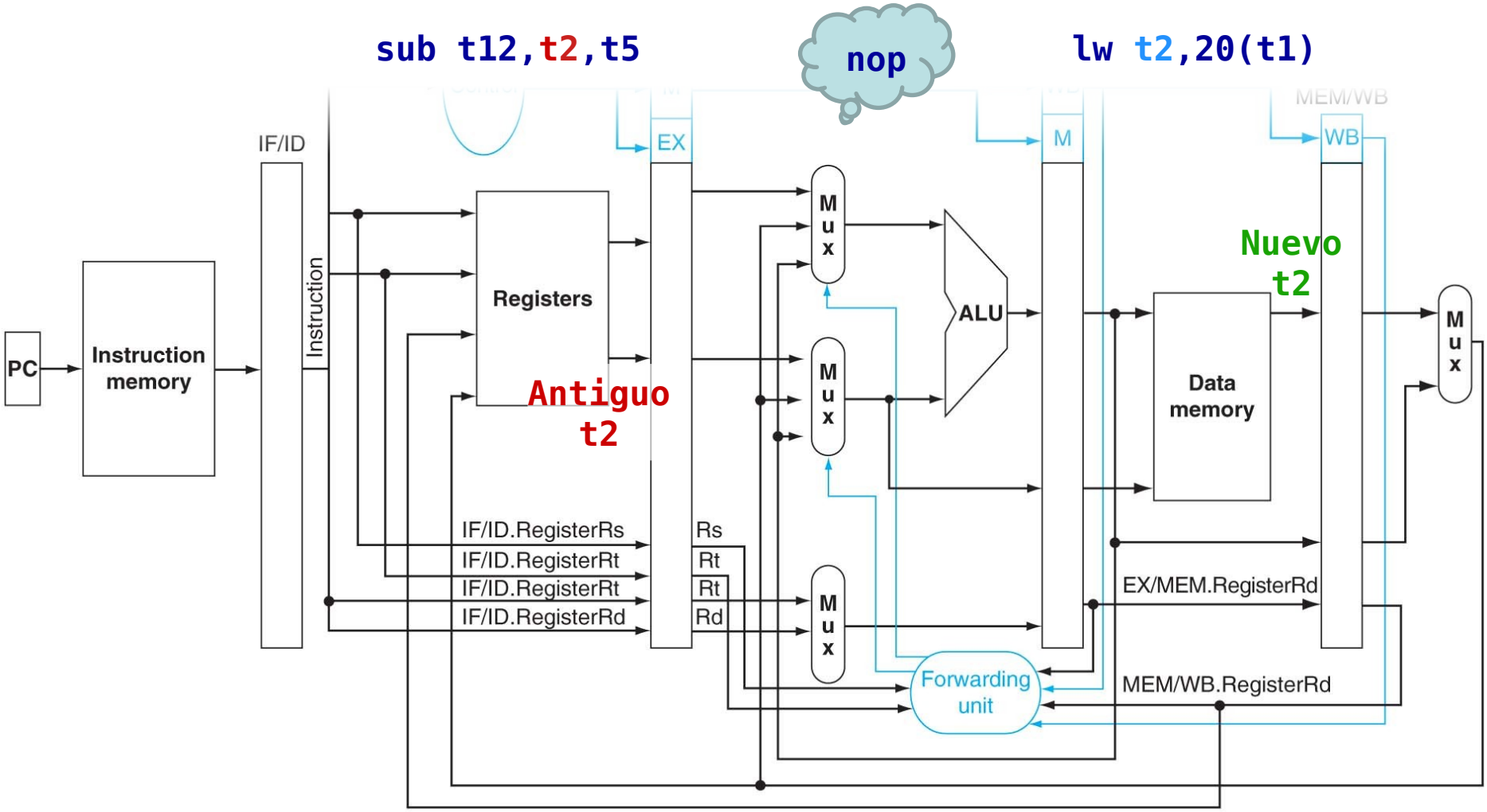
```
lw  t2,20(t1)   IF  ID  EX  MEM  WB
sub t12,t2,t5       IF  ID  ID   EX
```



# Anticipación de resultados
# Bloqueos inevitables (III)



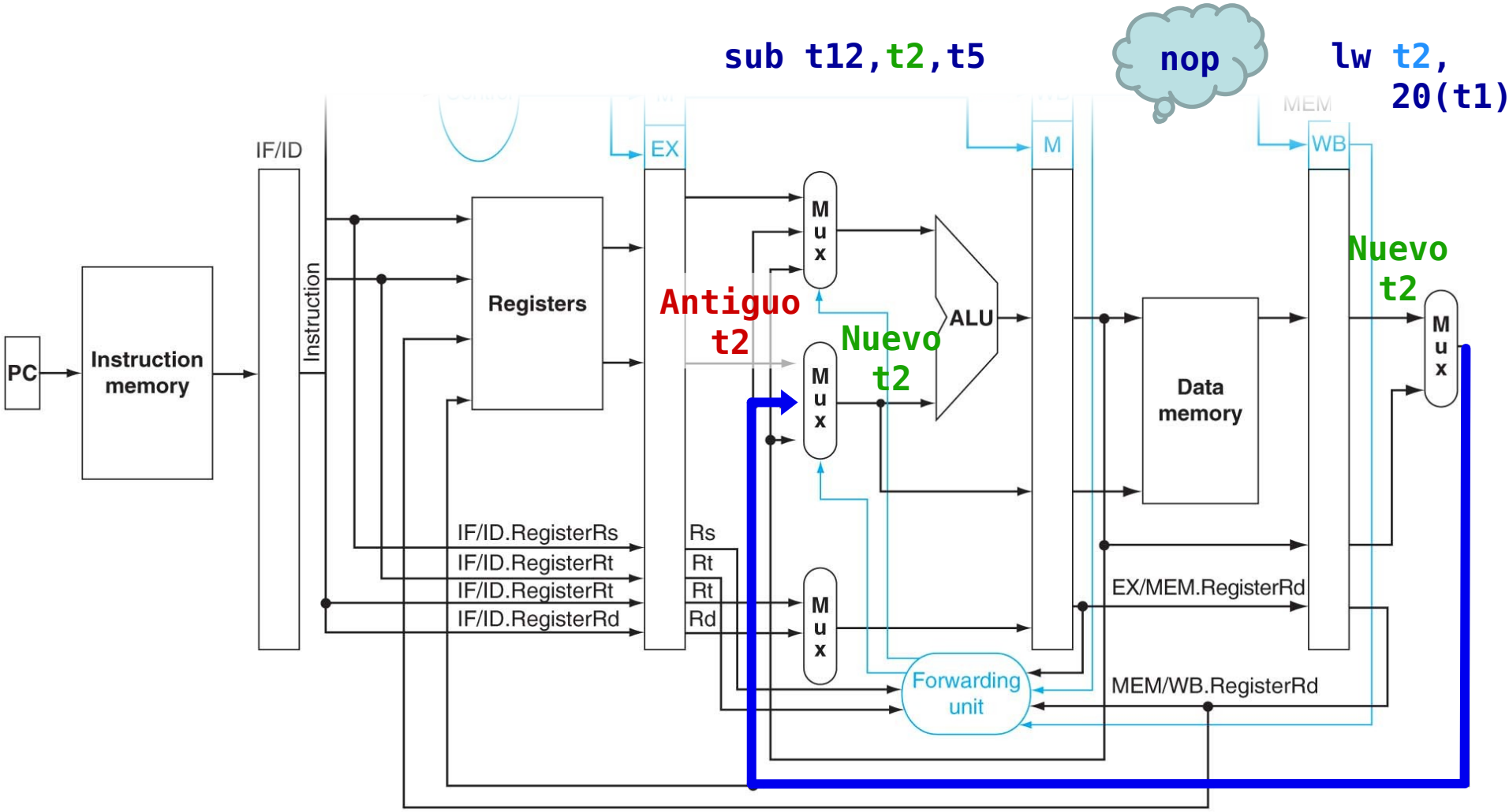
```
lw t2,20(t1)    IF   ID   EX   MEM   WB
sub t12,t2,t5        IF   -    ID    EX
```



# Anticipación de resultados
# Bloqueos inevitables (IV)



```
lw t2,20(t1)   IF   ID   EX   MEM  WB
sub t12,t2,t5      IF   -    ID   EX
```



# Riesgos de control



- Los riesgos de control son consecuencia de las instrucciones de salto.
  - Se producen cuando el salto es tomado:
    - Antes de que la instrucción de salto determine la dirección a la que hay que saltar, se leen instrucciones que no deberían ejecutarse si el salto es tomado.

- Ejemplo 2: Suponga que se ejecuta el siguiente código en el RISC-V (continúa en la siguiente diapositiva):

| PC | Instrucciones |
|---|---|
| 40 | **beq** t1, t3, 16 |
| 44 | and t12, t2, t5 |
| 48 | or t13, t6, t2 |
| 52 | add t14, t2, t2 |
| ... | ... |
| 72 | lw t4, 50(t7) |

Destino del salto

Si el salto es tomado, estas instrucciones no deberían ejecutarse, pero comienzan a cargarse antes de que se produzca el salto.

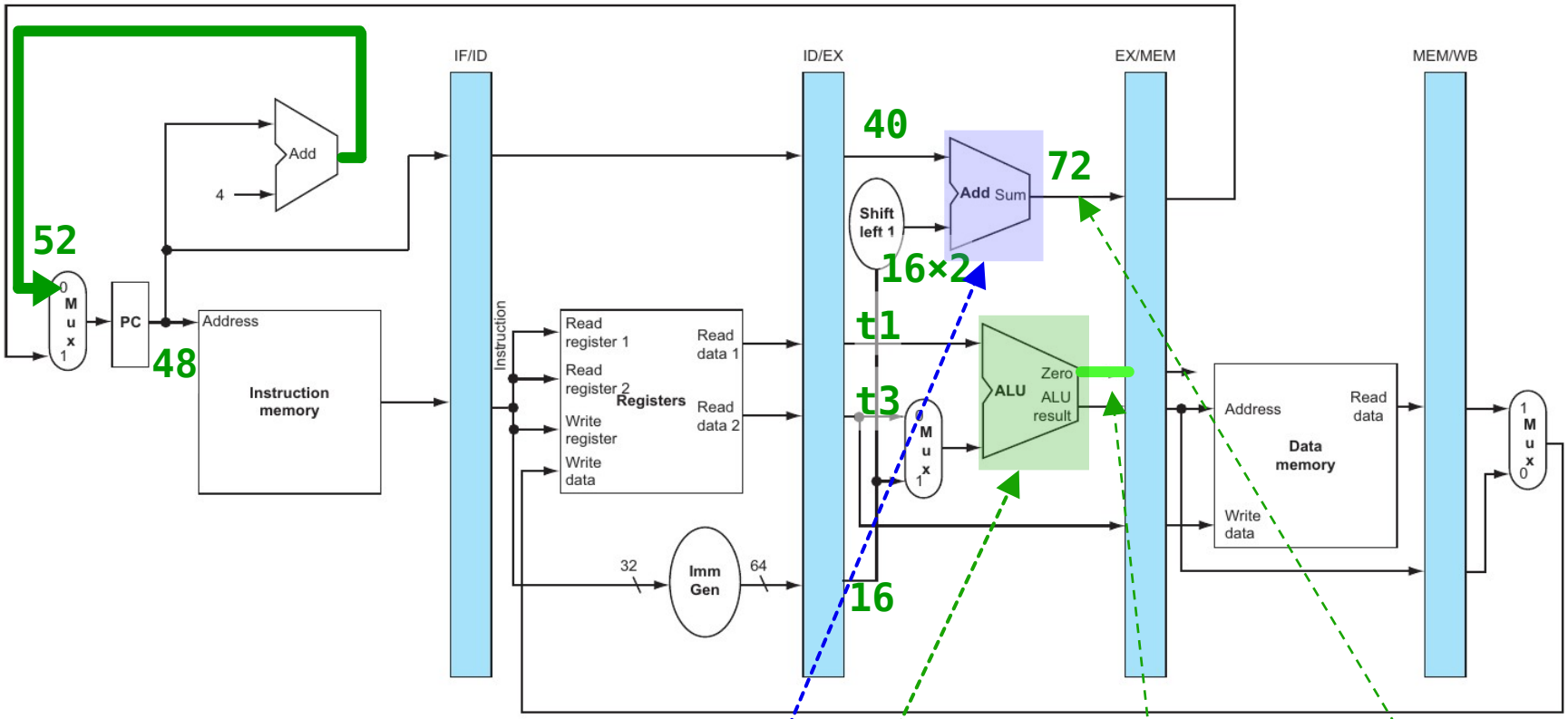
# Riesgos de control en el RISC-V
# Ejemplo 2 (I)



PC=48 `or t13,t6,t2`

PC=44 `and t12,t2,t5`

PC=40 `beq t1,t3,16`




El cálculo de la **dirección destino** del salto y la **evaluación de la condición** se realizan en la etapa **EX**

Suponemos que la condición se cumple (el salto es tomado)

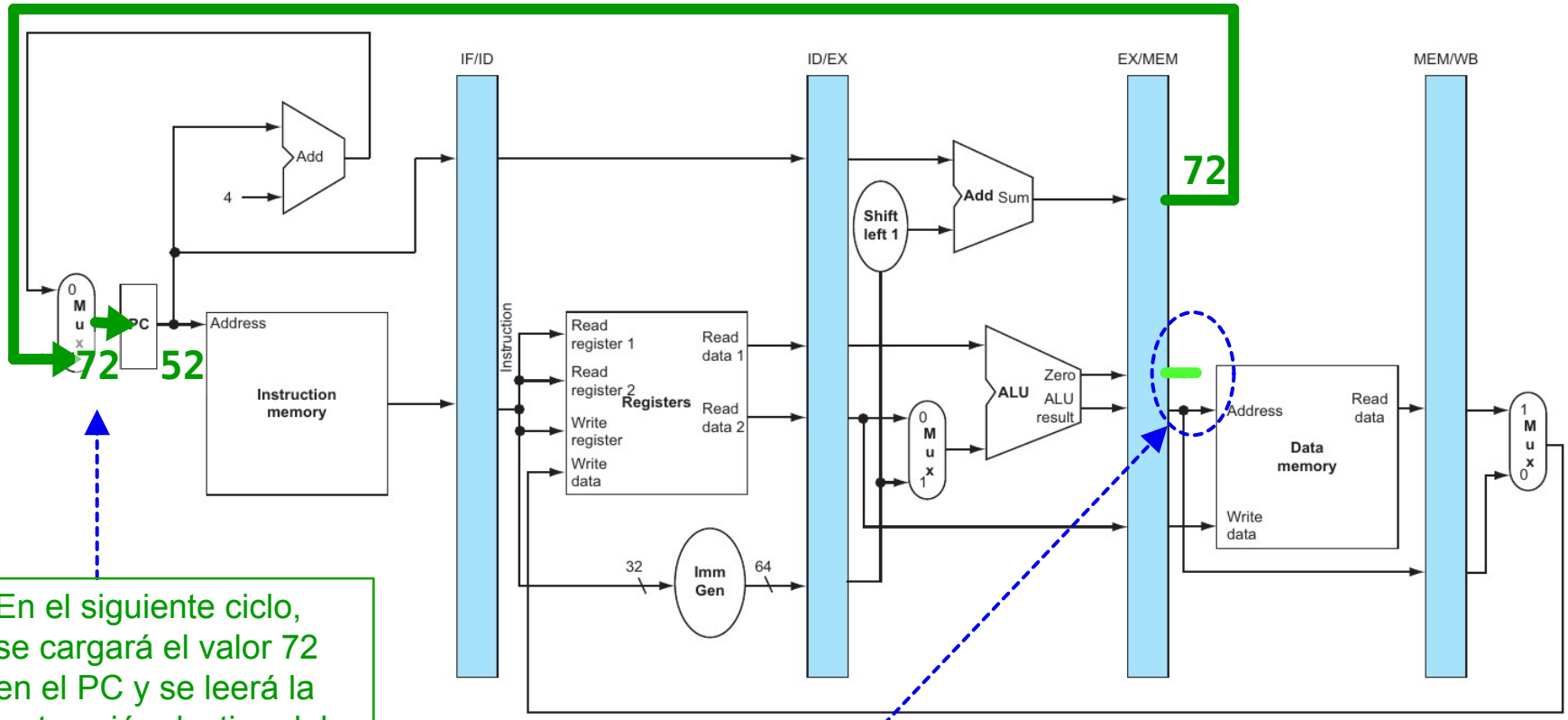
La instrucción a la que se salta se encuentra en la dirección 72

# Riesgos de control en el RISC-V
# Ejemplo 2 (II)



PC=52 PC=48 PC=44 PC=40

`add t14,t2,t2` `or t13,t6,t2` `and t12,t2,t5` `beq t1,t3,16`




En el siguiente ciclo, se cargará el valor 72 en el PC y se leerá la instrucción destino del salto

La línea Zero está activa

# Riesgos de control en el RISC-V
# Ejemplo 2 (III)

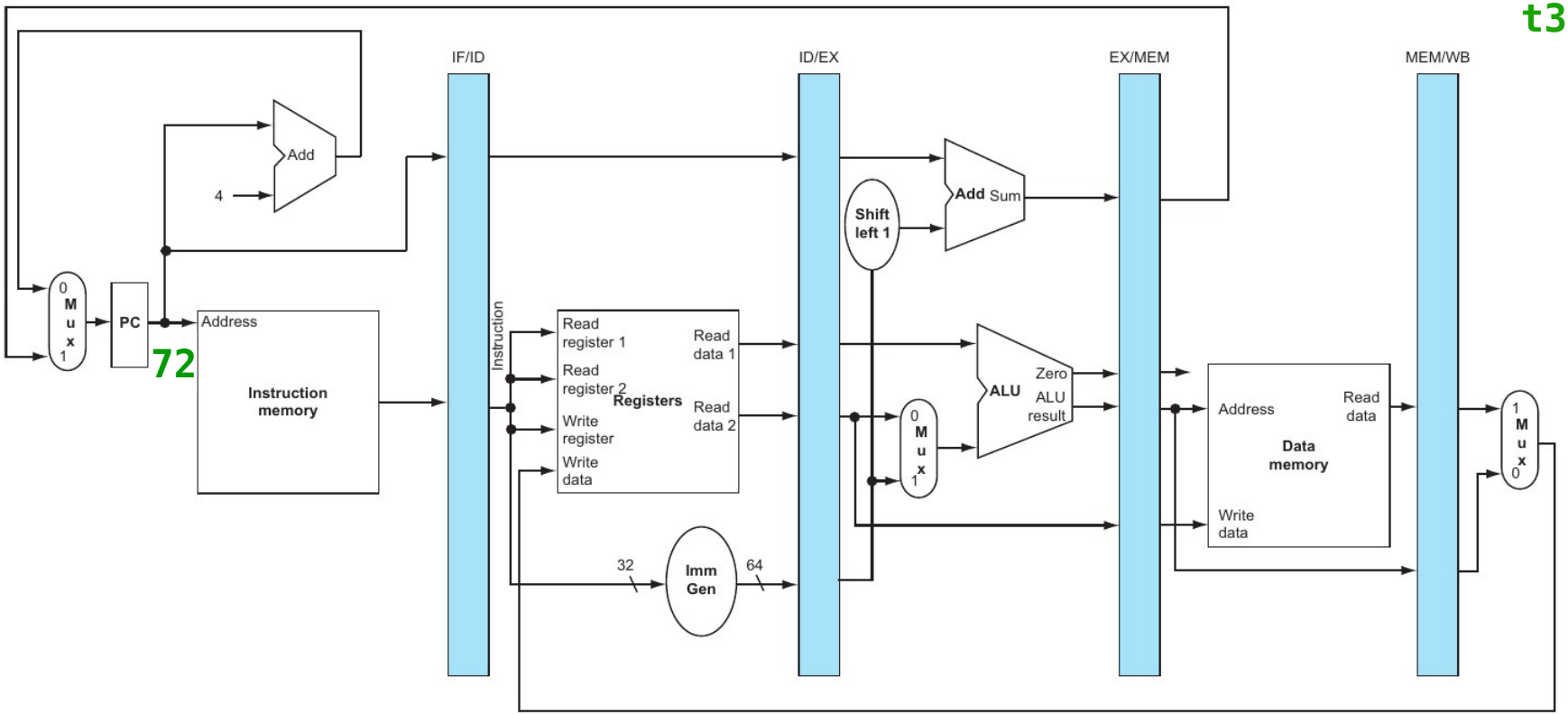


**Destino del salto: PC=72**

**Estas instrucciones no deben ejecutarse**

**PC=40**

**lw t4,50(t7)** **add t14,t2,t2** **or t13,t6,t2** **and t12,t2,t5** **beq t1, t3,16**




- La solución más simple es descartar (*flush*) 3 instrucciones siempre que se ejecute una instrucción de salto (véase la diapositiva siguiente).

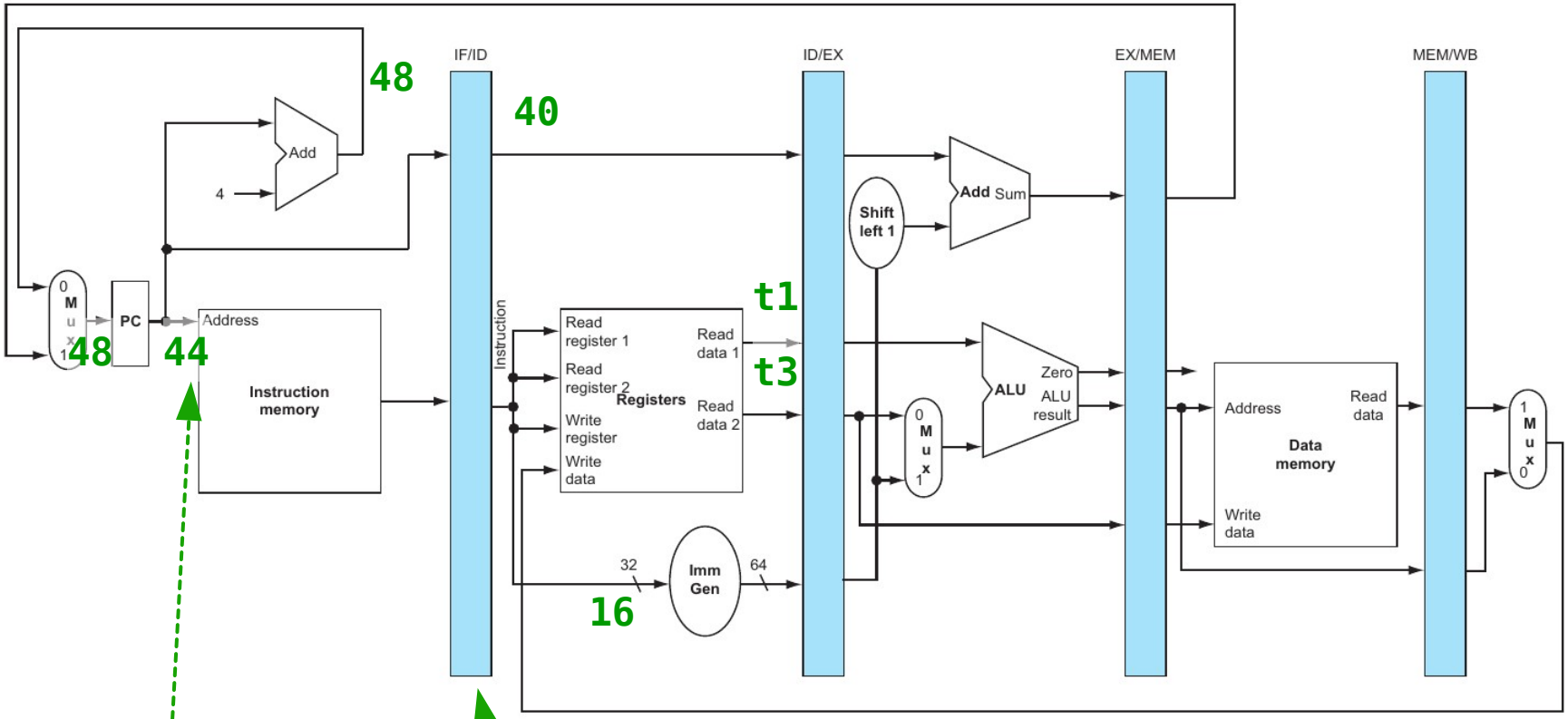
# Riesgos de control en el RISC-V
# Ejemplo 2 (IV): Bloqueos de control



PC=44

```
and t12,t2,t5      beq t1,t3,16
```

**Se detecta una instrucción de salto se insertarán 3 bloqueos**




Se desactiva la actualización del PC → se volverá a cargar el valor 44 (se repite IF).

En el siguiente ciclo se **insertará** una `nop` en IF/ID

| | | | | | |
|---|---|---|---|---|---|
| `beq t1,t3,16` | IF | ID | EX | MEM | WB |
| `and t12,t2,t5` | | IF | IF | IF | IF |

# Riesgos de control en el RISC-V
# Ejemplo 2 (V): Bloqueos de control



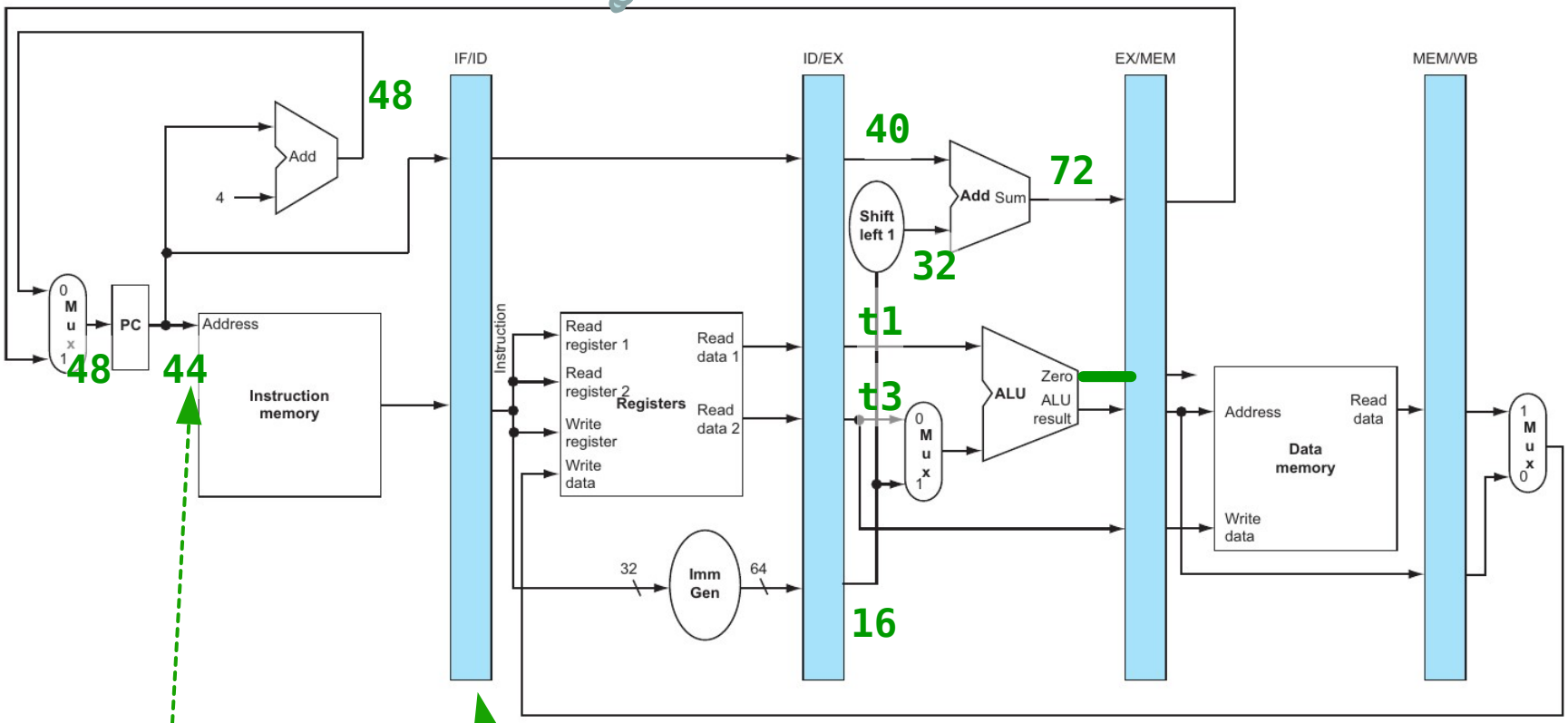
**1er bloqueo**

PC=44

`and t12,t2,t5`

nop

`beq t1,t3,16`




Se desactiva la actualización del PC → se volverá a cargar el valor 44 (se repite IF).

En el siguiente ciclo se **insertará** una `nop` en IF/ID

| | | | | | |
|---|---|---|---|---|---|
| `beq t1,t3,16` | IF | ID | EX | MEM | WB |
| `and t12,t2,t5` | | - | IF | IF | IF |

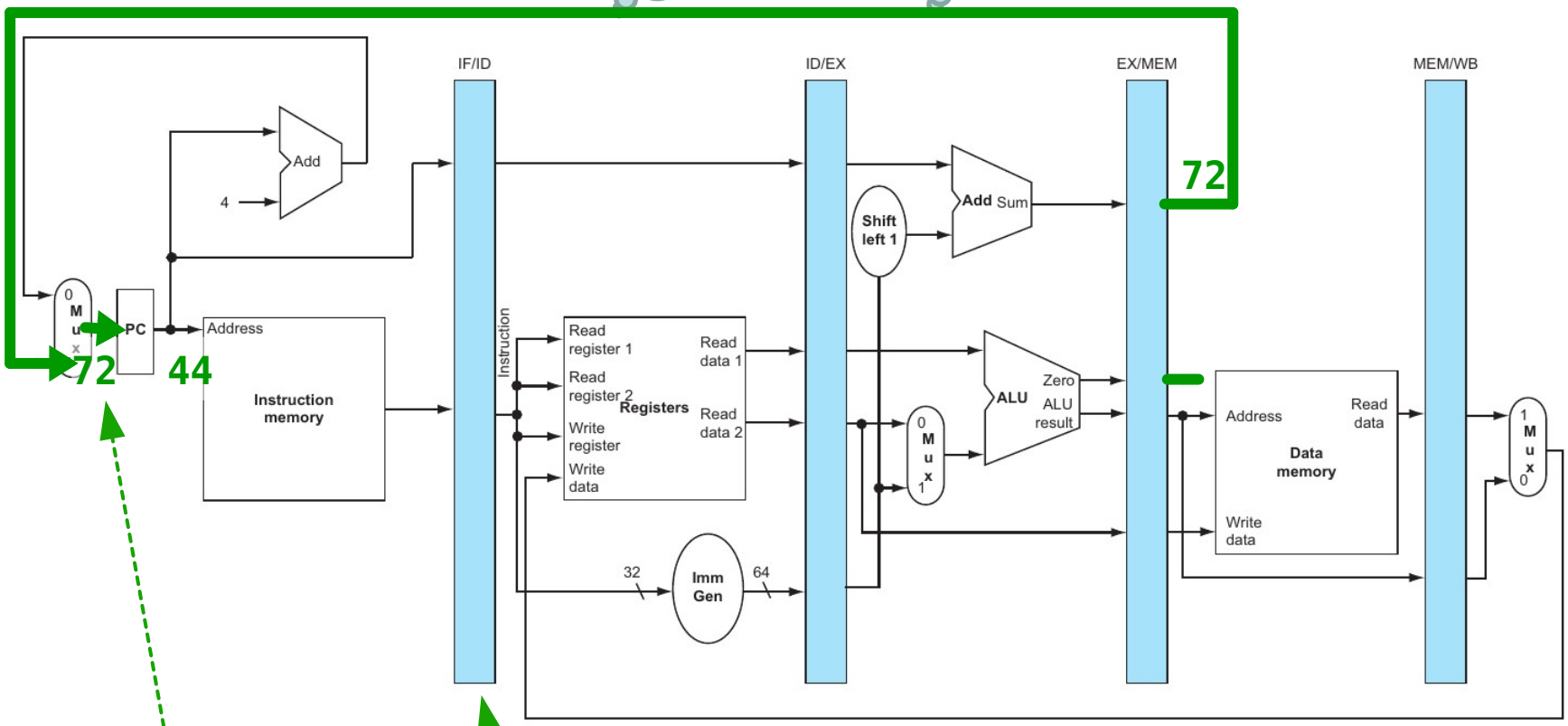
# Riesgos de control en el RISC-V
# Ejemplo 2 (VI): Bloqueos de control



PC=44

2º bloqueo

`and t12,t2,t5` `nop` `nop` `beq t1,t3,16`




En el próximo ciclo se cargará el valor 72 en el PC → se carga la instrucción destino del salto.

En el siguiente ciclo se **insertará** una `nop` en IF/ID

| | | | | | |
|---|---|---|---|---|---|
| `beq t1,t3,16` | IF | ID | EX | MEM | WB |
| `and t12,t2,t5` | | - | - | IF | IF |

# Riesgos de control en el RISC-V
# Ejemplo 2 (VII): Bloqueos de control



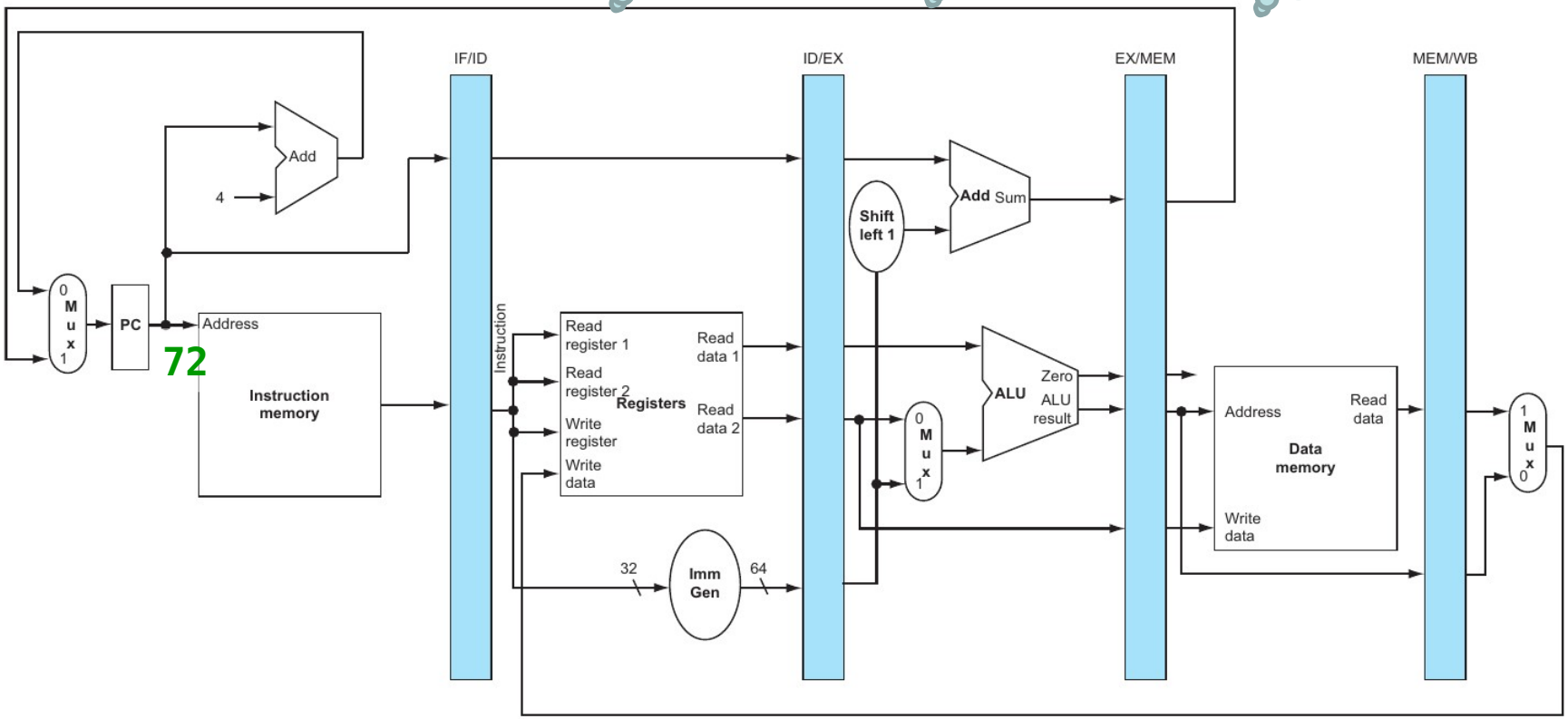
**Destino del salto: PC=72**

**lw t4,50(t7)**

**nop** **nop** **nop**

**3er bloqueo**

**beq t1,t3**




| | | | | | |
|---|---|---|---|---|---|
| **beq t1,t3,16** | **IF** | **ID** | **EX** | **MEM** | **WB** |
| **lw t4,50(t7)** | | **-** | **-** | **-** | **IF** |

La instrucción **and** es descartada y se carga **lw**

# Riesgos de control
# Predicción de salto



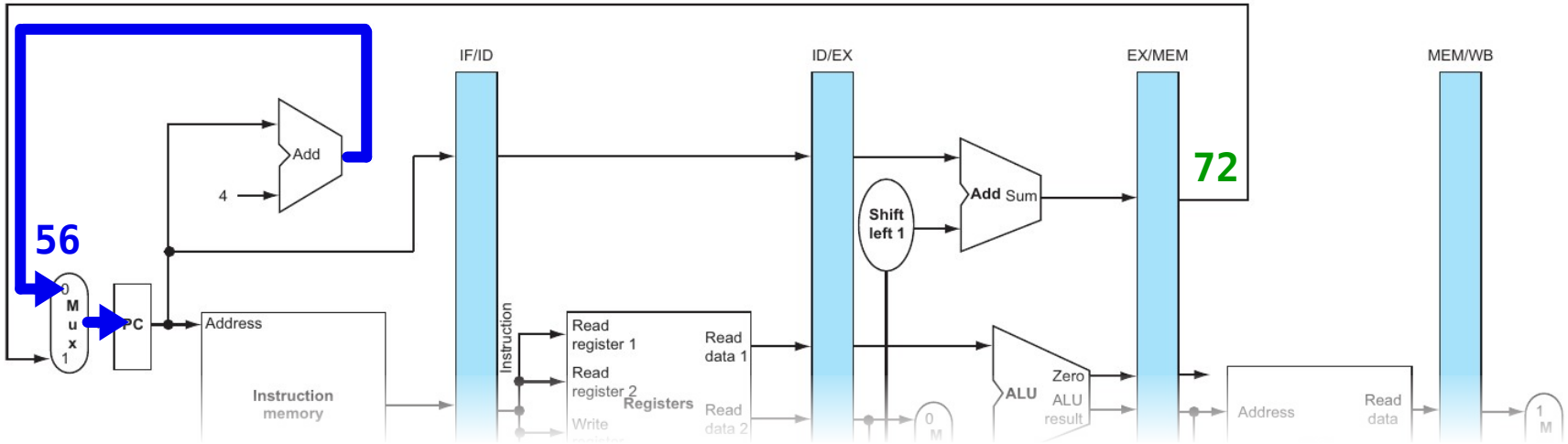
- Con la solución estudiada, las instrucciones de salto siempre producen 3 ciclos de bloqueo (independientemente de que el salto se tome o no).
- El rendimiento puede mejorarse con la técnica de predicción de salto.
  - Predicción de salto tomado:
    - Consiste en suponer que el salto se toma: la instrucción destino del salto se carga antes de evaluar la condición del salto.
    - Esta técnica no puede aplicarse en el camino de datos estudiado del RISC-V debido a que el cálculo de la dirección destino del salto y la evaluación de la condición se realizan en una etapa posterior a la decodificación.
  - Predicción de salto no tomado:
    - Consiste en suponer que el salto no se toma: las instrucciones siguientes al salto en memoria se cargan antes de evaluar la condición del salto.
      - Si el salto se toma finalmente, las instrucciones cargadas son descartadas (se producen 3 ciclos de bloqueo de control).
      - Si el salto no se toma, la ejecución continúa normalmente (no se producen ciclos de bloqueo de control).

# Predicción de salto en el RISC-V
# Ejemplo 2 (VIII): predicción de salto no tomado



PC=44 PC=40

```
and t12,t2,t5     beq t1,t3,16
```



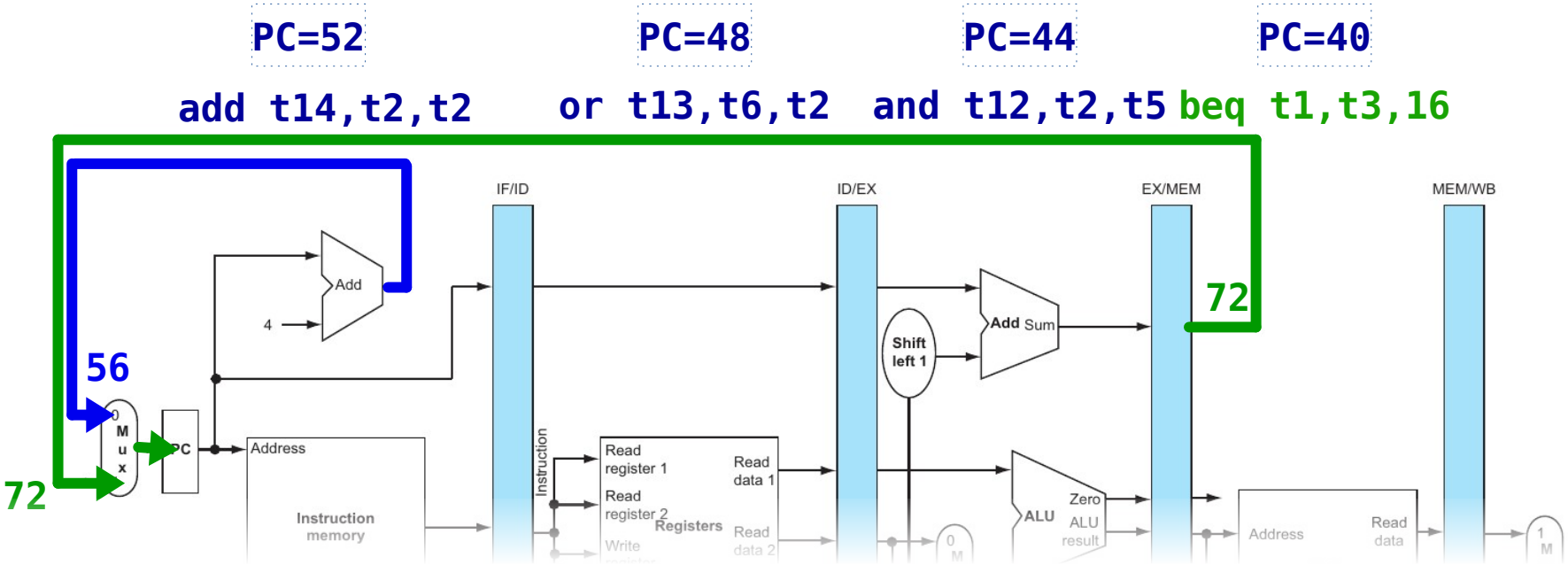

La unidad de control detecta la instrucción de salto y supone que el salto no será tomado: **predice salto no tomado**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| beq | IF | ID | EX | MEM | WB | | |
| and | | IF | ID | EX | MEM | WB | |
| or | | | IF | ID | EX | MEM | WB |
| add | | | | IF | ID | EX | MEM WB |

La ejecución continúa normalmente hasta que la condición del salto sea evaluada en la etapa MEM (se leerán dos intrucciones más)

# Predicción de salto en el RISC-V
# Ejemplo 2 (IX): predicción de salto no tomado






## SALTO TOMADO

Se carga el destino del salto (PC=72) y se descartan las tres instrucciones siguientes al salto

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| beq | IF | ID | EX | MEM | WB | | | | | |
| and | | IF | ID | EX | nop | nop | | | | |
| or | | | IF | ID | nop | nop | nop | | | |
| add | | | | IF | nop | nop | nop | nop | | |
| lw | | | | | IF | ID | EX | MEM | WB | |

3 bloqueos de control

## SALTO NO TOMADO

Se carga la instr. siguiente (PC=56) y se ejecutan todas las instrucciones.

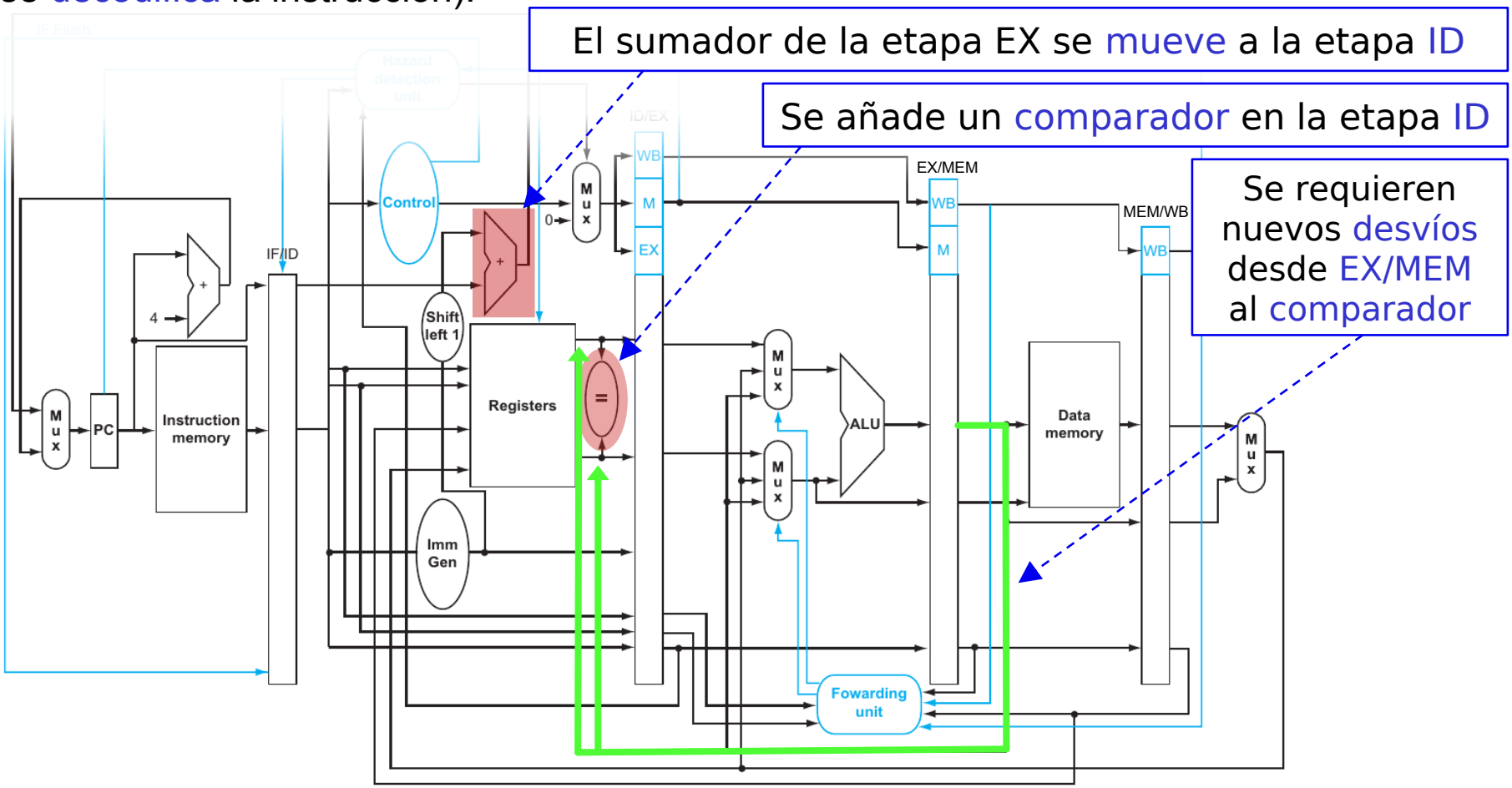
| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| beq | IF | ID | EX | MEM | WB | | | |
| and | | IF | ID | EX | MEM | WB | | |
| or | | | IF | ID | EX | MEM | WB | |
| add | | | | IF | ID | EX | MEM | WB |
| $instr_{PC=56}$ | | | | | IF | ID | ... | |

0 ciclos de bloqueo

# Riesgos de control
# Optimización del camino de datos del RISC-V (I)



- Un bloqueo de 3 ciclos es inadmisible desde el punto de vista del rendimiento. Para reducir la penalización por salto es necesario calcular lo antes posible la condición y la dirección destino del salto.
- En el RISC-V, la etapa más temprana en la que puede hacerse es la etapa ID (donde se decodifica la instrucción).

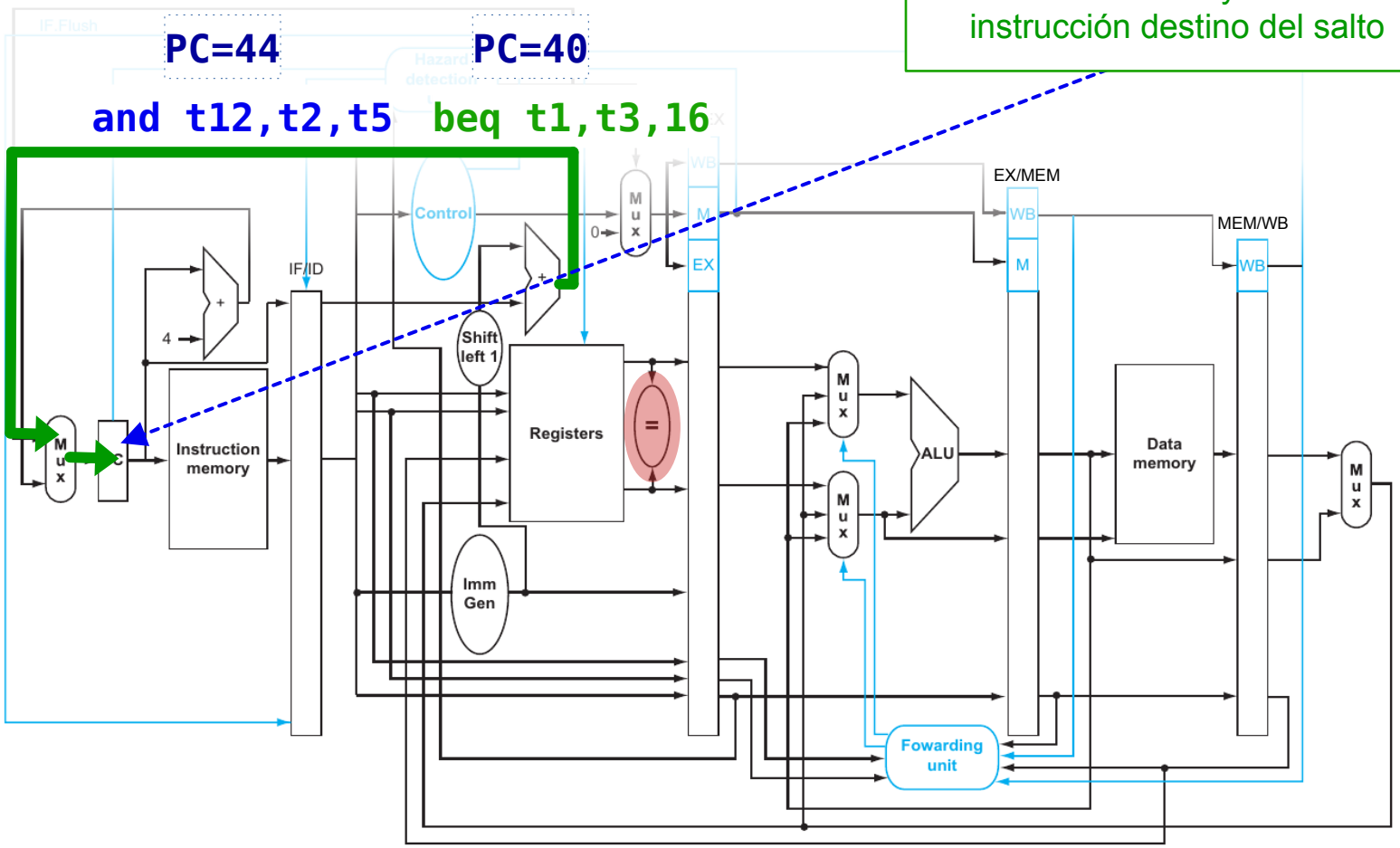


# Riesgos de control
# Optimización del camino de datos del RISC-V (II)



- Con el camino optimizado para saltos solo es necesario descartar una instrucción
- Continuación del Ejemplo 2

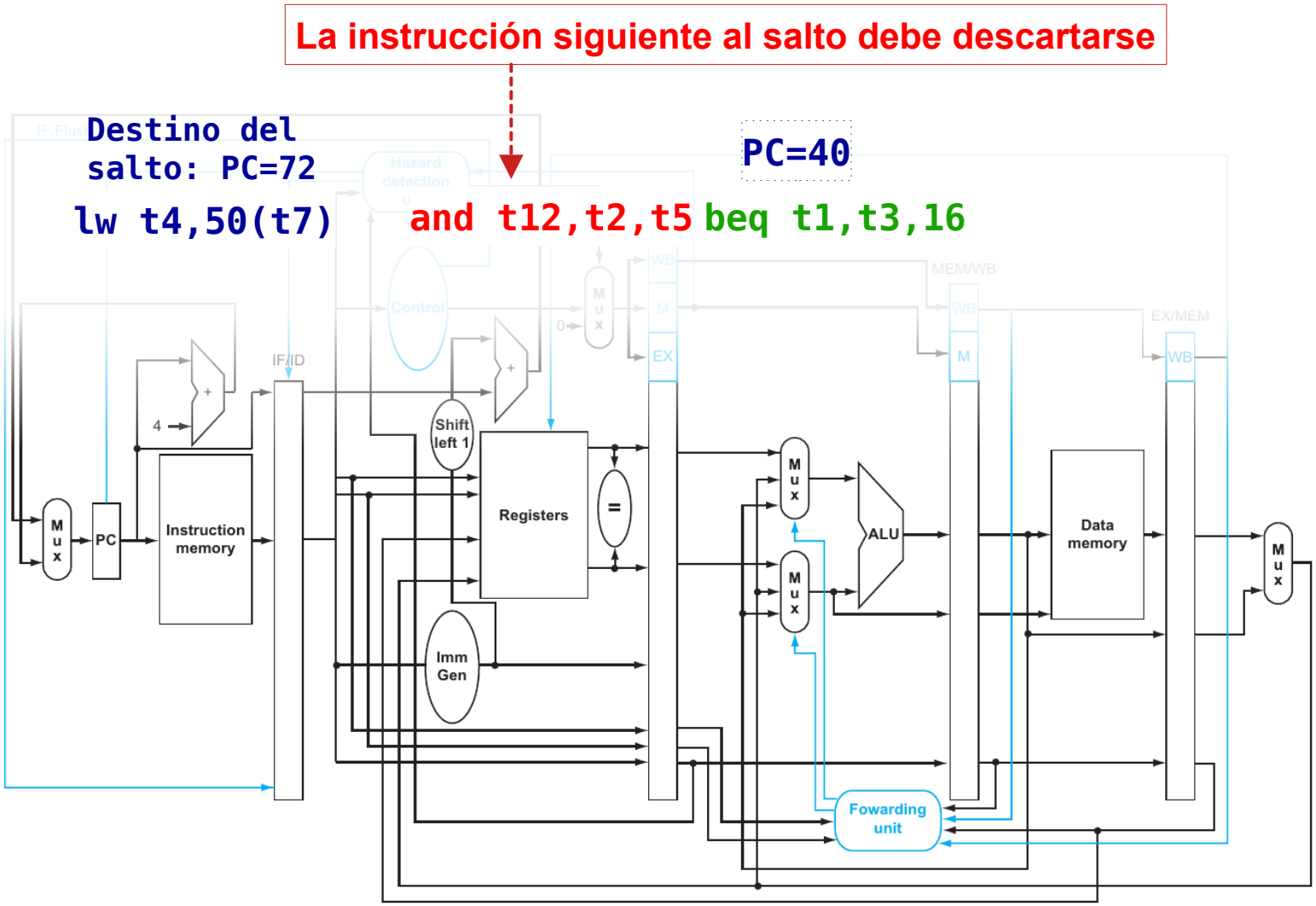
Si el salto es tomado, en el siguiente ciclo, se cargará el valor 72 en el PC y se leerá la instrucción destino del salto



# Riesgos de control
# Optimización del camino de datos del RISC-V (III)



- Con el camino optimizado para saltos solo es necesario descartar una instrucción
- Continuación del Ejemplo 2



# Optimización del camino de datos del RISC-V y predicción de salto



- Bloqueos de control en el RISC-V con el camino de datos optimizado y predicción de salto:

  - Ejemplo: **Salto no tomado** (la instrucción siguiente se ejecuta normalmente)

```
beq t1, t2, 16        IF  ID  EX  MEM WB
or t7, t8, t9             IF  ID  EX  MEM WB
```

0 ciclos de bloqueo

  - Ejemplo: **Salto tomado** (la instrucción siguiente se descarta)

```
beq t1, t2, 16        IF  ID  EX  MEM WB
or t7, t8, t9             IF  nop nop nop nop
…
lw t3, 300(t0)            IF  ID  EX  MEM WB
```

1 ciclo de bloqueo

Destino del salto

La instrucción siguiente al salto se descarta y se introduce una instrucción NOP en IF/ID

# Planificación de instrucciones
## Planificación estática vs. planificación dinámica



- **Planificación de instrucciones**: Conjunto de técnicas para reducir el número de bloqueos en procesadores segmentados mediante la modificación del orden de ejecución de las instrucciones.
- Existen dos enfoques distintos:
  - Planificación estática: Realizada por el compilador (en tiempo de compilación).
  - Planificación dinámica: Realizada por el procesador (en tiempo de ejecución).
- Ventajas de la planificación dinámica:
  - Permite tratar dependencias de datos desconocidas en tiempo de compilación.
    - Por ejemplo, las dependencias entre operandos en memoria como las que se muestra en el siguiente código:

```
sw $t4, 12($t3)
lw $t2, 100($t1)
```

No se puede detectar en tiempo de compilación si son el mismo operando, ya que no es posible saber si 12+$t3=100+$t1

  - Se gana en portabilidad (véase procesadores VLIW vs. superescalares más adelante).
  - Se simplifica el compilador.
- Inconvenientes de la planificación dinámica:
  - Aumenta la complejidad hardware.

- Véase el Ejercicio 1.6 (a)

# Estrategias para aumentar las prestaciones con ILP



- Básicamente, existen dos estrategias para incrementar las prestaciones obtenidas con ILP:
  - **Aumentar la profundidad del *pipeline*** (*superpipelines*): permite reducir la duración del ciclo del procesador, pero el CPI sigue siendo mayor o igual que 1:
    - En cada ciclo se completan como máximo una instrucción.
  - **Ejecución múltiple de instrucciones**: Permite ejecutar más de una instrucción en cada ciclo del procesador (CPI < 1).

# Estrategias para aumentar las prestaciones con ILP
# Aumentar la profundidad del *pipeline (superpipelines)*



- En condiciones ideales, el rendimiento del procesador segmentado (medido como el número de instrucciones ejecutadas por segundo) es aproximadamente igual a la inversa del tiempo de ciclo del procesador (T):

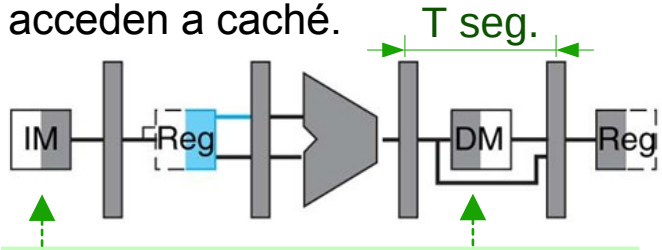
$$\text{rendimiento} \approx \frac{1}{T} \text{ instrucciones/seg.}$$

- Aumentar la profundidad del pipeline permite reducir el tiempo de ciclo; por lo tanto, permite incrementar las prestaciones obtenidas con ILP.
  - Véase el ejemplo de la diapositiva siguiente.
- En esta estrategia, el CPI sigue siendo mayor o igual que 1; es decir, en cada ciclo se completa una instrucción como máximo ($CPI_{ideal}$ = 1).

# Estrategias para aumentar las prestaciones con ILP
# Aumentar la profundidad del *pipeline*: Ejemplo



- Supongamos que en el camino de datos del RISC-V las etapas más lentas son las que acceden a caché.

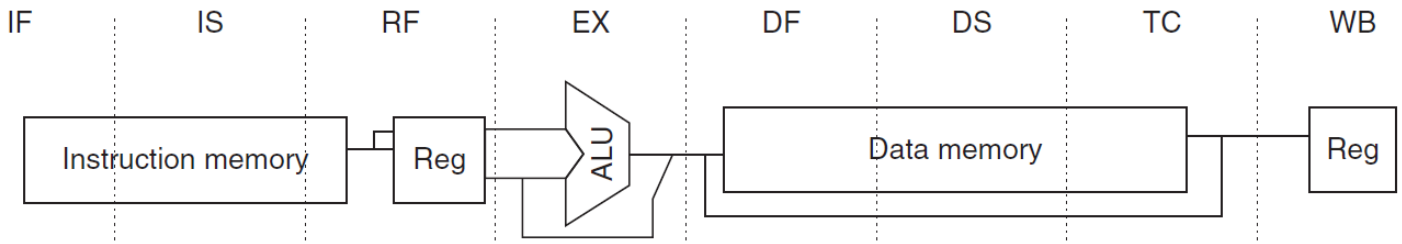



Etapas más lentas: determinan el tiempo de ciclo (T)

Rendimiento ≈ 1 instr. / T seg

Si se segmentan las etapas que acceden a caché, se puede reducir el tiempo de ciclo y aumentar el rendimiento.

El MIPS R4000 dispone de **8 etapas** en vez de 5, al segmentar el acceso a las cachés de instrucciones y datos (que son de **correspondencia directa**). La búsqueda en caché se divide en **tres fases**:




- Caché de instrucciones
  - **IF: Primera parte de la búsqueda** de la **etiqueta** candidata.
  - **IS: Segunda parte de la búsqueda** de la **etiqueta** candidata.
  - **RF: Comparación de etiquetas,** decodifición de la instrucción y **lectura** del banco de **registros**.
- Caché de datos
  - **DF: Primera parte de la búsqueda** de la **etiqueta** candidata.
  - **DS: Segunda parte de la búsqueda** de la **etiqueta** candidata.
  - **TC**: **Las etiquetas se comparan**.

# Estrategias para aumentar las prestaciones con ILP
# Ejecución múltiple de instrucciones (I)



- **Ejecución múltiple o emisión múltiple** (*multiple issue*): Consiste en replicar unidades funcionales para permitir la ejecución de múltiples instrucciones dentro de cada etapa de segmentación.
  - Los procesadores de ejecución múltiple disponen de caminos de datos independientes con sus propias unidades funcionales, denominados **ranuras de ejecución** (*issue slots*).
  - Habitualmente, las ranuras de ejecución **no** tienen todas las unidades funcionales necesarias para ejecutar cualquier tipo de instrucción.
    - Se establecen restricciones sobre las combinaciones de instrucciones que pueden ser emitidas en el mismo ciclo (es decir, que pueden comenzar su ejecución en el mismo ciclo).
- $$\boxed{\mathrm{CPI}_{\mathrm{ideal}} = \frac{1}{\text{número de ranuras de ejecución}}}$$ → El CPI puede ser inferior a 1
  - Se ejecutan más de 1 instrucción en cada ciclo del procesador.
    - Es habitual invertir la métrica del CPI y hablar del número de instrucciones ejecutadas por ciclo de reloj o IPC (*instructions per clock cycle*).

# Estrategias para aumentar las prestaciones con ILP
# Ejecución múltiple de instrucciones (II)



- Existen dos aspectos principales que deben gestionarse en un procesador de emisión múltiple:
  - Empaquetar las instrucciones en ranuras de ejecución: Consiste en determinar qué instrucciones van a emitirse en cada ciclo.
    - El conjunto de instrucciones que comienzan su ejecución en un determinado ciclo se denomina **paquete de ejecución** (*issue packet*).
  - Gestionar los riesgos de datos y de control.
- Según la forma de dividir el trabajo entre el compilador y el procesador, podemos tener:
  - **Procesadores de ejecución múltiple con planificación estática** (*static multiple issue*): Las decisiones son tomadas por el compilador (en tiempo de compilación).
  - **Procesadores de ejecución múltiple con planificación dinámica** (*dynamic multiple issue*): Las decisiones son tomadas por el procesador (en tiempo de ejecución).

# Procesadores de ejecución múltiple con planificación estática: procesadores VLIW (I)



- En estos procesadores, el compilador planifica la ejecución de las instrucciones:
  - Decide qué instrucciones se emiten en cada paquete de ejecución teniendo en cuenta las características de las ranuras
  - Evita los riesgos entre las instrucciones de un mismo paquete de ejecución.
    - Por ejemplo, no empaqueta juntas instrucciones con dependencias RAW.
- Existen dos tipos alternativas de implementación:
  - En algunos diseños, el procesador detecta los riesgos de datos entre los diferentes paquetes de ejecución y los riesgos de control, y genera los bloqueos o los desvíos necesarios.
  - En otros, el procesador no detecta ningún riesgo ni puede bloquear el *pipeline*.
    - El compilador asume la responsabilidad de eliminar todos los riesgos reordenando las instrucciones o insertando paquetes vacíos (formados solo por instrucciones *nop*).

# Procesadores de ejecución múltiple con planificación estática: procesadores VLIW (II)



- El código binario generado por el compilador está formado por paquetes de ejecución que serán leídos y ejecutados por el procesador.
  - Cada paquete puede verse como una única instrucción larga con múltiples operaciones, de ahí que estos procesadores se conozcan con el nombre de VLIW (Very Long Instruction Word).
    - La arquitectura IA-64 de Intel (procesadores Itanium) está basada en esta idea aunque con características más avanzadas.
  - La decodificación es sencilla porque las instrucciones ocupan posiciones fijas dentro del paquete.
  - Si no es posible empaquetar todas las instrucciones que permiten las ranuras, se colocan instrucciones *nop* para completar el paquete.
- Inconveniente:
  - El código generado no es portable a procesadores con distintas ranuras de ejecución aunque tengan el mismo repertorio de instrucciones.

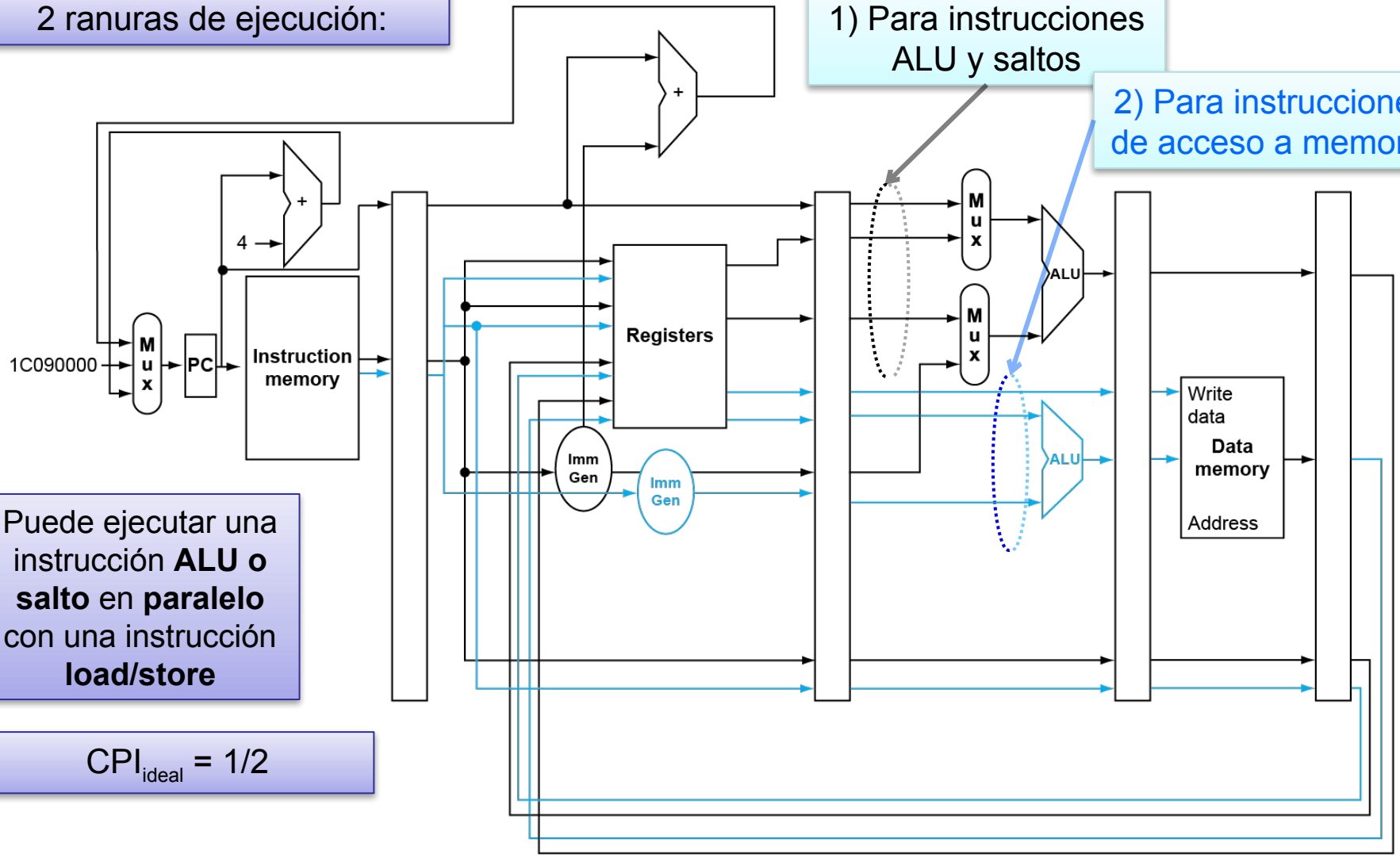
# Ejemplo de RISC-V VLIW
# Camino de datos



2 ranuras de ejecución:

1) Para instrucciones ALU y saltos

2) Para instrucciones de acceso a memoria




Puede ejecutar una instrucción **ALU o salto** en **paralelo** con una instrucción **load/store**

$CPI_{ideal} = 1/2$

- Véase el Ejercicio 1.6 (b)

# Procesadores de ejecución múltiple con planificación dinámica: superescalares



- Los procesadores de ejecución múltiple con planificación dinámica se conocen también como superescalares.
- El procesador:
  - decide qué instrucciones se emiten en el mismo ciclo
  - y gestiona los riesgos de datos y de control.

- Ventaja:
  - El hardware garantiza que el código se ejecutará correctamente (tanto si ha sido planificado por el compilador como si no).
    - A diferencia de los VLIW, los superescalares son compatibles con cualquier programa que utilice su juego de instrucciones.

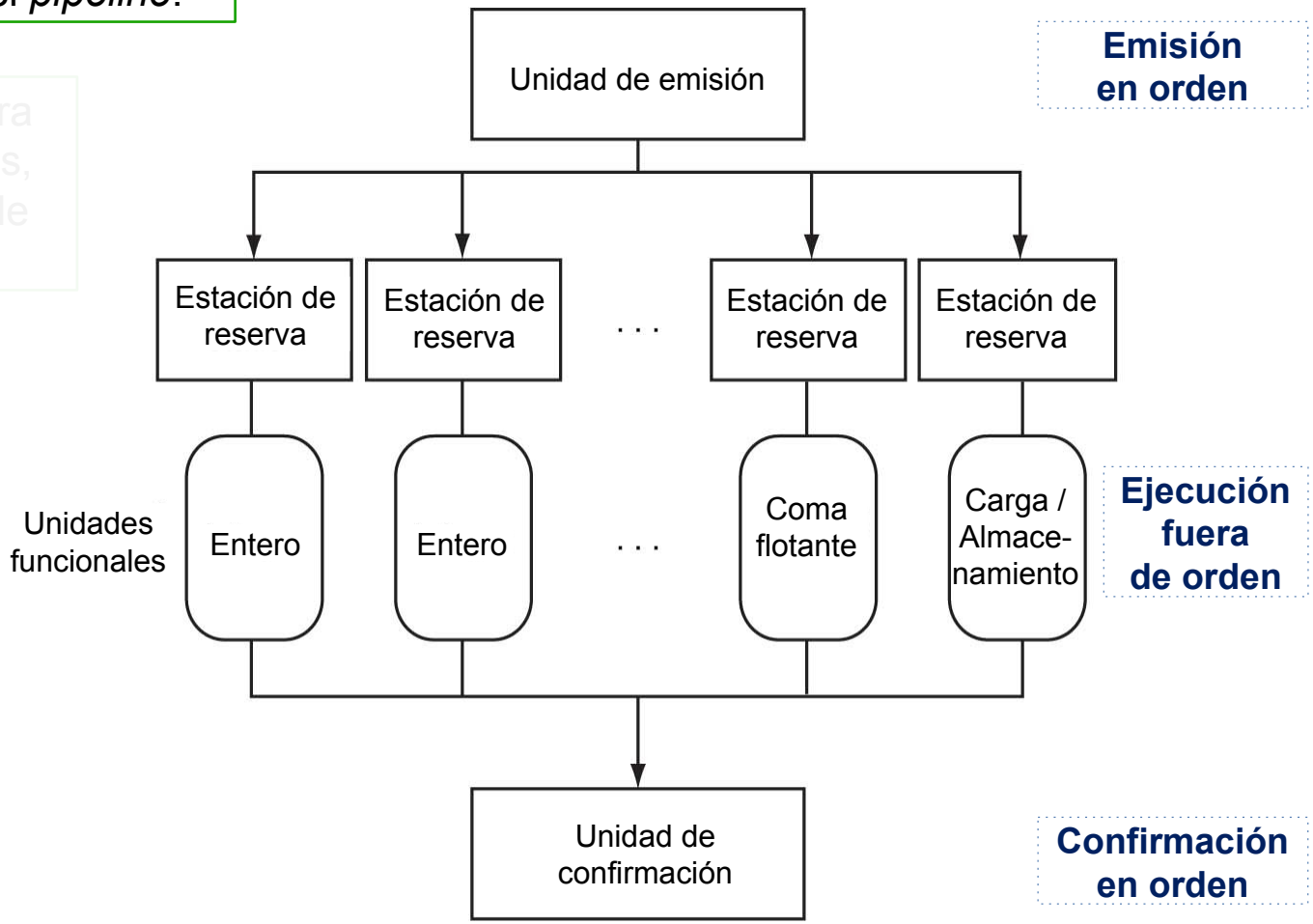
- Existe dos tipos de superescalares:
  - **Superescalares sin planificación dinámica del *pipeline***:
    - Las instrucciones se ejecutan en el mismo orden que aparece en el programa.
  - **Superescalares con planificación dinámica del *pipeline***:
    - Para evitar bloqueos, el procesador decide en qué orden se ejecutan las instrucciones.

# Procesadores superescalares con planificación dinámica del pipeline (I)



El diagrama muestra los elementos principales de un superescalar con planificación dinámica del *pipeline*.

Ranuras de ejecución para instrucciones ALU enteras, ALU en coma flotante y de carga/almacenamiento

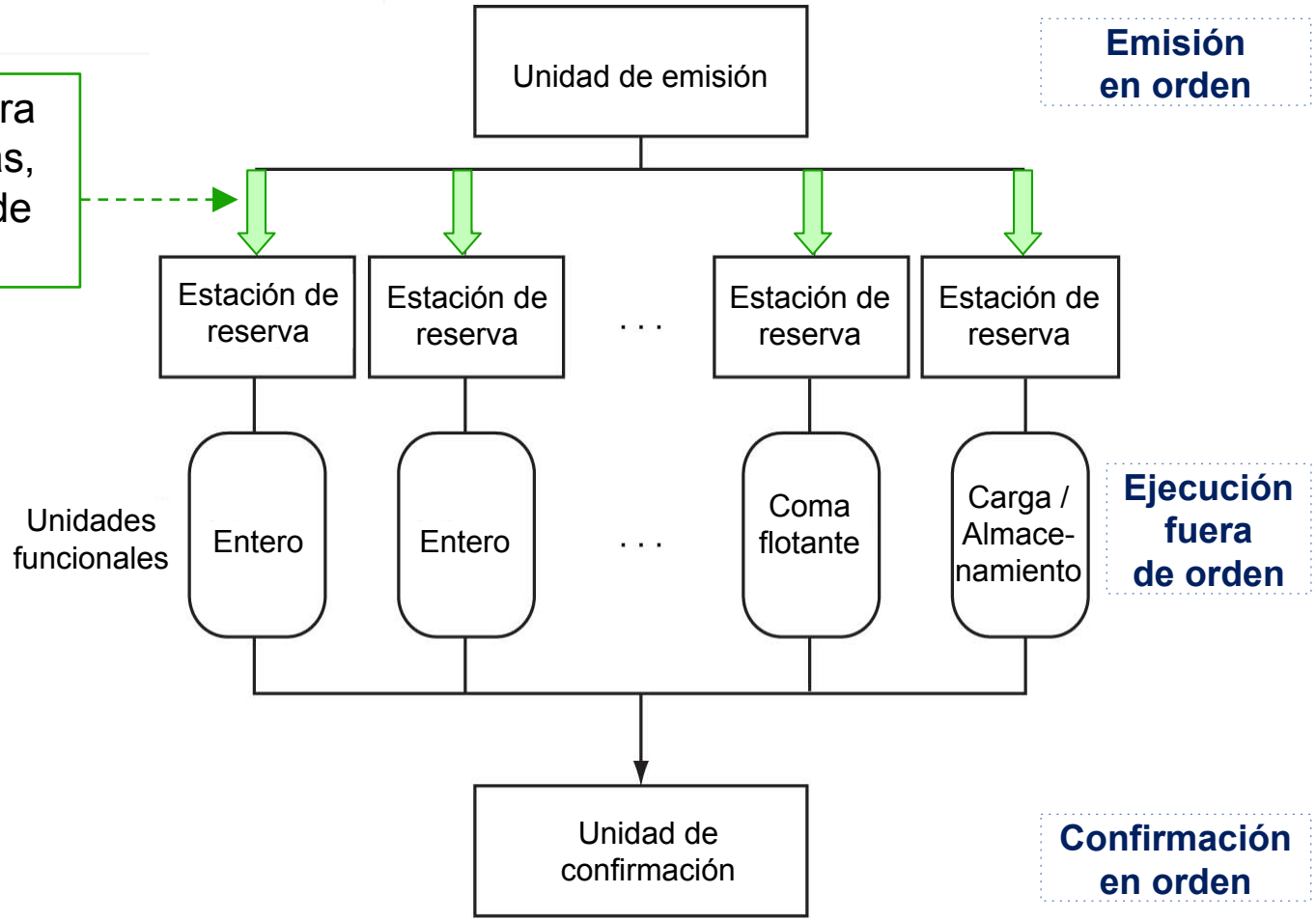


# Procesadores superescalares con planificación dinámica del pipeline (II)



**Unidad de emisión** (*issue unit*): Lleva a cabo la búsqueda y decodificación de instrucciones y envía cada instrucción a la estación de reserva correspondiente.

Ranuras de ejecución para instrucciones ALU enteras, ALU en coma flotante y de carga/almacenamiento

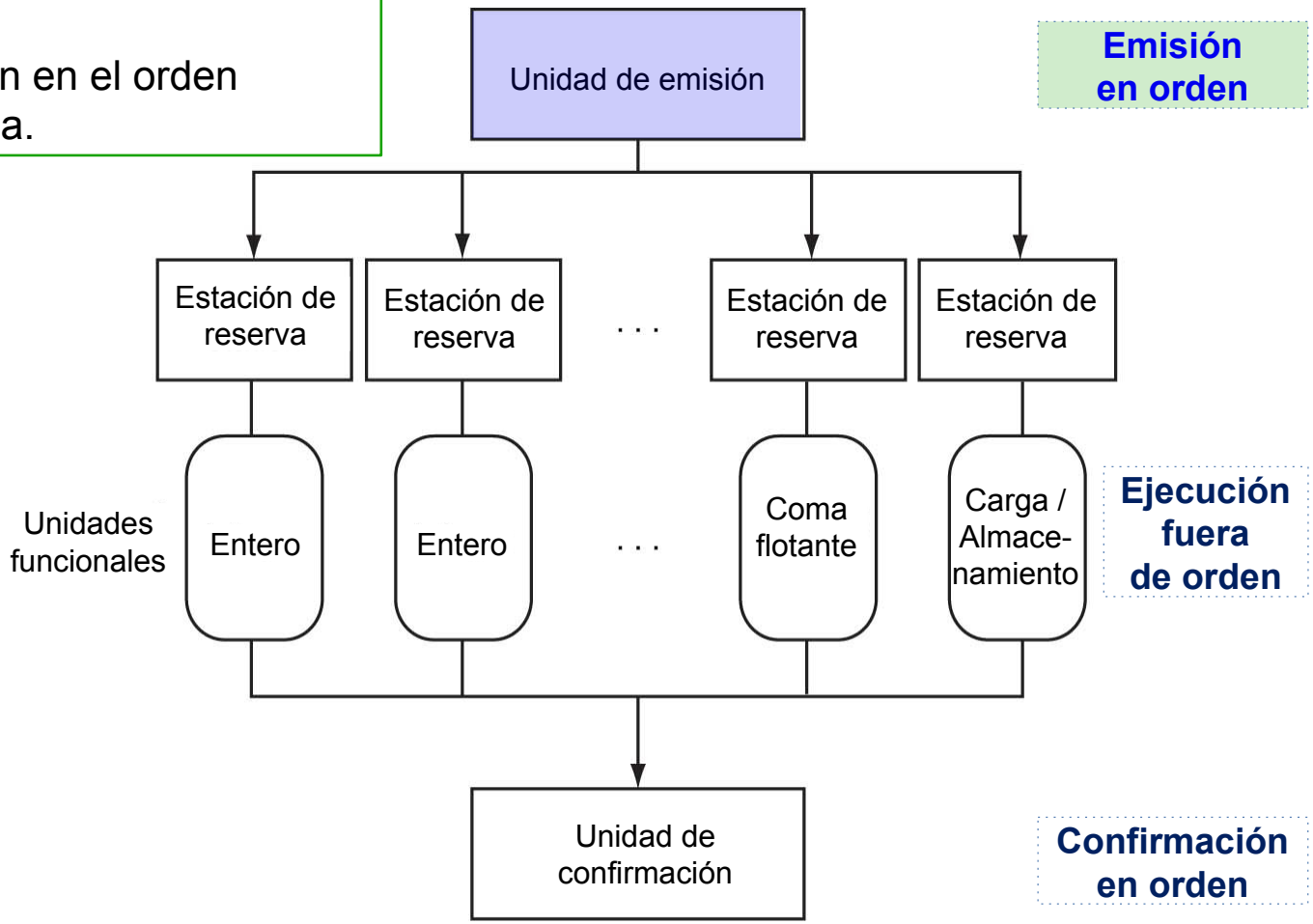


# Procesadores superescalares con planificación dinámica del pipeline (III)



**Unidad de emisión** (*issue unit*): Lleva a cabo la búsqueda y decodificación de instrucciones y envía cada instrucción a la estación de reserva correspondiente.

Las instrucciones se emiten en el orden establecido por el programa.



# Procesadores superescalares con planificación dinámica del pipeline (IV)

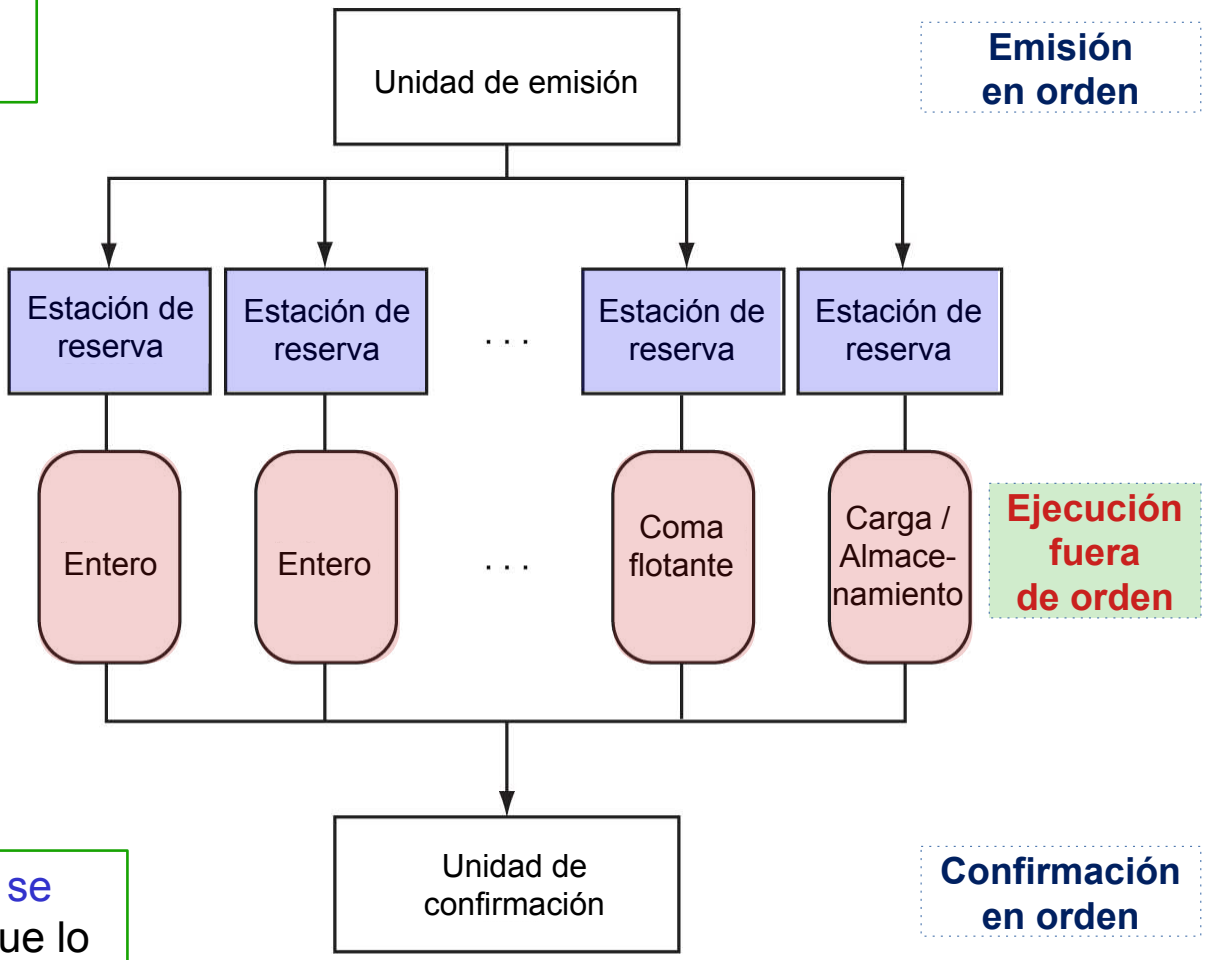


**Estación de reserva** (*reservation station*): Búfer que almacena la operación y los operandos de las instrucciones que se van a ejecutar en la unidad funcional.

Una instrucción se ejecuta cuando la unidad funcional esté preparada y todos sus operandos estén disponibles.

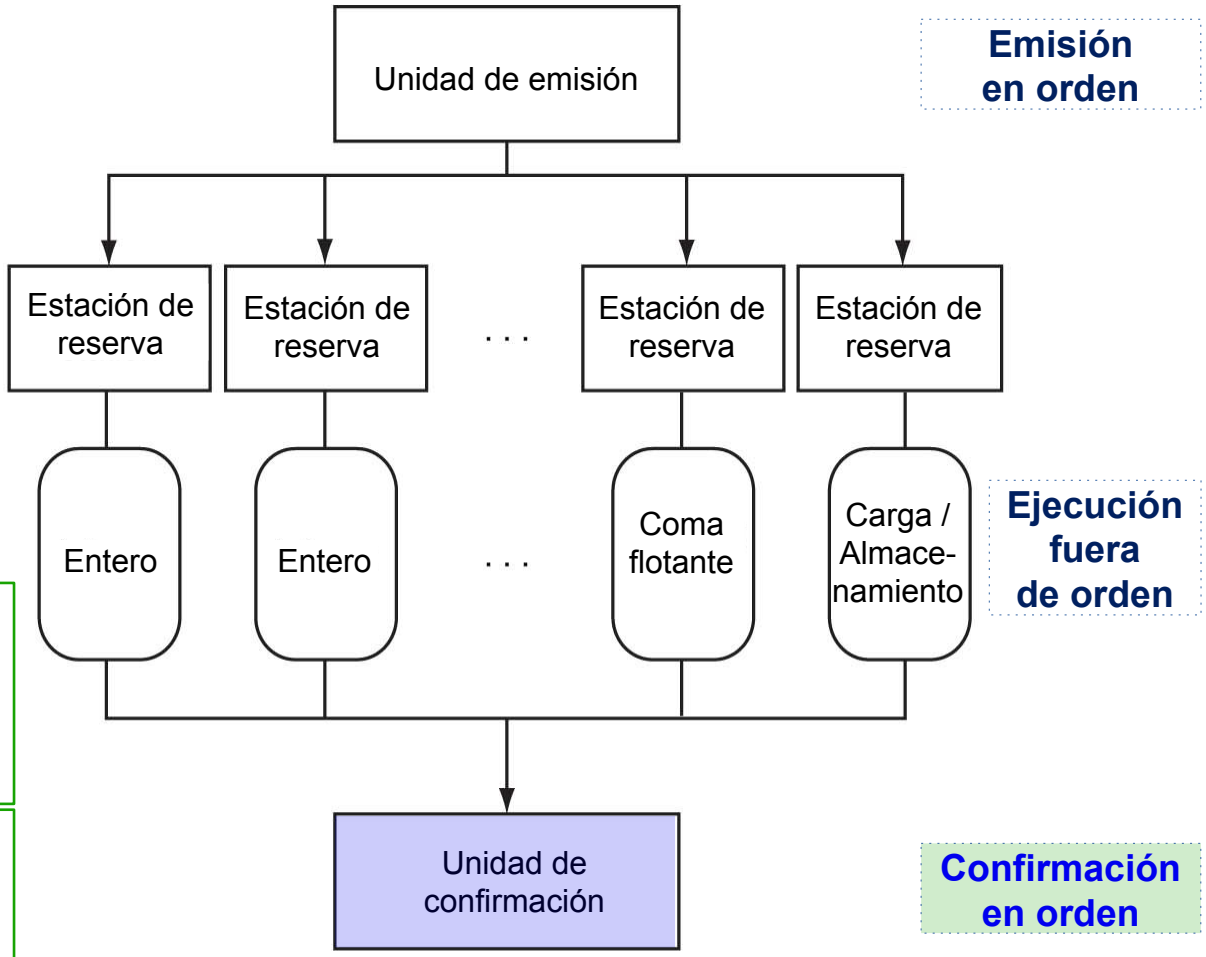
La primera instrucción que cumpla los requisitos es la que se ejecuta primero, independientemente del orden en que aparecen en el código.

Una vez obtenido el resultado, este se envía a las estaciones de reserva que lo necesitan y a la unidad de confirmación.



# Procesadores superescalares con planificación dinámica del pipeline (V)






**Unidad de confirmación** (*commit unit*): Guarda los resultados hasta sea seguro escribirlos en el banco de registro o en memoria.

El búfer donde almacena los resultados antes de ser escritos se denomina búfer de reordenación (*reorder buffer*).

# Ejecución multihilo en hardware (I)



- El rendimiento de los procesadores de emisión múltiple está limitado por los bloqueos debidos a dependencias de datos, riesgos de control o fallos de caché.
- Los hilos ofrecen una oportunidad para aumentar el rendimiento:
  - Comparten los recursos del proceso:
    - Cada hilo solo necesita su propio estado de ejecución: su propio contador de programa, pila de ejecución y estado de la CPU (registros).
  - El cambio de hilo puede hacerse en hardware: el procesador solo necesita almacenar el estado de cada hilo; p. ej., manteniendo una copia del banco de registro y del PC para cada hilo.
    - El cambio de procesos es más complejo y no puede realizarse fácilmente en hardware (es gestionado por el SO).
    - El cambio de proceso requiere de cientos a miles de ciclos, mientras que el cambio de hilo puede ser instantáneo.
  - No existen dependencias de datos entre las instrucciones de diferentes hilos.
    - No se producen bloqueos de datos entre las instrucciones de diferentes hilos.
  - Cuando un hilo se bloquea el procesador puede conmutar a otro hilo diferente.
- La ejecución multihilo en hardware (*hardware mulithreading*) permite que varios hilos compartan unidades funcionales y se ejecuten simultáneamente.

# Ejecución multihilo en hardware (II)



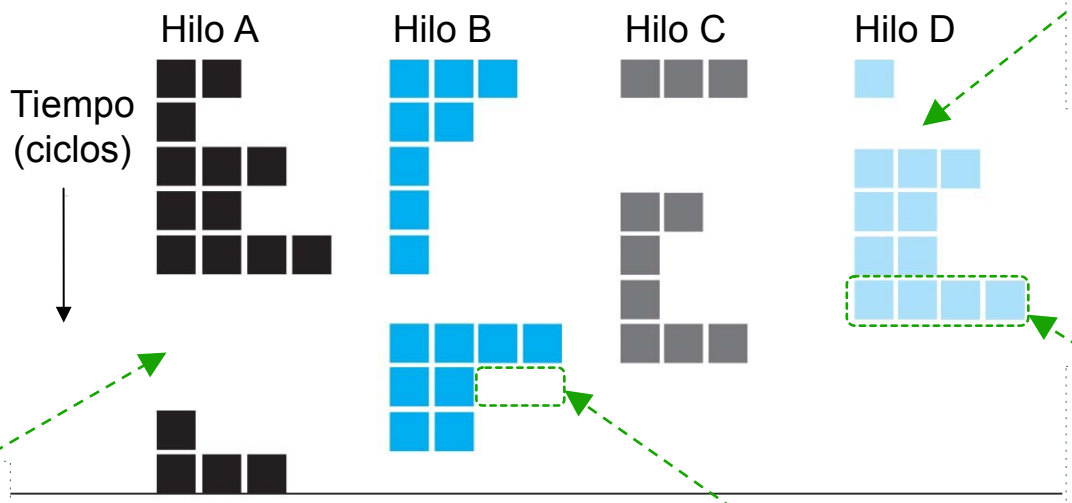
Existen tres alternativas de ejecución multihilo en hardware:

- **Ejecución multihilo de grano fino** (*fine-grained mulithreading*): Se conmuta entre hilos después de cada ciclo de reloj.
  - Ejemplo: Los procesadores SPARC T1 a T5 y las GPU de NVIDIA implementan la ejecución multihilo de grano fino.
- **Ejecución multihilo de grano grueso** (*coarse-grained mulithreading*): Se conmuta entre hilos solo después de un suceso importante (p. ej., un fallo de caché).
- **Ejecución multihilo simultánea o SMT** (*simultaneous multithreading*): Se utilizan los recursos de un superescalar con planificación dinámica del pipeline para explotar el paralelismo a nivel de hilo.
  - Los SMT no conmutan entre hilos, sino que están siempre ejecutando instrucciones de varios hilos, siendo el hardware el que determina qué instrucciones se ejecutan en un mismo ciclo → pueden ejecutar instrucciones de diferentes hilos en el mismo ciclo.
  - Los procesadores con emisión múltiple tienen a menudo más unidades funcionales que las que puede utilizar de forma efectiva un único hilo.
    - Los SMT permiten un uso más intensivo de las unidades funcionales.
  - Ejemplo: Procesadores Intel i7 920 y 6700 implementan SMT con 2 hilos por núcleo.
    - La aceleración media obtenida con SMT respecto a un núcleo sin ejecución multihilo es del 30% (pruebas realizadas con los benchmarks PARSEC y DaCAP/SPEC)

# Ejecución multihilo en hardware (III)



Ejecución de 4 hilos en un procesador sin soporte para ejecución multihilo (la conmutación la hace el SO)




**Bloqueo corto** (p. ej., por riesgo de datos).

**Bloqueo largo** (p. ej., por fallo de caché)

La CPU tiene 4 ranuras de ejecución

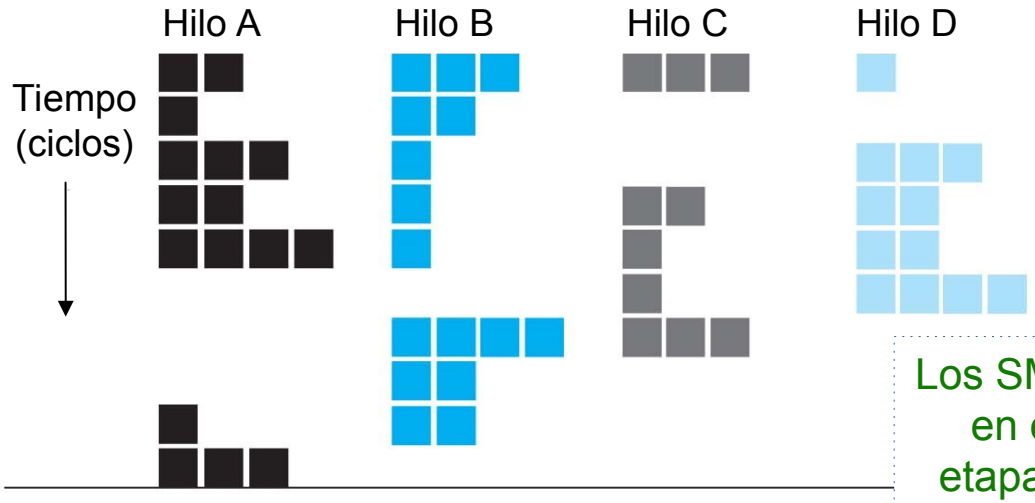
**Ranuras** de ejecución **sin aprovechar** (por **características** de las **ranuras** o **riesgos** de datos).

Procesador con soporte para ejecución multihilo (la conmutación la hace el hardware)

# Ejecución multihilo en hardware (IV)

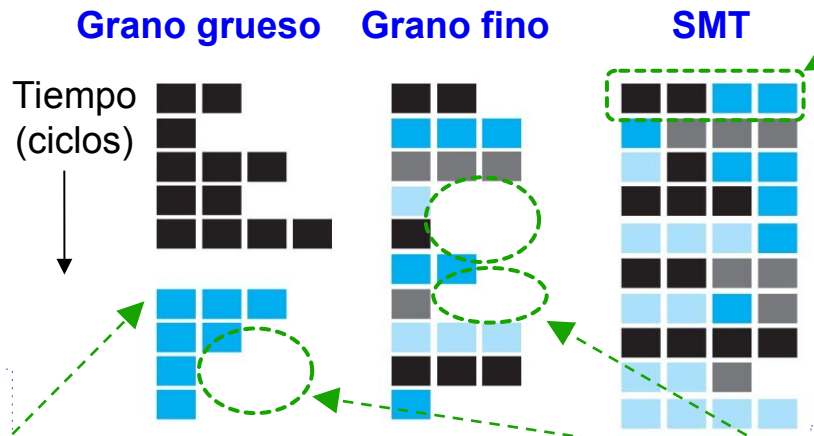


Ejecución de 4 hilos en un procesador sin soporte para ejecución multihilo (la conmutación la hace el SO)




Los SMT pueden ejecutar en el mismo ciclo (o etapa) instrucciones de diferentes hilos

Procesador con soporte para ejecución multihilo (la conmutación la hace el hardware)




SMT es la técnica que aprovecha mejor las ranuras de ejecución. Por lo tanto, es la que consigue ejecutar mayor núm. de instr.

Cambio de hilo solo en bloqueos largos.

Las ranuras de ejecución no se aprovechan completamente

Cambio de hilo en cada ciclo (aprovecha los bloqueos cortos y largos)